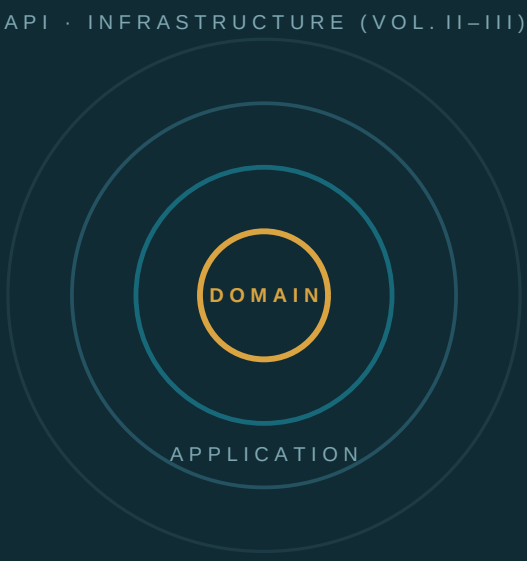
NEXUS-1 COMPANION SERIES

THE BACKEND TRILOGY · VOLUME I



From Blueprint to Core

Building the NEXUS-1 Backend in .NET

The Architecture, Domain, and Application Layers of the NEXUS-1 Digital Twin

"Nothing claims to exist that does not."

CLEAN / ONION ARCHITECTURE · DDD IN C# · CQRS · 17 BOUNDED CONTEXTS · PROVEN WITHOUT A DATABASE

GRIGORIOS AGATHANGELIDIS

A NEXUS-1 Companion · Volume I of the Backend Trilogy · working draft · 2026

Standing Boundary and Copyright Note

Front Matter

From Blueprint to Core · Building the NEXUS-1 Backend in .NET · NEXUS-1 Companion Series · Backend Trilogy, Volume I. Demonstration build NX1-094C79E0A366D34F.

This book is the first implementation volume of the NEXUS-1 backend and a companion to the eight published NEXUS-1 volumes: *From Grid to Core*, *From Table to Twin*, *From Entity to Context*, *From Schema to System*, *From Domain to Twin*, *From Flood to Cause*, *From Trial to Policy*, and *From Certainty to Calibration*. It translates the domain model those volumes defined into a compiling, unit-tested C# solution.

STANDING BOUNDARY

This volume is a software architecture and learning companion. It is not safety-class software, not certified operational guidance, and not connected to any real facility. NEXUS-1 remains a Phase-0 demonstrator: useful for learning architecture, language, modelling, and boundaries — never for operating a plant.

STANDING RULE

Nothing in this book claims to exist that does not. Every project compiles, every class shown is written in full, and every invariant demonstrated can be violated in a test and watched to fail. Where an abstraction is declared whose implementation is deferred to Volume II, the deferral is stated in plain sight — that seam is part of the design, not an omission.

SCOPE OF THIS VOLUME

Volume I covers everything inside the persistence boundary: the Clean/Onion solution structure, the SharedKernel, the Domain layer, and the Application layer with CQRS — provable with nothing installed but the .NET SDK. Persistence, REST APIs, and security belong to Volume II (*From Core to Contract*); integration testing, logging, configuration, and Docker belong to Volume III (*From Contract to Container*).

A NOTE ON THIS EDITION

This is a working draft. Sources and reference notes, glossary, and index will be added with the appendices, matching the final editions of the series. The contents page that follows is page-aware for the completed body; back-matter rows keep their dashes until those pages exist, per the standing rule.

Contents

Page-aware for the completed body, per the standing rule — these pages now exist. Back-matter rows keep their dashes until theirs do. A two-page Deep Dive companion follows each chapter.

Preface

Eight books into the NEXUS-1 series, a developer who has read all of them still cannot do the one thing developers actually do: open a solution. *From Domain to Twin* drew the domain model — the seventeen bounded contexts, the ubiquitous language, the aggregates and their invariants. *From Schema to System* and *From Entity to Context* wrote down the 654-table data backbone those models imply, column by column. *From Grid to Core*, *From Flood to Cause*, and *From Trial to Policy* built the engines — the physics, the root-cause reasoning, the Q-tables — that give every one of those tables its meaning. What none of them shows is the C# in between: the Nexus.sln a developer double-clicks, the folder that holds *RootCauseCase*, the test that proves a case cannot close without evidence. This book is that solution.

It was very nearly a different book. The original plan was a single implementation volume — twenty-five chapters from Clean Architecture to Docker, an estimated five to six hundred pages. Then the calibration lesson of *From Certainty to Calibration* was applied to the plan itself, and the plan failed its own audit: one volume carrying seven architectural concerns at NEXUS-1 scale would either stay superficial everywhere or collapse under its own weight. The backend is therefore a trilogy, and the trilogy mirrors the Onion it describes, from the inside out. This volume builds the core — architecture, Domain, Application. Volume II, *From Core to Contract*, wraps it in persistence and a REST API. Volume III, *From Contract to Container*, proves and ships it. Splitting a book is not a retreat; it is the same discipline the series applies to software, applied to the writing of it.

The boundary of Volume I is drawn where the Dependency Rule draws it: everything inside the persistence boundary, nothing outside it. That boundary has a consequence worth stating on the first page, because it shapes every chapter — **no database is required to run or test a single line of this volume.** Not as a testing convenience, but as proof. If the Domain and Application layers genuinely depend on nothing outside themselves, then they must compile, run, and pass every unit test with nothing installed but the .NET SDK — and they do. Where the Application layer needs the outside world, it declares an interface — *IRepository*, *IUnitOfWork* — and stops. Volume II implements those interfaces against the exact tables of *From Schema to System*. The seam between the volumes is the seam in the architecture.

The book keeps the habits of the series. Every chapter ends with a checkpoint, in the school-style tradition. Every class shown is written in full, not sketched. Every invariant is demonstrated twice: once holding, and once deliberately violated in a test and watched to fail, because an invariant nobody has watched fail is a claim, not a guarantee. And every honest boundary is printed where it applies — what this volume defers, it defers by name, with the volume that will pick it up.

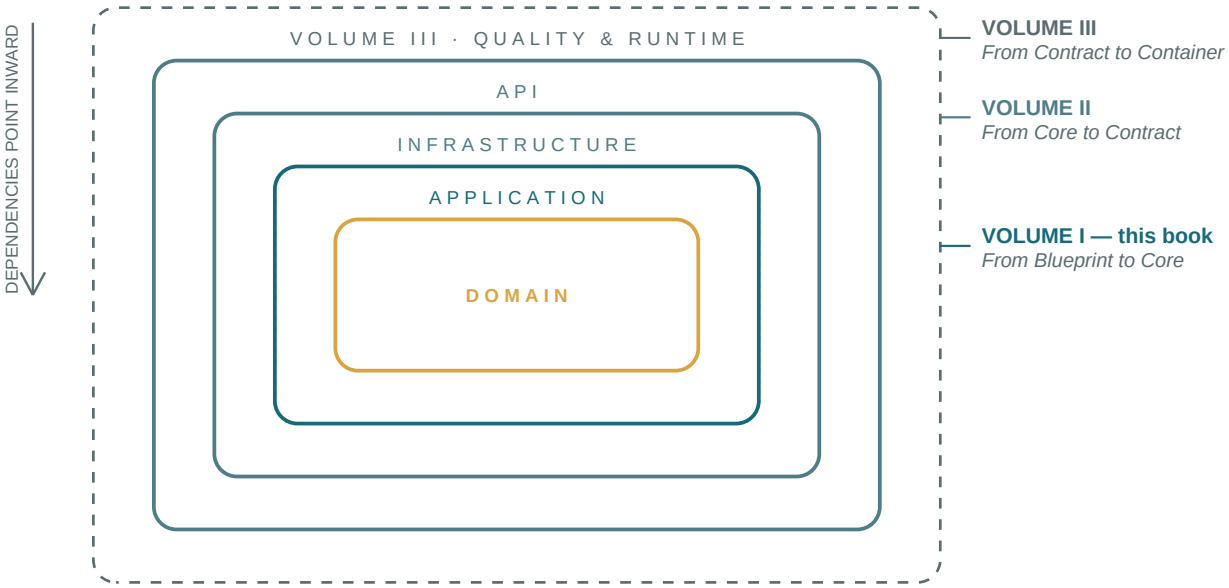
The reader this book expects is comfortable in C# and .NET 8 or later. ASP.NET Core and EF Core are not required here — neither appears in this volume beyond forward references — and the DDD vocabulary is re-introduced briefly as it is used, with the deep treatment remaining in *From Domain to Twin*. If you have read the earlier volumes, you will recognise every aggregate the moment it appears in code. If you have not, the code stands on its own; the earlier books simply explain why it has the shape it has.

— G.A., Edessa, 2026

How to Read This Book

(and the Trilogy Map)

This volume keeps the working habits of the series, and four of them shape every chapter. **The checkpoint.** Every chapter closes with a short, school-style checkpoint — a handful of questions you should be able to answer before turning the page. They are not decoration; if a checkpoint stops you, the chapter has more to give you on a second pass. **The full-listing rule.** Every class shown is written in full and compiles as printed. Nothing is elided behind an ellipsis unless the elision is marked and the omitted code appears elsewhere in the book, by page. **The watched-to-fail rule.** Every invariant demonstrated is demonstrated twice: once holding, and once deliberately violated in a unit test and watched to fail. **The honest boundary.** Amber-boxed sections state what a chapter defers, and name the volume that picks it up. They are not apologies; they are the seams of the architecture, printed where they occur.



The seam (dashed): IRepository · IUnitOfWork · IDateTimeProvider — declared as interfaces in Volume I, implemented against the schema atlas in Volume II.

READING PATHS

If Clean Architecture and DDD-in-code are new to you, read cover to cover; the parts build strictly on one another. If you are an architect who already lives in the Onion, skim Part I for the NEXUS-1-specific mapping decisions (Chapter 3 in particular) and go deep from Part II. If you arrive from *From Domain to Twin*, every aggregate here will be an old acquaintance in a new language — start at Chapter 3 and let Chapters 1–2 serve as reference. The earlier volumes are companions, not prerequisites: keep them within reach for the *why*, but nothing in this book requires them open to follow the *how*. Finally, a convention of this edition: every chapter is followed by a two-page **Deep Dive** for readers newer to architecture — plain-language re-explanations and UML figures, adding floors but never requirements — opening with a **WHERE WE ARE IN THE SOLUTION** panel that tracks the growing solution tree and test count.

PART I

The Architectural Foundation

Before a single entity is written, the ground it stands on must be poured: the Clean/Onion architecture and its Dependency Rule, the mapping of seventeen bounded contexts onto .NET projects and namespaces, the SharedKernel of honest primitives, and the composition root that wires it all together. Five chapters, one skeleton — every later class in this book has its address decided here.

CHAPTERS 1–5 · SOLUTION STRUCTURE · SHAREDKERNEL · DEPENDENCY INJECTION

1 · From Eight Books to One Solution

The NEXUS-1 project, as it stands on the day this chapter is written, consists of a running console and eight books — and not one line of backend C#. That sentence is the entire reason this volume exists, so it is worth unpacking with some care: what exactly exists, what exactly does not, and why the gap between them has the precise shape of a .NET solution.

What exists: the console

The oldest artifact is the Phase-0 console — a single HTML file, *index.html*, that runs a six-group point-kinetics simulation live in the browser, scores training drills against it, collapses alarm floods to a root cause on an authored incident archive, and renders the fleet in 3D. It carries no live plant connection and says so on screen. The console matters to this book for one reason: it is the eventual *consumer*. Everything the backend trilogy builds is, in the end, an API that console (or any client like it) could call — and wiring the two together is deliberately the closing act of Volume III, not an ambition of this one.

What exists: the paper plant

Around the console, eight volumes built the rest of the plant on paper — its physics, its data backbone, its domain model, its engines, and finally an audit of the series itself. Each of them hands something specific to this book. The ledger below is not a courtesy bibliography; it is the list of inputs this volume consumes, and every one of them is cited again at the exact chapter where it is used.

From Grid to Core	The physics and plant model behind ReactorFleet and Instrumentation — the meaning of every EngineeringValue this book will type as a C# record.
From Table to Twin	The data-design journey; the principle that a bounded context is a SQL schema with a passport — here restated as a .NET project with references.
From Entity to Context	The EF Core configuration atlas. Consumed in Volume II, but shaping every entity written here: owned-type readiness, key shapes, lookup discipline.
From Schema to System	The 17-sector, 654-table schema atlas — the target that the Volume-I abstractions (IRepository, IUnitOfWork) are shaped to meet.
From Domain to Twin	The DDD foundation: bounded contexts, ubiquitous language, entities, value objects, aggregates. This volume translates it into C#, class by class.
From Flood to Cause	The root-cause engine — source of the RootCauseCase aggregate and its central invariant: a case cannot close without evidence.
From Trial to Policy	The reinforcement-learning engine — source of the Policy and QTable aggregates walked through in Chapter 11.
From Certainty to Calibration	The retrospective — source of the honest-boundary discipline, and of the decision that split one overreaching book into this calibrated trilogy.

Table 1.1 — The inheritance ledger: eight volumes, and what Volume I takes from each.

What does not exist: the solution

Here is the gap, stated plainly. A developer who has read all eight volumes knows what a *RootCauseCase* is, which schema its table lives in, which columns that table has, which invariants the engine enforces, and which physics gives its signals meaning. What that developer cannot do is press F5. There is no *Nexus.sln*. There is no project to reference, no namespace to import, no test to run. The knowledge exists at every level of abstraction except the one developers actually inhabit: compiling code.

The backend trilogy closes that gap from the inside out, and this volume is the inside: the solution structure, the SharedKernel, the Domain layer, and the Application layer. Volume II adds persistence and the REST API; Volume III adds proof and runtime. The order is not editorial preference — it is the Dependency Rule applied to authorship. The inner layers depend on nothing outside themselves, so they can be written and proven first, with nothing installed but the .NET SDK. If they could not be, they would not be inner layers.

The honest scope of Volume I

By the end of this book you will have a complete *Nexus* solution: a SharedKernel of base types, seventeen bounded contexts mapped to projects and namespaces, domain entities and value objects with their business rules living inside them, aggregates whose invariants are enforced in code and violated on purpose in tests, domain events, domain services, and an Application layer that orchestrates it all through CQRS — commands, queries, handlers, validation, and pipeline behaviors — programmed entirely against abstractions. Every project compiles. Every test is green. And nothing persists, nothing listens on a port, and nothing authenticates anyone — by design, and by name, deferred to the volumes that own those concerns.

> **SEE IT IN NEXUS-1 · The eventual consumer**
>
> Open the Phase-0 console and look at three screens: **Alarms & Events**, **Root Cause Graph**, and **Optimization (RL)**. Every aggregate this volume writes in C# — *AlarmEvent*, *RootCauseCase*, *Policy*, *QTable* — is the server-side counterpart of something already visible on one of those screens. The console shows the plant; this trilogy builds the system of record behind it. Nothing in this volume talks to the console yet, and nothing will until the last chapters of Volume III.

> **HONEST BOUNDARY**
>
> This chapter has described the console and the eight volumes as inputs, and the claim deserves its boundary: this book does not re-derive them. The physics is taken as given from *From Grid to Core*; the schema is taken as given from *From Schema to System*; the domain model is taken as given from *From Domain to Twin*. Where a domain rule is implemented here, the rule's justification lives in the earlier volume and is referenced, not repeated. If a reader disagrees with a rule, the argument is there; this volume only guarantees that the code matches it.

The contract between the volumes

Because the trilogy is cut along an architectural boundary, the volumes can make each other promises that are checkable rather than rhetorical. Volume I promises Volume II a set of interfaces — *IRepository<T>*, *IUnitOfWork*, *IDateTimeProvider* — and entities shaped to satisfy the configuration atlas of *From Entity to Context* without modification: owned-type-ready value objects, honest key shapes, lookup discipline. Volume II promises to implement those interfaces against the exact tables of *From Schema to System*, changing nothing above the seam. Volume III promises to prove the whole under integration tests and put it in a container. Each promise is a compile-time or test-time fact, and each volume ends by auditing whether it kept its own — the calibration habit, made structural.

One consequence is worth flagging before the architecture chapter formalizes it: in this volume, **absence is a feature**. When Chapter 12 declares a repository interface and stops, the stopping is the point. The temptation to sketch "just a quick in-memory implementation for realism" is resisted everywhere except the test projects, where in-memory fakes appear precisely because they are fakes and are named as such. The first time a real database appears in the trilogy, it will be in a volume whose job is databases.

> **CHECKPOINT · CHAPTER 1**
>
> Before moving on, you should be able to answer these without looking back:
>
> **1.** Three kinds of artifact existed before this book: the console, the atlases, and the domain model. Which volume produced each, and which of the three does Volume I consume most directly?
>
> **2.** Why does the trilogy build the *inner* layers first? Answer in terms of the Dependency Rule, not in terms of convenience.
>
> **3.** Volume I ships no database, no HTTP, and no security. For each of the three, name the volume that owns it.
>
> **4.** What, concretely, does Volume I promise Volume II — and how would a failure to keep that promise show up? (Hint: it would not show up at runtime.)
>
> **5.** In what sense is "no database is required to test this volume" a proof rather than a convenience?

The next chapter stops narrating and starts justifying: why Clean/Onion Architecture — of all the shapes a backend can take — is the one NEXUS-1 needs, what the Dependency Rule actually forbids, and why only two of the four layers appear between these covers.

The Vocabulary Under the Story

Deep Dives are companion sections for readers meeting these ideas for the first time. They add floors, never requirements: everything in the chapter stands without them, and nothing in them will be quietly assumed later. Each opens with the panel below, which recurs throughout the book so you always know exactly where you stand in the solution:

WHERE WE ARE IN THE SOLUTION

```
Nexus.sln
|- src/Nexus.SharedKernel
|- src/Nexus.Domain
|    |- <17 context folders>
|- src/Nexus.Application
|    |- Abstractions (ports)
|- tests/Nexus.Domain.Tests
|- tests/Nexus.Application.Tests
```

Tests green: 0

Nothing exists yet. Chapter 1 is inventory and intent; the first file appears in Chapter 3, the first passing test in Chapter 4. This panel fills in as the book builds.

Solution, project, namespace, assembly — the four words everything rides on

A **solution** (*Nexus.sln*) is a binder: it holds projects together so tools can open, build, and test them as one, but it contains no code of its own. A **project** (a *.csproj* file plus a folder of C#) is the unit that compiles — each project becomes one **assembly**, a single *.dll* file, and the crucial consequence is that a project can only use types from assemblies it explicitly *references*. That is why this book keeps saying the architecture is “enforced by the compiler”: references are per-project settings, so an arrow that is not declared is code that cannot compile. A **namespace**, by contrast, is only an address — *Nexus.Domain.RootCause* tells you where a class lives, but namespaces cross assembly lines freely and enforce nothing. Projects are walls; namespaces are street signs. The book uses both, and never confuses which one is load-bearing.

“Bounded context” and “domain model”, in plain terms

A **domain** is the subject matter the software is about — here, the operation of a nuclear plant demonstrator. A **domain model** is the software’s working theory of that subject: the concepts (alarm, case, evidence, policy), their rules, and their relationships, expressed as code. A **bounded context** is the honest admission that one theory cannot cover a whole plant: the word “event” means something different to AlarmManagement than to ReinforcementLearning, so each context is a region where the vocabulary has exactly one meaning, with explicit borders to its neighbors. NEXUS-1 has seventeen such regions, and most of this volume is about keeping their borders real in code.

The cast and the use cases — a map of everything this trilogy serves

Before any architecture, it helps to see the system as its users will: actors on the outside, capabilities on the inside. The UML **use-case diagram** below is that view for the slice of NEXUS-1 this volume builds. Read it as promises: every oval becomes, in Part III, either a command (if it changes the plant) or a query (if it only asks).

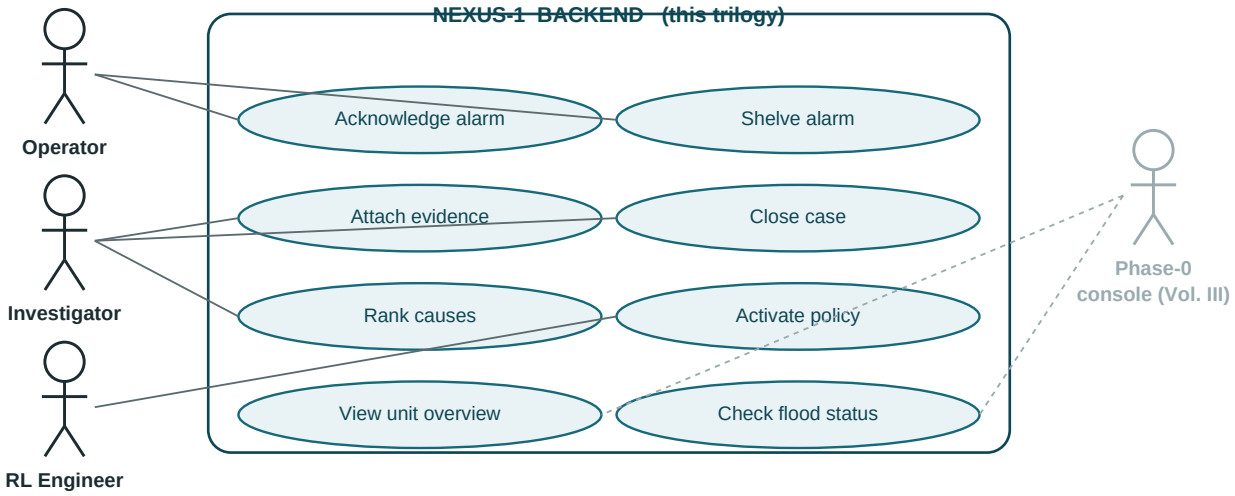


Figure D1.1 — UML use-case diagram of the Volume-I scope. Solid actors are human roles; the console joins in Volume III.

Two things the diagram quietly teaches. First, the system boundary is a real line: the actors are outside it, which is exactly why Chapter 12 will insist the core never reaches out — everything crosses that border inward, as requests. Second, the console is drawn gray and dashed on purpose: it exists today as a single HTML file, but it becomes a *client* of this boundary only in Volume III, and the book will not pretend otherwise. Keep this figure bookmarked — Chapters 13 through 15 turn each oval into a typed message, and Chapter 17 runs six of the eight in a single night.

2 · The Philosophy of Clean/Onion Architecture

Why NEXUS-1 needs boundaries more than most systems

Every architecture text argues for boundaries; few argue from a system large enough for the argument to bite. NEXUS-1 is seventeen bounded contexts over a 654-table schema, and at that scale the failure mode of a boundary-less codebase is not untidiness — it is a specific, predictable coupling disaster. AlarmManagement reaches into RootCause's tables because it is quicker than asking; ReinforcementLearning takes a reference to Instrumentation's internals; a change to one Unit property recompiles half the plant. *From Table to Twin* put it in one sentence: a bounded context is a schema with a passport. This chapter states the code-side equivalent: a bounded context is a project whose references are the only visas it holds — and the architecture exists to make every undocumented crossing fail to compile.

The Dependency Rule, stated precisely

Clean Architecture, Onion Architecture, and Hexagonal (Ports & Adapters) differ in vocabulary and diagrams, but they share one load-bearing rule, and NEXUS-1 adopts them at exactly that shared core. The rule: **source-code dependencies may only point inward, toward higher-level policy.** Three consequences follow, and each forbids something concrete. First, the Domain layer references *nothing* — not EF Core, not ASP.NET, not even the Application layer above it; its only dependency is the SharedKernel and the base class library. Second, the Application layer may reference Domain, but must express every need for the outside world — persistence, time, transactions — as an interface it *owns*. Third, Infrastructure and API depend on the inner layers, never the reverse: the database is a detail plugged into the core, not a foundation the core rests on. The rule says nothing about folders, naming, or frameworks. It constrains one thing only: the direction of the arrows.

The four layers — and the two that live here

The trilogy map on page 5 drew the Onion; here is what each ring owns. **Domain** (Part II): entities, value objects, aggregates, domain events, domain services — the business rules of the plant, with no knowledge of storage or transport. **Application** (Part III): use-case orchestration — commands, queries, handlers, validation — plus the ports (*IRepository*, *IUnitOfWork*) that name what it needs. **Infrastructure** (Volume II): the adapters — EF Core, the DbContext, repositories against the schema atlas. **API** (Volume II): controllers, DTO mapping, HTTP. Volume III wraps all four in proof and runtime. Only the first two appear between these covers, and the reason is the rule itself: the inner rings are the only ones that can be complete before the outer ones exist. The reverse is impossible.

Making the rule uncompilable to break

A rule that lives in a wiki dies in a sprint. The Dependency Rule survives at NEXUS-1 scale only because it is encoded where no reviewer is needed: in project references. *Nexus.Domain* holds a reference to *Nexus.SharedKernel* and nothing else; *Nexus.Application* references Domain and SharedKernel and nothing else. There is no Infrastructure project yet, but when Volume II adds one, it will reference inward and nothing will reference it back except the composition root. Per the watched-to-fail rule of page 5, a claim like that must be violated and observed. So: suppose a Domain entity tries to take the shortcut every large codebase eventually attempts — reaching for EF Core to decorate itself for persistence.

```
// src/Nexus.Domain/AlarmManagement/AlarmEvent.cs
using Microsoft.EntityFrameworkCore;   // <-- outward dependency

namespace Nexus.Domain.AlarmManagement;

[Index(nameof(RaisedAtUtc))]           // <-- persistence detail in the core
public sealed class AlarmEvent : Entity<AlarmEventId> { /* ... */ }

$ dotnet build src/Nexus.Domain
error CS0246: The type or namespace name 'Microsoft' could not be found
  (are you missing a using directive or an assembly reference?)
Build FAILED. 1 Error(s).
```

Listing 2.1 — The forbidden arrow: Nexus.Domain attempting to reference outward, and the compiler refusing.

The error is the architecture working. *Nexus.Domain.csproj* simply contains no package or project reference that could resolve the namespace, so the violation is not caught in review — it never reaches review. Chapter 3 turns this from an anecdote into a system: the full solution layout in which every legal arrow is a declared reference and every illegal one is a compile error. (The *[Index]* attribute the listing wanted does eventually exist — as Fluent API configuration in Volume II, on the Infrastructure side of the seam, exactly where *From Entity to Context* put it.)

> **SEE IT IN NEXUS-1 · One key, many views — the rule's payoff**
>
> In the console, the top-bar **UNIT** selector reorients every unit-scoped screen at once — one *UnitId*, many views, no synchronisation code. That works because the contexts agree on a shared key while owning their own data. The Dependency Rule is the same discipline vertically: many outer details, one inner truth, and all arrows pointing at it.

What the rule costs — stated before it is paid

The calibration habit requires pricing a decision, not only praising it. The Dependency Rule costs indirection: the Application layer talks to *IRepository<T>* when a DbContext would be two lines shorter. It costs ceremony: more projects, more interfaces, a composition root to maintain. And it costs patience: nothing in this volume serves a request, which can feel like building a plant that never opens its gates. Those costs are real and they are paid on purpose, because the alternative was priced by *From Certainty to Calibration*: at seventeen contexts, shortcuts compound into the coupling disaster of page 12, and the interest rate is measured in rewrites. For a to-do app, this architecture is overhead. NEXUS-1 is not a to-do app.

> **HONEST BOUNDARY**
>
> This chapter presents Clean/Onion as the architecture NEXUS-1 uses, not as the architecture all systems should use. No benchmark against a modular monolith without interfaces, a vertical-slice design, or a well-factored transaction script is offered here, and the claim "this would fail at scale" rests on the reasoned argument above, not on a controlled experiment. What *is* demonstrated, in this volume and the next two, is the promised payoff on this system: an inner core proven with no database, and an outer shell swapped in without touching it. Judge the rule by whether the trilogy keeps that promise.

> **CHECKPOINT · CHAPTER 2**
>
> Before moving on, you should be able to answer these without looking back:
>
> **1.** State the Dependency Rule in one sentence, then name the three concrete things it forbids in NEXUS-1.
>
> **2.** Why does Listing 2.1 fail to compile, mechanically? Where is the rule actually encoded — in a class, a convention, or a file?
>
> **3.** The Application layer needs the database. How does it express that need without violating the rule, and who owns the resulting interface?
>
> **4.** The *[Index]* attribute from Listing 2.1 describes something true about the system. Where does that truth legally live, and in which volume?
>
> **5.** Name two costs of the rule this chapter concedes, and the argument for paying them at NEXUS-1 scale specifically.

With the philosophy priced and the rule enforced by the compiler, the next chapter builds the thing itself: *Nexus.sln* — seventeen bounded contexts mapped onto projects, namespaces, and folders, with every reference declared and every illegal arrow impossible.

Dependencies, Literally

WHERE WE ARE IN THE SOLUTION

```
Nexus.sln
|- src/Nexus.SharedKernel
|- src/Nexus.Domain
|    |- <17 context folders>
|- src/Nexus.Application
|    |- Abstractions (ports)
|- tests/Nexus.Domain.Tests
|- tests/Nexus.Application.Tests
```

Tests green: 0

Still zero files, by design: this book decides the architecture before typing it. The decisions so far — four layers, dependencies inward, two layers in this volume — become folders and csproj files in Chapter 3.

What a dependency actually is

Strip the philosophy and a dependency is one mechanical fact: **A depends on B if A cannot compile without B.** In C# that happens two ways — a *using* of B's namespace plus any mention of B's types in A's code, and, underneath, a reference in A's .csproj that makes B's assembly visible at all. The Dependency Rule is therefore not etiquette; it is a statement about which .csproj files may contain which <*ProjectReference*> lines, and Listing 2.1's CS0246 is what the rule looks like when it wins: the compiler cannot even *find* the forbidden type, because no reference makes it visible. When this book says an arrow "does not exist to be pointed," that is the literal meaning.

Policy and detail — the two words behind "inward"

"Inward toward higher-level policy" uses two terms of art worth unpacking. **Policy** is the code that would survive a technology change: *a case cannot close without evidence* is true whether the data lives in SQL Server, a file, or a filing cabinet. **Detail** is everything that exists because of a technology choice: the DbContext, the HTTP route, the JSON shape. The rule says details may know about policy (the repository must understand what a RootCauseCase is) but policy may never know about details (the case must not know what a DbContext is) — because when a detail changes, and details always change, the blast radius must point outward, where the replaceable things live. Every diagram in this book with arrows on it is drawing that one sentence.

Reading your first architecture error

error CS0246: The type or namespace name 'Microsoft' could not be found reads, to a newcomer, like a typo report. Read it instead as a sentence about the build graph: "the assembly containing that name is not referenced by this project." The fix the compiler suggests — add the reference — is precisely the fix the architecture forbids, and knowing that distinction is the moment architecture stops being diagrams and starts being something you can feel in the tooling. The two figures overleaf put pictures to it.

The Onion, with its arrows shown honestly

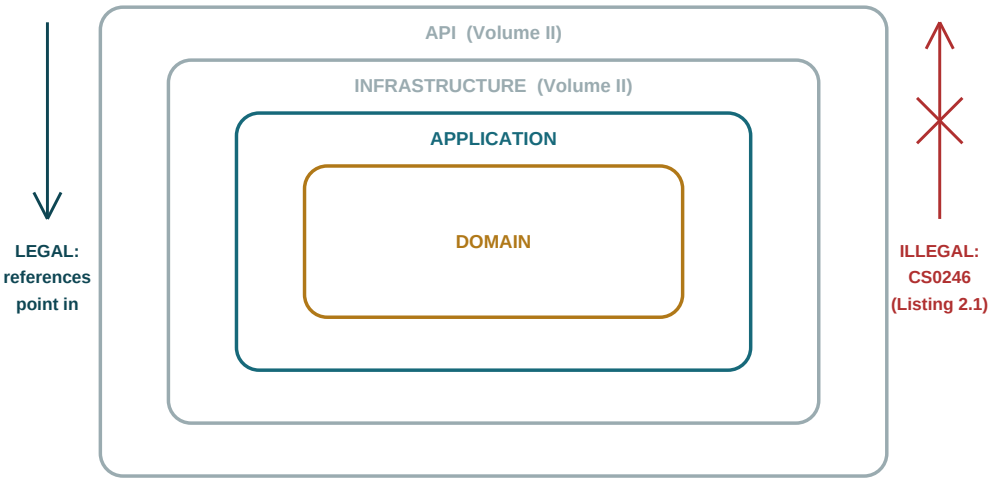


Figure D2.1 — The Onion with its arrows: gray rings are absent in this volume, which is why the red arrow cannot compile.

A preview: the life of one click

The UML **sequence diagram** below previews where this volume's code will sit in the finished system when an operator clicks ACK on the console. Gray participants do not exist yet — they are Volumes II and III — which is exactly the point: the solid middle is complete and testable without them.

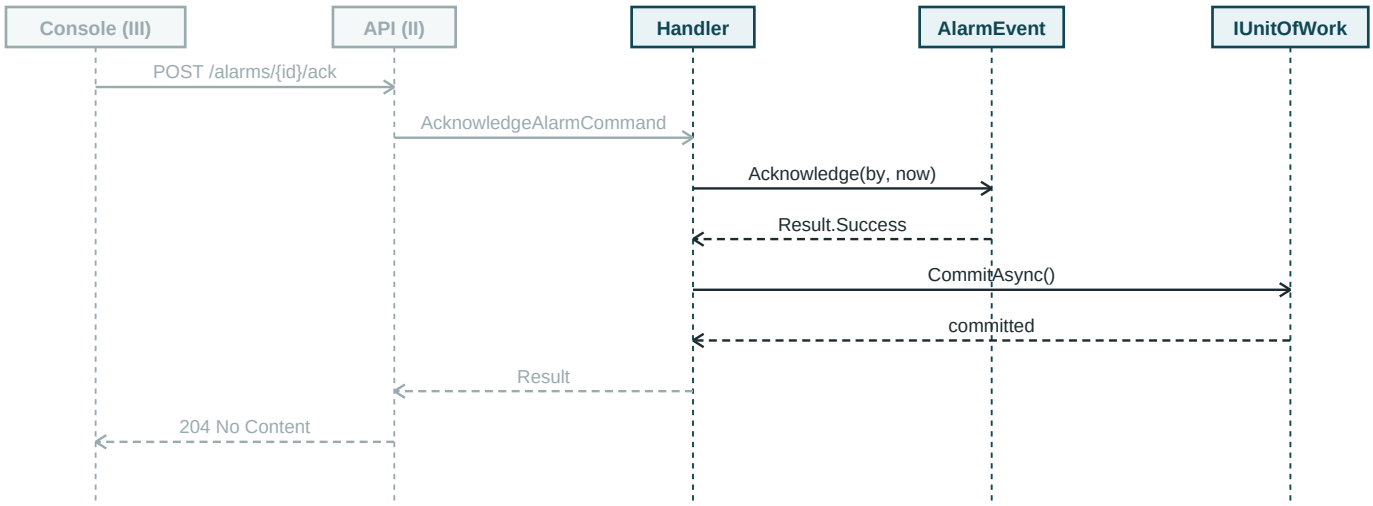


Figure D2.2 — Sequence diagram (simplified): one acknowledge, end to end. Solid lifelines are built in this volume.

Read a sequence diagram top-to-bottom as time, left-to-right as delegation: each solid arrow is a call, each dashed arrow an answer. Notice who makes the only decision — the *AlarmEvent* lifeline, in the one message that touches it — and that everything to its left merely carries messages. Chapters 6, 12, and 14 build the three solid lifelines; this figure returns, fully solid, in Volume II.

3 · Structuring the Nexus Solution

The first decision: how many projects?

Seventeen bounded contexts and four layers suggest an obvious grid — and the obvious grid is a trap. Project-per-context-per-layer yields dozens of assemblies before a single rule is written: slower builds, a solution explorer nobody can scan, and reference plumbing that must be repeated seventeen times. NEXUS-1 takes the other road, and takes it deliberately: **one project per layer, one folder per context.** The layer boundary — the one the Dependency Rule actually constrains — is enforced by the compiler through project references. The context boundary lives as a namespace inside each layer project, mirroring exactly how *From Schema to System* put seventeen SQL schemas inside one database. Volume I therefore ships five projects:

```
Nexus.sln
|- src/
|  |- Nexus.SharedKernel/          # Entity, ValueObject, DomainEvent, Result  (Ch. 4)
|  |- Nexus.Domain/                # PART II lives here
|  |    |- AlarmManagement/  CorePlatform/  RootCause/  ReactorFleet/
|  |    |- ReinforcementLearning/  Instrumentation/  DigitalTwin/  ...(17 total)
|  |- Nexus.Application/           # PART III lives here
|       |- Abstractions/           # IRepository<T>, IUnitOfWork, IDateTimeProvider (Ch. 12)
|       |- AlarmManagement/  RootCause/  ...(same 17 folders)
|- tests/
|  |- Nexus.Domain.Tests/          # invariants, watched to fail          (Ch. 18)
|  |- Nexus.Application.Tests/     # handlers against in-memory fakes     (Ch. 19)
|- Directory.Build.props           # net8.0, nullable enable, warnings as errors
```

Listing 3.1 — The Volume-I solution tree. Five projects; seventeen contexts recur as folders inside two of them.

Two absences in that tree are the chapter's real content. There is no *Nexus.Infrastructure* and no *Nexus.Api* — not “not yet written” but *not present*, so that nothing in this volume can accidentally lean on them. When Volume II adds both, this tree gains two folders and the existing three src projects change by zero lines. That claim is checkable, and Volume II opens by checking it.

Names that carry their address

The namespace convention is *Nexus.<Layer>.<Context>*, and it is chosen so that every type carries its architectural address in its full name. *Nexus.Domain.RootCause.RootCauseCase* tells a reader the layer (inner, no dependencies), the context (sector 10 of the schema atlas), and the aggregate — before a single line of its body is read. The seventeen folder names are not invented here: they are, character for character, the seventeen schema names of *From Schema to System*, from CorePlatform to Audit. Appendix D prints the full three-way map — context → schema → namespace — so that when Volume II wires *RootCause.RootCauseCase* (the table) to *Nexus.Domain.RootCause.RootCauseCase* (the class), the join is a lookup, not a guess.

References as law

Chapter 2 showed one illegal arrow dying at compile time; here is the entire legal graph. It fits in a sentence — SharedKernel references nothing; Domain references SharedKernel; Application references Domain (and SharedKernel transitively); each test project references the project it tests — and it fits in the two csproj fragments below, printed in full because their brevity *is* the architecture:

```
<!-- src/Nexus.Domain/Nexus.Domain.csproj -->
<ItemGroup>
  <ProjectReference Include="..\Nexus.SharedKernel\Nexus.SharedKernel.csproj" />
</ItemGroup>

<!-- src/Nexus.Application/Nexus.Application.csproj -->
<ItemGroup>
  <ProjectReference Include="..\Nexus.Domain\Nexus.Domain.csproj" />
</ItemGroup>
<ItemGroup>
  <PackageReference Include="MediatR" Version="12.*" />
  <PackageReference Include="FluentValidation" Version="11.*" />
</ItemGroup>
```

Listing 3.2 — Every ItemGroup of both src csproj files. What is absent is what is forbidden.

Note what the Application project's package list already concedes: MediatR and FluentValidation are third-party dependencies inside an inner layer. Chapter 13 prices that decision properly — including what a hand-rolled dispatcher would look like — rather than smuggling it past the reader here. The Domain project, by contrast, holds no PackageReference at all, and Part II will keep it that way: the innermost ring stays framework-free the whole book.

> **TRY IT**
>
> Clone the Appendix-B skeleton (or type Listing 3.1 by hand — it is fifteen minutes well spent), run *dotnet build*, and confirm five green projects. Then re-run the experiment of Listing 2.1: add the EF Core using to any Domain file and watch CS0246 return. The solution is now a machine that rejects that class of mistake forever.

Inside a context folder

Each of the seventeen Domain folders follows one internal shape, so that learning one context is learning them all: aggregate roots and entities at the folder root, a *ValueObjects/* subfolder, an *Events/* subfolder for the context's domain events, and — only where the context needs one — a *Services/* subfolder for domain services like RootCause's *CausalRankingService*. Application folders mirror the pattern with *Commands/*, *Queries/*, and *Validators/*. Chapter 11 walks three contexts end to end under this shape; the other fourteen then hold no surprises, which is precisely the point of having a shape.

> **HONEST BOUNDARY**
>
> One project per layer buys build speed and a scannable solution, and it pays for them with a real weakness: **inside** Nexus.Domain, the compiler will not stop *AlarmManagement* from referencing *RootCause* internals — both are namespaces in one assembly, and namespaces are not visas. In this book that context boundary is held by convention: cross-context communication happens through domain events and through the Application layer, never by direct type reference, and every listing obeys the convention in plain sight. Teams that want it machine-enforced can add an architecture-test library that asserts namespace isolation; this volume names that option honestly rather than shipping a test it never shows. The layer boundary is law; the context boundary, inside a layer, is discipline.

> **CHECKPOINT · CHAPTER 3**
>
> Before moving on, you should be able to answer these without looking back:
>
> **1.** NEXUS-1 rejects project-per-context-per-layer. What does folder-per-context buy, and what precise weakness does it accept in exchange?
>
> **2.** From the full name *Nexus.Domain.RootCause.RootCauseCase* alone, what three facts do you know — and which earlier volume fixed the middle segment's spelling?
>
> **3.** Reproduce Listing 3.2's reference graph from memory as one sentence. Which project holds zero PackageReferences, and for how long will that stay true?
>
> **4.** Why is the absence of Nexus.Infrastructure from Listing 3.1 a design feature rather than an incompleteness — and how will Volume II check the claim?
>
> **5.** By what two mechanisms may one context legally affect another in this codebase?

The skeleton stands, but its projects are empty of everything except structure. The next chapter pours the first concrete: the SharedKernel — Entity, ValueObject, DomainEvent, strongly-typed identifiers, and the Result type — the small set of honest primitives every one of the seventeen contexts will build on.

Five csproj Files, Read Slowly

WHERE WE ARE IN THE SOLUTION

```
> Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 0

First milestone: the whole tree of Listing 3.1 now exists on disk and compiles — five projects, empty of everything except structure and references. The test projects exist too; the first test arrives next chapter.

What a ProjectReference literally does

A .csproj file is a small XML recipe the build reads: which framework to target, which packages to download, and — the load-bearing part — which sibling projects this one may see. *<ProjectReference Include="..\Nexus.SharedKernel…"/>* means: compile that project first, then make every *public* type in its assembly visible here. Two consequences matter for reading Listing 3.2. **Visibility is transitive for compilation:** Nexus.Application references Domain, Domain references SharedKernel, so Application code can use *Result* and *Entity<TId>* without naming the kernel — the arrow exists through the chain. **And *public* vs *internal* is measured at this same wall:** *internal* means "visible only inside my own assembly", which is why Chapter 8 will call it "the tightest fence the language sells" — the fence is the project boundary you are looking at right now.

Why not one project per context? The arithmetic

The "obvious grid" Chapter 3 rejects is worth pricing in numbers a newcomer can check. Seventeen contexts × four layers is 68 projects before tests; each project adds a csproj to maintain, an assembly to load, and a compile step to schedule, and every cross-layer reference must be repeated seventeen times. The chosen design spends 5 projects and puts the seventeen-ness where it is cheap — folders and namespaces, which cost nothing to build and nothing to load. The trade, stated once more in newcomer terms: **the compiler guards the boundary that must never break (layers), and discipline guards the boundary that bends occasionally under refactoring (contexts).** Putting the expensive guard on the expensive mistake is the whole decision.

The reference graph, drawn

Below is Listing 3.2 as a picture — a component view of the five projects. Solid arrows are declared references (“may see”); the red crossed arrows are the references that do not exist, each with the chapter where its absence does real work.

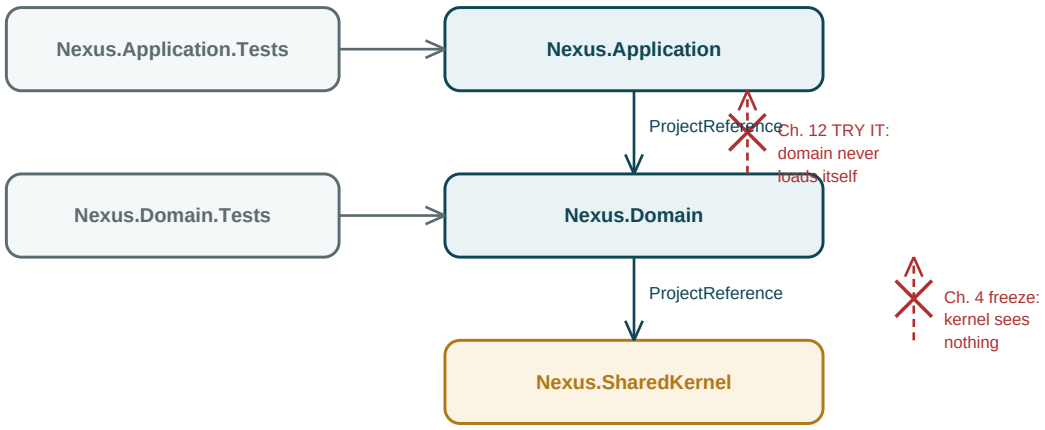


Figure D3.1 — Component view of Listing 3.2. Every solid arrow is one line of XML; every red cross is a compile error waiting to defend you.

Reading the picture against the future

Three observations to carry forward. The kernel is a leaf: nothing it could reference exists beneath it, which is what makes freezing it (Chapter 4) cheap to promise. The test projects sit *beside* the stack, not on top of it — they reference inward like everything else, which is why Chapter 5 can appoint them composition root without bending a rule. And the diagram has an empty right margin on purpose: when Volume II adds Infrastructure and API, they will dock above Application with arrows pointing *down* into this picture, and not one existing arrow will move. Redraw this figure from memory before starting Part II; it is the map the whole volume walks on.

4 · The SharedKernel

The smallest project in the solution, on purpose

Nexus.SharedKernel is the one project every other project references, which makes it the most dangerous project in the solution: anything placed here is coupled to everything forever. The admission rule is therefore brutal and worth memorising — **a type enters the kernel only if it is business-meaning-free and needed by many contexts.** Five citizens pass: the *Entity<TId>* base class, strongly-typed identifiers, the *IDomainEvent* contract, the *Result*/*Error* pair, and the *Ensure* guard clauses. Everything else — including types that *feel* universal, like *EngineeringValue* — belongs to a context and is refused entry; the end of this chapter prosecutes exactly that temptation. First, the citizens themselves, each printed in full.

Entity: identity, equality, and a mailbox for events

```
namespace Nexus.SharedKernel;

public abstract class Entity<TId> : IEquatable<Entity<TId>> where TId : notnull
{
    private readonly List<IDomainEvent> _domainEvents = [];

    protected Entity(TId id) => Id = id;

    public TId Id { get; }

    public IReadOnlyList<IDomainEvent> DomainEvents => _domainEvents.AsReadOnly();

    protected void Raise(IDomainEvent domainEvent) => _domainEvents.Add(domainEvent);

    public void ClearDomainEvents() => _domainEvents.Clear();

    public bool Equals(Entity<TId>? other) =>
        other is not null && GetType() == other.GetType() && Id.Equals(other.Id);

    public override bool Equals(object? obj) => Equals(obj as Entity<TId>);

    public override int GetHashCode() => HashCode.Combine(GetType(), Id);

    public static bool operator ==(Entity<TId>? a, Entity<TId>? b) =>
        a is null ? b is null : a.Equals(b);

    public static bool operator !=(Entity<TId>? a, Entity<TId>? b) => !(a == b);
}
```

Listing 4.1 — src/Nexus.SharedKernel/Entity.cs, complete. Identity-based equality plus the domain-event mailbox.

Two design facts carry the weight. Equality compares *GetType()* and *Id* — nothing else — because an entity that changes every property is still itself; that is the definition Chapter 6 builds on. And the event mailbox is *protected* on the way in, public and read-only on the way out: only the entity may raise its own events, while dispatching them is deliberately left to a seam Volume II owns.

Strongly-typed identifiers: making Guids unswappable

A 654-table system runs on identifiers, and raw Guids make every pair of them silently interchangeable. The kernel's answer costs three lines per context type:

```
// src/Nexus.SharedKernel/IEntityId.cs
namespace Nexus.SharedKernel;

public interface IEntityId { Guid Value { get; } }

// each context declares its own, e.g. in Nexus.Domain/AlarmManagement/:
public readonly record struct AlarmEventId(Guid Value) : IEntityId
{
    public static AlarmEventId New() => new(Guid.NewGuid());
    public override string ToString() => Value.ToString("N");
}

// the bug this abolishes -- two ids passed in the wrong order:
ackService.Acknowledge(operator.Id, alarm.Id);   // swapped
error CS1503: argument 1: cannot convert from 'OperatorId' to 'AlarmEventId'
```

Listing 4.2 — The identifier pattern, and the argument-swap bug dying at compile time.

A *readonly record struct* gives value equality, immutability, and zero allocation for the price of one line, and the wrapper evaporates at the database boundary: *From Entity to Context* already reserved the conversion seam, and Volume II fills it with EF Core value converters without the Domain noticing.

Domain events: the contract, not yet the plumbing

```
namespace Nexus.SharedKernel;

public interface IDomainEvent
{
    Guid EventId { get; }
    DateTime OccurredAtUtc { get; }
}

public abstract record DomainEvent(Guid EventId, DateTime OccurredAtUtc) : IDomainEvent;
```

Listing 4.3 — src/Nexus.SharedKernel/IDomainEvent.cs and DomainEvent.cs, complete.

Nine lines, and deliberately no more. The kernel defines what an event *is* — an immutable record of something that happened, named in the past tense — and stays silent on how events travel. Concrete events like *AlarmRaisedEvent* belong to their contexts (Chapter 9); dispatch belongs to the Application pipeline (Chapter 16) and, across process boundaries, to Volume II. Value objects, the other famous kernel citizen, are conspicuously absent: in .NET 8 a *sealed record* with a private constructor and a static factory does the job with no base class at all, and Chapter 7 makes that case with the kernel's blessing.

Result and Error: two failure channels, kept apart

The kernel's doctrine on failure is the most consequential decision in this chapter. **Business-rule failures are values; programmer errors are exceptions.** An operator acknowledging an already-cleared alarm is not exceptional — it is Tuesday — and modelling it with a throw turns control flow into stack unwinding and makes the rule invisible in the method signature. So domain operations that can legitimately fail return *Result*:

```
namespace Nexus.SharedKernel;

public sealed record Error(string Code, string Message)
{
    public static readonly Error None = new(string.Empty, string.Empty);
}

public class Result
{
    protected Result(bool isSuccess, Error error)
    {
        if (isSuccess != (error == Error.None))
            throw new ArgumentException("Invalid success/error combination.");
        IsSuccess = isSuccess;
        Error = error;
    }

    public bool IsSuccess { get; }
    public bool IsFailure => !IsSuccess;
    public Error Error { get; }

    public static Result Success() => new(true, Error.None);
    public static Result Failure(Error error) => new(false, error);
    public static Result<T> Success<T>(T value) => new(value, true, Error.None);
    public static Result<T> Failure<T>(Error error) => new(default, false, error);
}

public sealed class Result<T> : Result
{
    private readonly T? _value;

    internal Result(T? value, bool ok, Error error) : base(ok, error) => _value = value;

    public T Value => IsSuccess ? _value!
        : throw new InvalidOperationException("No value on a failed result.");
}
```

Listing 4.4 — src/Nexus.SharedKernel/Error.cs and Result.cs, complete.

Note where the two channels meet: reading *Value* off a failed result throws, because ignoring a failure you were handed as a value is a programmer error, not a business outcome. Error codes are strings by convention *Context.Rule* — “RootCause.CaseWithoutEvidence” — which Volume II will map onto HTTP problem details without the Domain changing a line.

Ensure: the exception channel, fenced

```
namespace Nexus.SharedKernel;

public static class Ensure
{
    public static void NotEmpty(string value, string name)
    {
        if (string.IsNullOrWhiteSpace(value))
            throw new ArgumentException($"{name} must not be empty.", name);
    }

    public static void NotDefault<T>(T value, string name) where T : struct
    {
        if (EqualityComparer<T>.Default.Equals(value, default))
            throw new ArgumentException($"{name} must not be default.", name);
    }
}
```

Listing 4.5 — src/Nexus.SharedKernel/Ensure.cs, complete. Guards protect invariants of construction, not business rules.

The first green — and the first watched failure

The kernel is small enough to prove in one sitting, and doing so establishes the test rhythm the whole book keeps. Two tests below: the first pins the equality contract; the second is the book's first *watched-to-fail* — it deliberately does the wrong thing (reading a value off a failure) and asserts that the kernel punishes it.

```
namespace Nexus.Domain.Tests.SharedKernel;

public sealed class KernelContractTests
{
    private sealed record TestId(Guid Value);
    private sealed class TestEntity(TestId id) : Entity<TestId>(id);

    [Fact]
    public void Entities_with_the_same_id_are_equal_whatever_their_state()
    {
        var id = new TestId(Guid.NewGuid());
        Assert.Equal(new TestEntity(id), new TestEntity(id));
    }

    [Fact]
    public void Reading_Value_from_a_failed_Result_throws()
    {
        var failed = Result.Failure<int>(new Error("Test.Rule", "nope"));
        Assert.Throws<InvalidOperationException>(() => failed.Value);
    }
}

$ dotnet test tests/Nexus.Domain.Tests
Passed! - Failed: 0, Passed: 2, Skipped: 0, Duration: 41 ms
```

Listing 4.6 — tests/Nexus.Domain.Tests/SharedKernel/KernelContractTests.cs, and the run.

The refusal that keeps the kernel honest

Now the prosecution promised on page 22. *EngineeringValue* — a number with a unit and a quality flag, defined by *From Grid to Core* and used by half the plant — looks like the perfect kernel citizen. It is refused, on the admission rule's first clause: it is saturated with business meaning. Which units exist, how quality degrades, what conversions are legal — those are Instrumentation-context decisions, and freezing them into the kernel would let every context depend on their details forever. It lives in *Nexus.Domain.Instrumentation* (Chapter 7), and contexts that need it say so in the ordinary way. The kernel stays five citizens small — and the stability contract is explicit: **within this trilogy, Nexus.SharedKernel gains no new public type after this chapter.** Volumes II and III will be the audit of that sentence.

> **HONEST BOUNDARY**
>
> Terminology, priced: in DDD strategic design, “Shared Kernel” names a negotiated subset of the *domain model* that two teams co-own — a relationship between contexts. This project is the humbler .NET usage: shared technical base types with no domain meaning at all. *From Domain to Twin* uses the strategic sense; this book uses the project sense, and the two must not be conflated. Also priced: the Result pattern is a stance, not a consensus — respected codebases model rule failures with exceptions and gain simpler call sites at the cost of invisible signatures. NEXUS-1 chooses visible signatures; Chapter 14 shows the price being paid, in full, at every handler.

> **CHECKPOINT · CHAPTER 4**
>
> Before moving on, you should be able to answer these without looking back:
>
> **1.** State the kernel admission rule, and use it to convict *EngineeringValue* in one sentence.
>
> **2.** Entity equality inspects exactly two things. Which two, why not the rest — and why does *GetType()* participate at all?
>
> **3.** Why is *Raise* protected while *ClearDomainEvents* is public? Who is each visibility aimed at?
>
> **4.** Name the two failure channels and assign each of these to one: a case closed without evidence; a Guid identifier that is all zeros; reading *Value* off a failure.
>
> **5.** What does *readonly record struct* buy an identifier, and where does the wrapper get unwrapped — in which volume, and by whom?

Five projects, five kernel citizens, two green tests. One structural question remains before Part II can pour the domain itself: who constructs all these objects, and where do interfaces meet their implementations? The next chapter builds the composition seam — dependency injection as Volume I practices it, including the discipline of a composition root that does not yet have an application to compose.

Generics, Records, and Two Kinds of No

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
> |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 2

The kernel is populated and frozen: five citizens, two green tests, and a promise — no new public kernel type for the rest of the trilogy — that later chapters will audit.

Generics as fill-in-the-blank contracts

Entity<TId> reads, to a newcomer, like punctuation soup. Read it as a form with one blank: “I am an Entity, and my identifier is of type ____”. Each aggregate fills the blank once — *AlarmEvent : Entity<AlarmEventId>* — and from then on the compiler holds it to the answer: *Id* is an *AlarmEventId*, not a Guid, not anyone else’s key. The *where TId : notnull* clause is a condition on the blank itself (“no nullable answers”), and Chapter 12’s *where TRoot : Entity<TId>, IAggregateRoot* will stack two conditions the same way. Generics here are not cleverness; they are how one base class serves seventeen contexts without ever confusing their keys.

Records vs classes, in one paragraph

A **class** compares by reference: two objects are equal only if they are literally the same object in memory. A **record** is a class where the compiler rewrites equality to compare *contents* — plus generated *GetHashCode*, *ToString*, and copy machinery — which is exactly the semantics Chapter 7 wants for values and exactly wrong for entities, whose whole identity is “sameness despite change”. Hence the kernel’s split: *Entity* is a class with equality hand-written to compare Id and type only, while events and value objects are records and take the generated behavior. *Equals* and *GetHashCode* travel as a pair because every dictionary and set consults the hash first — disagree between them and collections quietly lose your objects, which is why Listing 4.1 overrides both or neither, never one.

The kernel as a UML class diagram

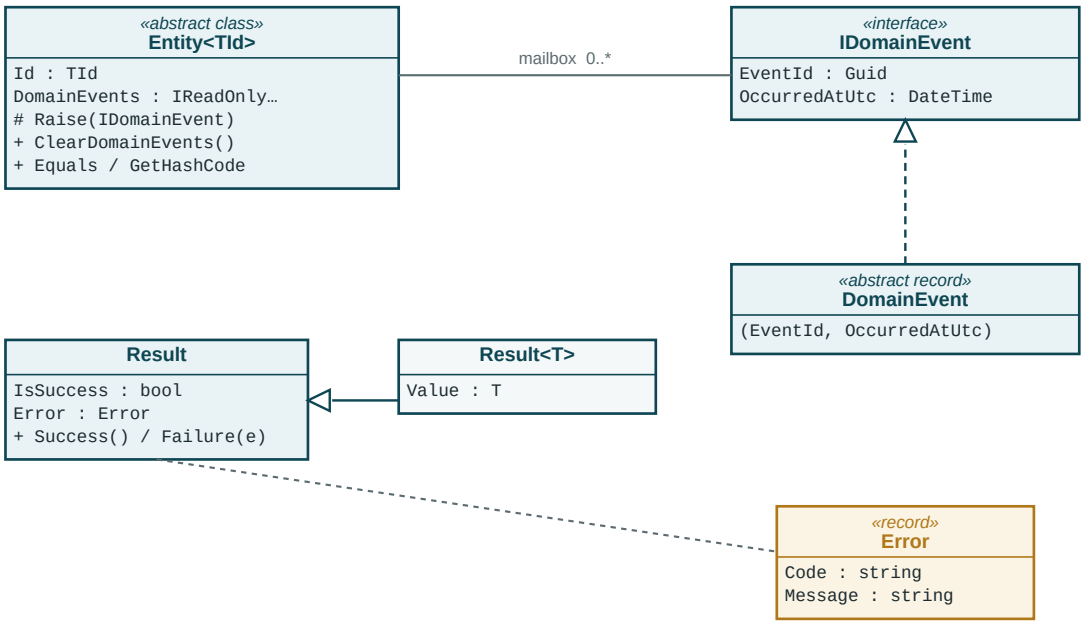


Figure D4.1 — UML class diagram of Nexus.SharedKernel. Dashed triangle: implements; solid triangle: inherits; plain line: holds.

The two channels, as a decision you can run

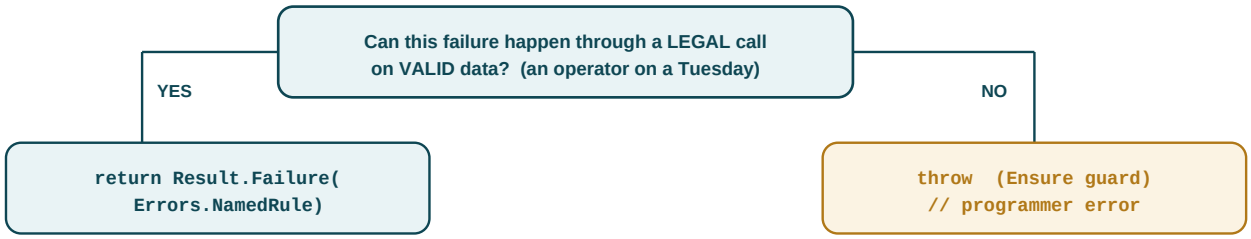


Figure D4.2 — The two-channel decision. Run it on every failure in the book; Chapters 6–17 never deviate from it.

Try the figure on three cases from later chapters before you meet them: an alarm acknowledged twice (legal call, valid data — left box, *AlarmManagement.NotRaised*); an identifier that is all zeros (no legal path produces one — right box, *Ensure.NotDefault*); reading *Value* off a failed *Result* (the caller ignored an answer it was handed — right box, and Listing 4.6 watches it throw). If you can route those three, you can route every failure in the book.

5 · Dependency Injection and Composition

One style, stated once: constructor injection

Every dependency in this codebase arrives through a constructor, and no other channel exists. No property injection, no static service locator, no ambient context reached through a global. The reason is the same one that runs through the whole book: **signatures must tell the truth.** A handler declaring *ctor(IRepository<RootCauseCase>, IUnitOfWork, IDateTimeProvider)* has published its complete needs; a class that quietly pulls a service from a locator has not, and every such lie is paid for in the test project, where the fake must be discovered by reading the body instead of the signature. Constructor injection is also what makes Chapter 19 cheap: testing a handler is *new*-ing it with fakes — no container required.

The doctrine: services are injected, entities are newed

The container manages *services* — handlers, domain services, pipeline behaviors, the eventual repositories. It never manages *entities*. An *AlarmEvent* is created by a factory method on its aggregate and rehydrated (in Volume II) by a repository; it is never resolved, never has dependencies, and never sees an interface to the outside world. This line is worth drawing in ink because the most common DI disease at scale is injecting services into entities — which drags the container into the innermost ring and quietly repeals the Dependency Rule. When domain logic needs a service, the *Application handler* receives the service and passes *values* into the domain (Chapter 10 shows the pattern on *CausalRankingService*).

Lifetimes, as far as Volume I needs them

Three lifetimes exist; this volume needs a working definition and one commitment. **Transient**: a new instance per resolution — the default for handlers and validators, which are cheap and stateless. **Scoped**: one instance per logical operation — reserved here, because “scope” only acquires its real meaning (one HTTP request, one DbContext, one transaction) in Volume II; nothing in Volume I is registered scoped, on purpose. **Singleton**: one instance forever — legal only for stateless, thread-safe services like *IDateTimeProvider*'s system-clock implementation. The commitment: no singleton may depend on anything with a shorter lifetime. Volume II inherits these choices and adds the scoped tier; the registrations written here do not change.

Each layer registers itself

Composition follows the same ownership rule as everything else: each layer publishes one extension method that registers its own services, and only the composition root calls them. The Domain project publishes nothing — it has no services to register in Volume I and no reference to any container package, keeping the innermost ring framework-free as promised. The Application layer's registration is printed complete:

```
namespace Nexus.Application;

public static class DependencyInjection
{
    public static IServiceCollection AddApplication(this IServiceCollection services)
    {
        services.AddMediatR(cfg =>
            cfg.RegisterServicesFromAssembly(typeof(DependencyInjection).Assembly));

        services.AddValidatorsFromAssembly(typeof(DependencyInjection).Assembly);

        // pipeline behaviors (validation, logging seam) are added in Chapter 16,
        // in this same method -- it is printed again there, grown.

        return services;
    }
}
```

Listing 5.1 — src/Nexus.Application/DependencyInjection.cs, complete.

A composition root with no application to compose

Now the oddity this volume cannot dodge. The composition root belongs by definition to the outermost layer — and Volume I has no outermost layer, no Program.cs, nothing to host. The honest resolution: **in Volume I, the test projects are the composition root.** They are the only executables in the solution, and the arrangement is a preview, not a trick — composing the system with fakes is exactly what an integration-test host does in Volume III:

```
namespace Nexus.Application.Tests;
public sealed class ApplicationFixture
{
    public IServiceProvider Provider { get; }

    public ApplicationFixture()
    {
        var services = new ServiceCollection();
        services.AddApplication();                      // the real registrations

        // every port declared in Chapter 12 meets a named fake here:
        services.AddSingleton(typeof(IRepository<>), typeof(InMemoryRepository<>));
        services.AddSingleton<IUnitOfWork, InMemoryUnitOfWork>();
        services.AddSingleton<IDateTimeProvider, FixedDateTimeProvider>();
        Provider = services.BuildServiceProvider(new ServiceProviderOptions
            { ValidateOnBuild = true });   // bad graph = failed test, not latent bug
    }
}
```

Listing 5.2 — tests/Nexus.Application.Tests/ApplicationFixture.cs, complete. The temporary composition root.

ValidateOnBuild, in the last listing, is the line to underline: it makes a missing registration a test-time explosion rather than a latent landmine — the watched-to-fail rule applied to wiring. When Volume II arrives, its Program.cs will call the same *AddApplication()* plus its own *AddInfrastructure()*, and the fixture keeps existing unchanged, swapping nothing but the fakes.

HONEST BOUNDARY

Listing 5.1 required *Microsoft.Extensions.DependencyInjection.Abstractions* inside Nexus.Application — the third third-party package to enter an inner layer, after MediatR and FluentValidation, and the concession should be stated rather than smoothed over: the “framework-free core” is precisely one project deep. The defense is scope — the abstractions package defines *IServiceCollection* and nothing else, is stable across the entire .NET ecosystem, and touching only one file (this one) keeps the exit cheap. A purist alternative exists: move registration out to the composition root entirely and keep Application package-free. NEXUS-1 declines it because it scatters each layer's wiring knowledge into whoever hosts it — but it is a defensible road, and Chapter 13 revisits the whole “packages in the core” ledger when MediatR itself goes on trial.

CHECKPOINT · CHAPTER 5

Before moving on, you should be able to answer these without looking back:

1. Why does this codebase permit exactly one injection style? Answer in terms of what a constructor signature does that a service locator cannot.

2. State the services/entities doctrine, and explain how injecting a repository into an entity would repeal the Dependency Rule.

3. Nothing in Volume I is registered scoped. Why is that a claim about meaning rather than about laziness — and which volume gives “scope” its meaning?

4. Where is Volume I's composition root, why is that placement honest rather than a workaround, and what does *ValidateOnBuild* convert a missing registration into?

5. Which three packages now live inside Nexus.Application, and what is the one-sentence defense of the newest one?

Part I, audited

The calibration habit closes each part by checking its own claims. Part I promised a skeleton in which every legal dependency is a declared reference and every illegal one fails to compile: delivered in Listings 2.1 and 3.2, and re-checkable by any reader with the Appendix-B skeleton. It promised a kernel of exactly five citizens with a freeze after Chapter 4: delivered, with the freeze on record for Volumes II–III to audit. It promised composition without a host, honestly named: delivered as the test-fixture root of Listing 5.2. Two concessions were priced along the way — three packages inside the Application project, and context boundaries held by convention rather than compiler. Both remain open items on the trilogy's ledger, revisited at Chapters 13 and 18 respectively. The ground is poured. Part II builds the plant on it: entities with identity, value objects with meaning, aggregates with invariants — beginning with what it means, precisely, for an object to be the same object after everything about it has changed.

What a Container Actually Does

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
> |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 2

AddApplication() exists with MediatR and validator scanning; the test fixture is appointed temporary composition root. The Domain remains service-free and registration-free — a fact the panel will keep asserting.

Someone has to call new

Constructor injection can sound circular to a newcomer: if every class demands its dependencies ready-made, who makes anything? The answer is deliberately boring — **ordinary code, in exactly one place.** A dependency-injection container is not magic and not a framework requirement; it is a dictionary of recipes ("when someone needs *IDateTimeProvider*, hand them this instance") plus a loop that reads constructor signatures via reflection and calls *new* with the recipes' results, recursively, until the requested object stands complete. You could write one in an afternoon — Chapter 13 nearly does, for a related job — and Chapter 19's tests will bypass it entirely and call *new* by hand, because for one object with three fakes, a dictionary is overhead. The container earns its keep only where graphs get deep and registrations get many, which is precisely the composition root and nowhere else.

Lifetimes, with household objects

Transient is the paper cup: a fresh one per request, discarded after, safe because never shared — right for handlers and validators, which are stateless and cheap. **Singleton** is the office coffee machine: one instance, shared by everyone, forever — legal only if it is stateless or genuinely thread-safe, which is why Chapter 10's judges qualify (read-only configuration, no memory between calls) and why the rule "no singleton may depend on anything shorter-lived" exists: a coffee machine holding a paper cup keeps the cup forever, long after it should have been thrown away. **Scoped** is the tray assembled per table order — one instance per logical operation — and Volume I registers nothing scoped because "operation" only acquires its real meaning (one HTTP request, one transaction) when Volume II supplies requests and transactions to mean it.

The recipe and the meal

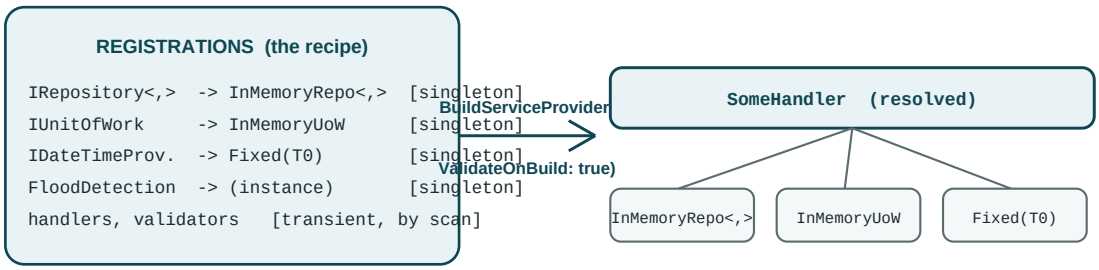


Figure D5.1 — Listing 5.2 as a picture: recipes in, validated object graphs out. The meal is cooked at resolution, not registration.

ValidateOnBuild now has a picture too: it cooks every registered meal once, at startup, so a missing ingredient explodes in the kitchen instead of at a customer's table — the difference between a failed test run and a latent production bug.

The same class, honest and dishonest

```
// constructor injection: the signature is the dependency manifest
public sealed class ShiftReport(IDateTimeProvider clock)
{
    public string Header() => $"Shift of {clock.UtcNow:yyyy-MM-dd}";
}

// service locator: same behavior, hidden manifest -- FORBIDDEN in this book
public sealed class ShiftReport
{
    public string Header() =>
        $"Shift of {Locator.Get<IDateTimeProvider>().UtcNow:yyyy-MM-dd}";
}
```

Deep-dive contrast — two ways to obtain a clock. The signatures differ; the second one lies.

Both compile; both work; only one can be tested by reading its first line. The locator version demands that every test know the class's *body* to know its needs, and that a global registry exist wherever the class travels — the two costs Chapter 5 summarized as "signatures must tell the truth." If you remember one sentence from this deep dive: **the container may only appear at the root; everywhere else, dependencies arrive by constructor and are visible from orbit.**

PART II

The Domain Layer: The Heart of the System

The skeleton stands and the kernel is frozen; now the plant itself moves in. Six chapters build the innermost ring: entities that stay themselves while everything about them changes, value objects that make wrong states unrepresentable, aggregates that draw the consistency lines, events that let contexts speak without touching, services for the rules that belong to no single object — and three full context walkthroughs to prove the patterns at NEXUS-1 scale. Nothing in this part references anything but the SharedKernel. That is not a constraint on the domain; it is the domain's privilege.

CHAPTERS 6–11 · ENTITIES · VALUE OBJECTS · AGGREGATES · EVENTS · SERVICES

6 · Entities — Objects with Identity

The same alarm, after everything about it has changed

At 02:14 an RCS pressure alarm is raised in Unit 2. At 02:15 an operator acknowledges it; the shift supervisor annotates it; at 02:40 the condition clears. Every property that can change has changed — status, acknowledger, timestamps — and yet the incident report, the regulator, and the root-cause engine all insist, correctly, that it was *one alarm* throughout. That insistence is the whole concept: **an entity is an object whose identity outlives its state.** Chapter 4 already encoded the consequence — *Entity<TId>* compares nothing but type and Id — and this chapter supplies the other half: what the state may do, and who is allowed to change it.

Rich, not anemic: the entity is the only writer of its own state

The failure mode this chapter exists to prevent has a name — the anemic domain model: entities as bags of public setters, with the rules that govern them scattered across whatever services happen to touch them. In NEXUS-1 the rule “only a raised alarm can be acknowledged” would then live wherever acknowledgement is called from — three screens, two jobs, and a migration script — and would eventually disagree with itself. The cure is structural: every setter is private, and the only way to change an entity is to ask it, through a method named in the ubiquitous language, which enforces its own rules and answers with a *Result*. First, the vocabulary of the alarm's small world:

```
namespace Nexus.Domain.AlarmManagement;

public enum AlarmSeverity { Advisory = 1, Warning = 2, Critical = 3 }

public enum AlarmStatus   { Raised = 1, Acknowledged = 2, Cleared = 3 }

public static class AlarmErrors
{
    public static readonly Error NotRaised = new(
        "AlarmManagement.NotRaised", "Only a raised alarm can be acknowledged.");

    public static readonly Error NotAcknowledged = new(
        "AlarmManagement.NotAcknowledged", "Only an acknowledged alarm can be cleared.");
}
```

Listing 6.1 — AlarmManagement vocabulary: lifecycle, severity, and the context's named rules.

Errors are declared once, next to the rules they name, with codes in the *Context.Rule* convention from Chapter 4 — so a failure carries its own address all the way to whatever surface eventually reports it.

Construction: guarded, private, and spoken in the domain's tense

An alarm is not “new-ed”; in the control room it is *raised*, and the code says so. The constructor is private and does the dumb structural checks (guards, Chapter 4's exception channel); the static factory speaks the ubiquitous language, decides the initial state, and drops the birth announcement into the event mailbox. The private constructor also quietly reserves a seam: it is exactly what Volume II's rehydration path will use, so persistence arrives later without a single signature changing here.

```
namespace Nexus.Domain.AlarmManagement;

public sealed class AlarmEvent : Entity<AlarmEventId>
{
    private AlarmEvent(AlarmEventId id, UnitId unitId, TagId tagId,
                       AlarmSeverity severity, string message, DateTime raisedAtUtc)
        : base(id)
    {
        Ensure.NotDefault(unitId, nameof(unitId));
        Ensure.NotDefault(tagId, nameof(tagId));
        Ensure.NotEmpty(message, nameof(message));

        UnitId = unitId;  TagId = tagId;  Severity = severity;
        Message = message;  RaisedAtUtc = raisedAtUtc;
        Status = AlarmStatus.Raised;
    }

    public UnitId UnitId { get; }
    public TagId TagId { get; }
    public AlarmSeverity Severity { get; }
    public string Message { get; }
    public DateTime RaisedAtUtc { get; }
    public AlarmStatus Status { get; private set; }
    public OperatorId? AcknowledgedBy { get; private set; }
    public DateTime? AcknowledgedAtUtc { get; private set; }
    public DateTime? ClearedAtUtc { get; private set; }

    public static AlarmEvent Raise(UnitId unitId, TagId tagId, AlarmSeverity severity,
                                   string message, DateTime raisedAtUtc)
    {
        var alarm = new AlarmEvent(AlarmEventId.New(), unitId, tagId,
                                   severity, message, raisedAtUtc);
        alarm.Raise(new AlarmRaisedEvent(Guid.NewGuid(), raisedAtUtc,
                                         alarm.Id, unitId, severity));
        return alarm;
    }
    // behavior continues in Listing 6.3
```

Listing 6.2 — src/Nexus.Domain/AlarmManagement/AlarmEvent.cs, part 1: state, construction, factory.

Two details are load-bearing. *UnitId*, *TagId*, and *OperatorId* are declared in *Nexus.Domain.CorePlatform*, not here — which sharpens Chapter 3's convention into its final form: **cross-context references are by identifier only.** Holding another context's ID is pointing at it; holding its entity would be reaching into it. And *raisedAtUtc* arrives as a parameter — no *DateTime.UtcNow* appears anywhere in the Domain layer, ever. Time is an input owned by *IDateTimeProvider* at the Application seam (Chapter 12), which is why every timestamp in Chapter 18's tests can be exact instead of approximately-now.

Behavior: the lifecycle as methods that can refuse

```
    public Result Acknowledge(OperatorId by, DateTime atUtc)
    {
        if (Status != AlarmStatus.Raised)
            return Result.Failure(AlarmErrors.NotRaised);

        Status = AlarmStatus.Acknowledged;
        AcknowledgedBy = by;
        AcknowledgedAtUtc = atUtc;
        Raise(new AlarmAcknowledgedEvent(Guid.NewGuid(), atUtc, Id, by));
        return Result.Success();
    }

    public Result Clear(DateTime atUtc)
    {
        if (Status != AlarmStatus.Acknowledged)
            return Result.Failure(AlarmErrors.NotAcknowledged);

        Status = AlarmStatus.Cleared;
        ClearedAtUtc = atUtc;
        return Result.Success();
    }
}
```

Listing 6.3 — AlarmEvent.cs, part 2: the state machine, enforcing its own rules.

Read *Acknowledge* as a sentence and the pattern of the entire Domain layer is visible: check the rule, refuse by name if it fails, mutate privately, leave evidence in the mailbox, answer. Nothing outside this file can put an alarm into an illegal state, because nothing outside this file can put an alarm into *any* state — the setters are private, and the compiler enforces that the way it enforces everything else in this book. The events the lifecycle produces are the context's public biography:

```
namespace Nexus.Domain.AlarmManagement.Events;

public sealed record AlarmRaisedEvent(Guid EventId, DateTime OccurredAtUtc,
    AlarmEventId AlarmId, UnitId UnitId, AlarmSeverity Severity)
    : DomainEvent(EventId, OccurredAtUtc);

public sealed record AlarmAcknowledgedEvent(Guid EventId, DateTime OccurredAtUtc,
    AlarmEventId AlarmId, OperatorId By) : DomainEvent(EventId, OccurredAtUtc);
```

Listing 6.4 — src/Nexus.Domain/AlarmManagement/Events/, complete.

Past tense, sealed, immutable, and carrying IDs rather than entities — everything Chapter 9 will require of events is already true of these two. Note also what *Clear* does not do: it raises no event, yet. Whether clearance matters to anyone outside this context is a question Chapter 9 answers with a subscriber in hand, not a guess made here.

The lifecycle, pinned — and violated on schedule

Three tests close the loop: one pins the happy path, one is the watched-to-fail — it commits the exact crime the rule forbids and asserts the refusal arrives with the right name — and one confirms the mailbox holds the birth announcement. Notice what the tests do *not* need: no container, no fixture, no fake. Entities are newed, as Chapter 5 promised, and testing the innermost ring is plain C# all the way down.

```
namespace Nexus.Domain.Tests.AlarmManagement;

public sealed class AlarmEventTests
{
    private static readonly DateTime T0 =
        new(2026, 7, 2, 2, 14, 0, DateTimeKind.Utc);

    private static AlarmEvent NewAlarm() => AlarmEvent.Raise(
        UnitId.New(), TagId.New(), AlarmSeverity.Critical,
        "RCS pressure high", T0);

    [Fact]
    public void A_raised_alarm_acknowledges_exactly_once()
    {
        var alarm = NewAlarm();
        Assert.True(alarm.Acknowledge(OperatorId.New(), T0.AddMinutes(1)).IsSuccess);
        Assert.Equal(AlarmStatus.Acknowledged, alarm.Status);
    }

    [Fact]
    public void Acknowledging_twice_is_refused_by_name()      // watched to fail
    {
        var alarm = NewAlarm();
        alarm.Acknowledge(OperatorId.New(), T0.AddMinutes(1));
        var second = alarm.Acknowledge(OperatorId.New(), T0.AddMinutes(2));
        Assert.True(second.IsFailure);
        Assert.Equal("AlarmManagement.NotRaised", second.Error.Code);
    }

    [Fact]
    public void Raising_leaves_the_birth_announcement_in_the_mailbox()
    {
        var evt = Assert.Single(NewAlarm().DomainEvents);
        Assert.IsType<AlarmRaisedEvent>(evt);
    }
}

$ dotnet test tests/Nexus.Domain.Tests
Passed! - Failed: 0, Passed: 5, Skipped: 0, Duration: 63 ms
```

Listing 6.5 — tests/Nexus.Domain.Tests/AlarmManagement/AlarmEventTests.cs, and the run.

> **SEE IT IN NEXUS-1 · The list this class becomes**
>
> The console's **Alarms & Events** screen is a scrolling table of exactly these objects: severity chips, RAISED/ACK'D/CLEARED badges, an acknowledge action per row. Every badge transition the screen animates is one of Listing 6.3's methods succeeding — and every grayed-out acknowledge button is *AlarmErrors.NotRaised* rendered as UI.

HONEST BOUNDARY

Three simplifications are on the record. **The lifecycle is a teaching cut:** *From Domain to Twin* models the fuller alarm state machine (shelving, return-to-normal before acknowledgement, re-alarm) in the spirit of industry alarm-management practice; this chapter's three-state machine is its smallest honest subset, and Chapter 11's walkthrough restores the missing transitions. **Concurrency is deferred, not solved:** nothing here stops two processes acknowledging the same alarm in parallel — in-process, entities are not thread-safe by design, and the cross-process answer is optimistic concurrency at the persistence seam, owned by Volume II. **The state is flat:** three nullable properties tracking acknowledgement is tolerable at three fields and degrades from there; Chapter 7 shows the value-object cure (*Acknowledgement* as one type) and prices when it is worth taking.

CHECKPOINT · CHAPTER 6

Before moving on, you should be able to answer these without looking back:

1. Define an entity in one sentence, and explain why the 02:14 alarm remains “the same alarm” at 02:40 in terms Chapter 4's equality already enforces.

2. What is an anemic domain model, and which two compiler-visible features of Listing 6.2 make it impossible here?

3. The constructor is private and the factory is named *Raise*. Give the two jobs of each — and name the Volume-II mechanism the private constructor is quietly reserved for.

4. State the sharpened cross-context rule, and classify: holding a *UnitId*; holding a *Unit*; subscribing to *AlarmRaisedEvent* from RootCause.

5. Why does no *DateTime.UtcNow* appear in the Domain layer, and what does that buy Listing 6.5 concretely?

The alarm's identity is settled; its *data* is not yet honest. “RCS pressure high” hides a measurement, a unit, and a quality flag inside a string, and three nullable properties are pretending to be one concept called an acknowledgement. The next chapter builds the type that fixes both: the value object — meaning without identity, and the workhorse of the NEXUS-1 domain.

Reading an Entity Like a Mechanic

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
> |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 5

First real domain citizen: AlarmManagement now holds AlarmEvent, its enums, its errors, and its two events. Three new tests pin the lifecycle; the kernel's two still stand beneath them.

private set: who may write, spelled out

public AlarmStatus Status { get; private set; } splits one property into two permissions: anyone may *read* Status, but the *set* half is private — callable only from code inside the AlarmEvent class itself. That single keyword is the entire enforcement of "the entity is the only writer of its own state": a handler, a test, or a future controller that types *alarm.Status = Cleared* gets compiler error CS0272 before the thought is finished. Properties with no setter at all (*UnitId*, *Message*) go further — assignable only in the constructor, immutable for the object's whole life. The pattern to internalize: immutable facts get no setter; mutable state gets a private one; nothing ever gets a public one.

The question mark, and what null is allowed to mean

OperatorId? is C#'s nullable annotation: this value may legitimately be absent, and the compiler will nag anyone who touches it without checking. The book's discipline is stricter than the syntax: **null is permitted only where absence is a meaningful domain state** — "not yet acknowledged" — never as a lazy default. Chapter 7 will show what happens when several nullables try to describe one concept together (they start telling contradictory stories) and cure it; for now, read every *?* in a listing as a deliberate sentence: "there is a moment in this object's legal life when this is genuinely not there yet."

Anatomy of a factory call

AlarmEvent.Raise(unitId, tagId, severity, message, raisedAtUtc) is a **static factory method**: static because you call it on the type (no instance exists yet to call anything on), a factory because its job is manufacturing. Inside, it runs the private constructor (structural guards), sets the initial state (Status = Raised — a decision, not a default), and drops the birth announcement into the mailbox before handing the object over. Three responsibilities, one named verb from the control room. When Chapter 14's handlers call these factories, notice they never add a fourth responsibility — the factory is complete or it is wrong.

The lifecycle as a UML state machine

A state machine diagram shows every legal life an object can live: rounded boxes are states, arrows are the only permitted transitions, and — this book's addition — the red dashed loops are the *refusals*: attempted transitions that bounce off, each answering with its published error code. If a change of Status is not an arrow here, Listing 6.3 has no code path that performs it.

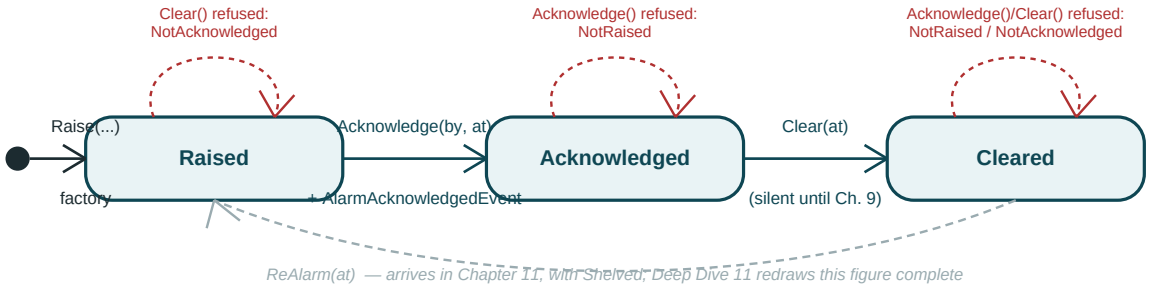


Figure D6.1 — UML state machine of AlarmEvent as of Chapter 6. Red dashed loops are refusals, labeled with their published codes.

Walk the figure against Listing 6.3 once, slowly. Every solid arrow corresponds to one method whose first lines check that the arrow's tail matches the current state; every red loop is that check failing and answering through the Result channel — state unchanged, which is why the loop returns to the same box. The diagram also makes one absence visible that prose buried: nothing leaves *Cleared*. As of this chapter, a cleared alarm is terminal — a simplification the boundary box priced and Chapter 11 repairs, exactly along the gray curve. When you can reproduce this figure from the code, or the code from this figure, the chapter has done its work: **an entity's methods are its state machine, written in the only notation that compiles.**

7 · Value Objects — Meaning Without Identity

Two smells, one missing concept

Chapter 6 closed owing the reader two repairs. “RCS pressure high” smuggles a measurement, a unit, and a trust judgement inside a string — data wearing a costume. And the acknowledgement lives as two nullable properties that the type system happily lets disagree: nothing stops *AcknowledgedBy* being set while *AcknowledgedAtUtc* stays null, a state that means nothing in any control room on earth. Both smells have the same cure: a kind of object that is its data — compared by it, immutable in it, and impossible to construct in a meaningless configuration.

What a value object is

A value object has no identity because it needs none. Two readings of 155.2 bar, Good quality, are not “equal but distinct” — they are interchangeable, the way two 7s are. From that one fact everything follows: equality is by value (identity would compare nothing real); immutability is mandatory (you do not change the number 7, you use a different number); and validity is a property of construction (a reading of NaN bar is not a broken reading — it is not a reading). Entities are the plant's *things*; value objects are its *measurements, judgements, and descriptions* — and in a 654-table system the second population is far larger, which is why this is the workhorse chapter of Part II.

The pattern, once — then used everywhere

Chapter 4 refused a *ValueObject* base class and promised the case here. In .NET 8 a *sealed record* supplies value equality and *GetHashCode* mechanically; what it does not supply — a guarded front door — the pattern adds by hand:

```
public sealed record SomeValue
{
    private SomeValue(/* fields */) { /* assign only */ }   // no public door

    public T SomeField { get; }        // get-only, NOT init: see Listing 7.4

    public static Result<SomeValue> Create(/* raw input */)
        => /* validate */ ? Result.Success(new SomeValue(...))
                           : Result.Failure<SomeValue>(SomeErrors.Named);
}
```

Listing 7.1 — The NEXUS-1 value-object pattern. Every value object in the book has this silhouette.

EngineeringValue, finally — and in the right address

Chapter 4 convicted this type of business meaning and deported it from the kernel; here it is at its lawful residence, *Nexus.Domain.Instrumentation*, carrying the three facts a naked double loses: magnitude, unit, and how much the signal deserves to be trusted.

```
namespace Nexus.Domain.Instrumentation;

public enum EngineeringUnit { Bar = 1, Celsius = 2, PercentPower = 3, LitersPerSecond = 4 }

public enum SignalQuality  { Good = 1, Uncertain = 2, Bad = 3 }

public static class InstrumentationErrors
{
    public static readonly Error NotFinite = new(
        "Instrumentation.NotFinite", "An engineering value must be a finite number.");
    public static readonly Error UnitMismatch = new(
        "Instrumentation.UnitMismatch", "Values in different units cannot be compared.");
}

public sealed record EngineeringValue
{
    private EngineeringValue(double value, EngineeringUnit unit, SignalQuality quality)
    {
        Value = value;  Unit = unit;  Quality = quality;
    }

    public double Value { get; }
    public EngineeringUnit Unit { get; }
    public SignalQuality Quality { get; }

    public static Result<EngineeringValue> Create(
        double value, EngineeringUnit unit, SignalQuality quality = SignalQuality.Good)
        => double.IsFinite(value)
            ? Result.Success(new EngineeringValue(value, unit, quality))
            : Result.Failure<EngineeringValue>(InstrumentationErrors.NotFinite);

    public Result<bool> Exceeds(EngineeringValue threshold)
        => Unit == threshold.Unit
            ? Result.Success(Value > threshold.Value)
            : Result.Failure<bool>(InstrumentationErrors.UnitMismatch);

    public EngineeringValue Degrade(SignalQuality to) => new(Value, Unit, to);

    public override string ToString() => $"{Value:G6} {Unit} [{Quality}]";
}
```

Listing 7.2 — src/Nexus.Domain/Instrumentation/EngineeringValue.cs, complete (unit set abridged for print).

Two methods carry the philosophy. *Exceeds* refuses to compare bars with degrees — the unit mismatch that a naked double makes silently and a value object makes impossible — and it refuses through the ordinary *Result* channel, by name. *Degrade* shows how value objects “change”: they do not. It returns a *new* value, the old one untouched, exactly as arithmetic does.

Why get-only and not init: locking the side door

Records ship with a side door: the *with*-expression, which clones a record and rewrites chosen properties — *bypassing every factory ever written*, but only if the properties are declared *init*. Listing 7.1 declares them get-only instead, and per the watched-to-fail rule, here is the attempted break-in and the refusal:

```
var reading = EngineeringValue.Create(155.2, EngineeringUnit.Bar).Value;

var forged = reading with { Value = double.NaN };   // around the factory?

error CS0200: Property or indexer 'EngineeringValue.Value' cannot be
  assigned to -- it is read only
```

Listing 7.3 — The side door, found locked. Validation cannot be bypassed because mutation cannot be expressed.

The cost is stated with the benefit: giving up *with* means every derived value needs a named method like *Degrade*. In a domain where “derive a variant” should be a deliberate, nameable act, that cost is the feature.

The cure for the three pretenders

```
namespace Nexus.Domain.AlarmManagement;

public sealed record Acknowledgement
{
    private Acknowledgement(OperatorId by, DateTime atUtc)
    {
        By = by;  AtUtc = atUtc;
    }

    public OperatorId By { get; }
    public DateTime AtUtc { get; }
    public static Acknowledgement Of(OperatorId by, DateTime atUtc)
    {
        Ensure.NotDefault(by, nameof(by));       // guard, not Result: see below
        return new Acknowledgement(by, atUtc);
    }
}
```

Listing 7.4 — src/Nexus.Domain/AlarmManagement/Acknowledgement.cs, complete.

```
    // state: the two pretenders become one value object
    public OperatorId? AcknowledgedBy { get; private set; }
    public DateTime? AcknowledgedAtUtc { get; private set; }
    public Acknowledgement? Acknowledgement { get; private set; }

    // inside Acknowledge(by, atUtc), after the rule check:
    AcknowledgedBy = by;
    AcknowledgedAtUtc = atUtc;
    Acknowledgement = Acknowledgement.Of(by, atUtc);
```

Listing 7.5 — AlarmEvent.cs, revised: one concept replaces two properties that could disagree.

The remaining *null* now means exactly one thing — *not yet acknowledged* — instead of four combinations meaning one thing, nothing, and two absurdities. And why does *Of* guard rather than return *Result*? A default *OperatorId* has no legal path — programmer error, exception channel — whereas NaN arrives from real sensors on real Tuesdays, and gets the value channel.

Value semantics, pinned — and two named refusals

```
namespace Nexus.Domain.Tests.Instrumentation;

public sealed class EngineeringValueTests
{
    [Fact]
    public void Two_readings_of_the_same_measurement_are_interchangeable()
    {
        var a = EngineeringValue.Create(155.2, EngineeringUnit.Bar).Value;
        var b = EngineeringValue.Create(155.2, EngineeringUnit.Bar).Value;
        Assert.Equal(a, b);              // value equality, free from record
        Assert.True(a == b);             // operators included
    }

    [Fact]
    public void A_reading_that_is_not_a_number_is_not_a_reading()   // watched to fail
    {
        var result = EngineeringValue.Create(double.NaN, EngineeringUnit.Bar);
        Assert.True(result.IsFailure);
        Assert.Equal("Instrumentation.NotFinite", result.Error.Code);
    }

    [Fact]
    public void Bars_refuse_comparison_with_degrees()               // watched to fail
    {
        var p = EngineeringValue.Create(155.2, EngineeringUnit.Bar).Value;
        var t = EngineeringValue.Create(310.0, EngineeringUnit.Celsius).Value;
        Assert.Equal("Instrumentation.UnitMismatch", p.Exceeds(t).Error.Code);
    }
}

$ dotnet test tests/Nexus.Domain.Tests
Passed! - Failed: 0, Passed: 8, Skipped: 0, Duration: 71 ms
```

Listing 7.6 — tests/Nexus.Domain.Tests/Instrumentation/EngineeringValueTests.cs, and the run.

> **SEE IT IN NEXUS-1 · Value objects are the whole dashboard**
>
> Every gauge on the console's **Reactor Overview** is an *EngineeringValue* rendered: the number, the unit suffix, and the quality shown as the needle's color state. When a sensor drops to *Uncertain*, the console grays the gauge rather than showing a confident lie — that is *Degrade* as pixels, and the reason Quality travels inside the value instead of alongside it.

HONEST BOUNDARY

Three prices on the record. **Floating-point equality is bitwise:** record equality on a double compares bits, so two readings *computed* by different routes may differ in the last bit and count as unequal. NEXUS-1 tolerates this because equality here means “the same recorded measurement”, not “close enough” — tolerance is a question for domain services (Chapter 10), never for *Equals*. **The unit system is a teaching subset:** four units, no conversions, no dimensional analysis. The real discipline lives in *From Grid to Core*; a production system would grow a conversion matrix or adopt a units library, and Chapter 11's Instrumentation walkthrough restores part of that depth. **Records are a .NET choice, not a DDD law:** the pattern works in any language with less sugar; what is doctrine is the guarded factory and the immutability, not the keyword.

CHECKPOINT · CHAPTER 7

Before moving on, you should be able to answer these without looking back:

1. Why do two readings of 155.2 bar need no identity, and what three obligations follow from that single fact?

2. Recite the pattern's silhouette: what is private, what is get-only, what is static — and which compiler error guards the side door, against what attack?

3. *Exceeds* returns *Result<bool>* while *Acknowledgement.Of* throws. Assign each choice to its failure channel and justify both.

4. After Listing 7.5, what does *Acknowledgement == null* mean — and how many meaningless states did the refactor delete?

5. Why does *Quality* live inside *EngineeringValue* rather than as a separate property beside it? Answer with the console's gray gauge in hand.

Entities carry identity; value objects carry meaning. What neither can do alone is guarantee consistency across a *cluster* — an alarm and its annotations, a root-cause case and its evidence — where the rule spans several objects and a half-applied change is corruption. Drawing those cluster boundaries, and policing every door through them, is the aggregate's job, and the next chapter's.

Immutability, Null, and Unrepresentable States

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
> |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 8

Instrumentation joins AlarmManagement inside Nexus.Domain: EngineeringValue and Acknowledgement exist, the alarm's nullable pretenders are gone, and three value-semantics tests bring the suite to eight.

Why immutability is not a sacrifice

Newcomers often hear "immutable" as "inconvenient" — you must build a new object just to change a field. Reverse the lens: an object that cannot change is an object you can hand to anyone, cache anywhere, use as a dictionary key, and share across threads **without a single defensive thought**, because nothing anyone does to their copy can surprise you in yours. That is why *Degrade* returns a new value instead of editing the old: the reading you logged at 02:14 stays the reading you logged, forever, even after the live gauge has moved on. Mutability is a sharp tool for identity-bearing things whose change we track (entities, behind private setters); for measurements and judgements, immutability is not the price of correctness — it *is* the correctness.

The credo, spelled out: make illegal states unrepresentable

There are two ways to keep bad states out of a system. **Validate:** allow the type to represent nonsense, then check for nonsense at every border and hope no border was forgotten. **Design:** shape the type so the nonsense has no representation at all — no constructor produces it, no property combination spells it, no code path needs to check for it because it cannot arrive. This chapter is the design strategy applied twice: NaN-bar is not a rejected reading but a *non-reading* (the factory returns Failure; no EngineeringValue with NaN ever exists), and "acknowledged by someone at no particular time" is not a bug to test for but a sentence the revised AlarmEvent can no longer say. The figures overleaf draw both moves.

Figure one: the anatomy of a guarded value object

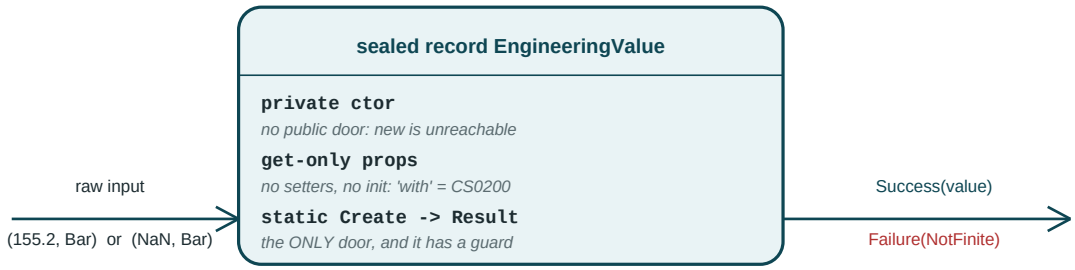


Figure D7.1 — The pattern of Listing 7.1 as a machine: one guarded door in, two honest answers out, no other entrance.

Figure two: the state space, before and after

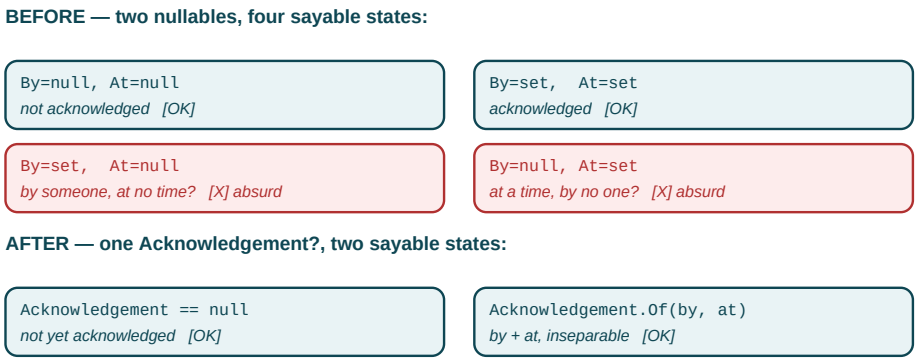


Figure D7.2 — Listing 7.5 as a state-space diet: the refactor deleted the two red cells from the language itself.

The is-it-a-value-object checklist, to run on any new concept: *Would two instances with identical contents be interchangeable? Does it need a history, or only a present? Would anyone ever ask "which one" rather than "what value"?* Interchangeable, present-only, what-not-which — value object; any other answers — entity, and Chapter 6's rules apply instead. Every type in Parts II and III passed through exactly this gate.

8 · Aggregates and Consistency Boundaries

A rule that no single object can see

At 03:00, following the alarm flood of Chapter 6, a root-cause case is opened for Unit 2. By 04:30 an investigator is ready to close it — confirmed cause, condenser vacuum loss — while, in the next session over, a colleague is pruning what she believes is a duplicate evidence entry. Interleave those two operations naively and the system commits a lie: a case whose record says *closed, with evidence* and whose evidence table says *empty*. Neither object misbehaved. The case checked its status; the evidence row deleted cleanly. The corruption lives *between* them, because the rule — **a case cannot close without evidence** — is a property of the cluster, and Chapters 6 and 7 built guards that can only see one object at a time.

The aggregate: a boundary you can hold in one hand

An aggregate is a cluster of entities and value objects that changes as one unit, guarded by a single entity — the **aggregate root** — which is the cluster's only door. Everything follows from the door. Outside code holds a reference to the root and never to its internals; every mutation is a method on the root, so every invariant that spans the cluster has exactly one place to stand; and the boundary of the cluster *is* the boundary of the transaction, because “changes as one unit” is a promise about persistence as much as about objects. The 04:30 corruption becomes impossible not because anyone got more careful, but because *delete evidence* and *close case* are now the same object's methods, serialized through the same door, checked against the same in-memory truth.

Three rules, each with its reason

Reference other aggregates by ID only — Chapter 6's convention, now with its deepest justification: an object reference is a claim of consistency (“what I hold is current”) that no one can honor across a boundary; an ID is an honest pointer. **One aggregate per transaction** — inside the boundary, consistency is immediate and transactional; between boundaries it is eventual, carried by the domain events of Chapter 9. Wanting two aggregates in one transaction is usually the design saying the boundary is drawn wrong. **Keep aggregates small** — every object inside the boundary is loaded to change any of it and locked to protect all of it; size is paid for on every operation. The cluster this chapter builds is the series' flagship, straight from *From Flood to Cause*: the root-cause case and its evidence.

Inside the boundary: the child entity

Evidence is an *entity* — it has identity (a specific finding, attached at a specific moment, from a specific signal) — but it is a *child*: it lives and dies inside the case's boundary, is created only by the root, and is never fetched, referenced, or repaired from outside. No repository will ever exist for it; Chapter 12's ports traffic in aggregate roots only.

```
namespace Nexus.Domain.RootCause;

public static class RootCauseErrors
{
    public static readonly Error CaseWithoutEvidence = new(
        "RootCause.CaseWithoutEvidence", "A case cannot close without evidence.");

    public static readonly Error CaseAlreadyClosed = new(
        "RootCause.CaseAlreadyClosed", "A closed case is a sealed record.");
}

public sealed class Evidence : Entity<EvidenceId>
{
    internal Evidence(EvidenceId id, string description, TagId source,
                      DateTime attachedAtUtc) : base(id)
    {
        Ensure.NotEmpty(description, nameof(description));
        Description = description;  Source = source;  AttachedAtUtc = attachedAtUtc;
    }

    public string Description { get; }
    public TagId Source { get; }
    public DateTime AttachedAtUtc { get; }
}
```

Listing 8.1 — src/Nexus.Domain/RootCause/: the context's named rules and the child entity.

The constructor is *internal*, and the choice deserves its footnote now rather than in the boundary box: C# has no “aggregate-private” visibility, so *internal* — reachable anywhere in Nexus.Domain — is the tightest fence the language sells. The true discipline, “only the root constructs its children”, is held the way the context boundary of Chapter 3 is held: by convention, visible in every listing. Note also that Evidence, once attached, is immutable — every property get-only. Corrections in this domain are made by attaching better evidence, not by editing history; the plant's paper trail works the same way.

The root, part one: state and the guarded collection

```
namespace Nexus.Domain.RootCause;

public enum CaseStatus { Open = 1, Closed = 2 }

public sealed class RootCauseCase : Entity<RootCauseCaseId>
{
    private readonly List<Evidence> _evidence = [];

    private RootCauseCase(RootCauseCaseId id, UnitId unitId,
                          AlarmEventId triggeredBy, DateTime openedAtUtc) : base(id)
    {
        Ensure.NotDefault(unitId, nameof(unitId));
        UnitId = unitId;  TriggeredBy = triggeredBy;  OpenedAtUtc = openedAtUtc;
        Status = CaseStatus.Open;
    }

    public UnitId UnitId { get; }
    public AlarmEventId TriggeredBy { get; }       // another aggregate: ID only
    public DateTime OpenedAtUtc { get; }
    public CaseStatus Status { get; private set; }
    public string? ConfirmedCause { get; private set; }
    public DateTime? ClosedAtUtc { get; private set; }

    public IReadOnlyList<Evidence> Evidence => _evidence.AsReadOnly();

    public static RootCauseCase Open(UnitId unitId, AlarmEventId triggeredBy,
                                     DateTime atUtc)
    {
        var rcc = new RootCauseCase(RootCauseCaseId.New(), unitId, triggeredBy, atUtc);
        rcc.Raise(new CaseOpenedEvent(Guid.NewGuid(), atUtc,
                                      rcc.Id, unitId, triggeredBy));
        return rcc;
    }
```

Listing 8.2 — src/Nexus.Domain/RootCause/RootCauseCase.cs, part 1: state, construction, and the only door.

The two lines that make the aggregate an aggregate are the field and the property around it. The *List* is private; the world sees an *IReadOnlyList*, which has no *Add* — so *theCase.Evidence.Add(...)* is not a caught mistake but an unwritable sentence (CS1061, the compiler doing door duty again). And *TriggeredBy* practices rule one in plain sight: the case points at the alarm that spawned it with an ID, claiming a fact, not custody.

The root, part two: every change through the door

```
    public Result AttachEvidence(string description, TagId source, DateTime atUtc)
    {
        if (Status == CaseStatus.Closed)
            return Result.Failure(RootCauseErrors.CaseAlreadyClosed);

        _evidence.Add(new Evidence(EvidenceId.New(), description, source, atUtc));
        return Result.Success();
    }

    public Result Close(string confirmedCause, DateTime atUtc)
    {
        if (Status == CaseStatus.Closed)
            return Result.Failure(RootCauseErrors.CaseAlreadyClosed);

        if (_evidence.Count == 0)
            return Result.Failure(RootCauseErrors.CaseWithoutEvidence);   // the flagship

        Ensure.NotEmpty(confirmedCause, nameof(confirmedCause));

        Status = CaseStatus.Closed;
        ConfirmedCause = confirmedCause;
        ClosedAtUtc = atUtc;
        Raise(new CaseClosedEvent(Guid.NewGuid(), atUtc, Id, UnitId,
                                  confirmedCause, _evidence.Count));
        return Result.Success();
    }
}

// Events/CaseEvents.cs, complete:
public sealed record CaseOpenedEvent(Guid EventId, DateTime OccurredAtUtc,
    RootCauseCaseId CaseId, UnitId UnitId, AlarmEventId TriggeredBy)
    : DomainEvent(EventId, OccurredAtUtc);

public sealed record CaseClosedEvent(Guid EventId, DateTime OccurredAtUtc,
    RootCauseCaseId CaseId, UnitId UnitId, string ConfirmedCause, int EvidenceCount)
    : DomainEvent(EventId, OccurredAtUtc);
```

Listing 8.3 — RootCauseCase.cs, part 2: the flagship invariant, standing where it can see the whole cluster.

Read *Close* against the 04:30 story. The check and the mutation touch the same private list, in the same method, on the same object — there is no gap for a concurrent delete to slip through, because delete is *also* this object's method (an exercise in Chapter 11 adds *StrikeEvidence*, which must refuse to strike the last evidence of an open case — the flagship invariant seen from the other side). *CaseClosedEvent* carries IDs and facts, never the case itself: it is the announcement that lets ReinforcementLearning and the reporting stack react in *their* transactions, eventually — rule two, kept.

The flagship, watched to fail

```
namespace Nexus.Domain.Tests.RootCause;

public sealed class RootCauseCaseTests
{
    private static readonly DateTime T0 = new(2026, 7, 2, 3, 0, 0, DateTimeKind.Utc);

    private static RootCauseCase OpenCase() =>
        RootCauseCase.Open(UnitId.New(), AlarmEventId.New(), T0);

    [Fact]
    public void A_case_cannot_close_without_evidence()          // the flagship, failing
    {
        var rcc = OpenCase();
        var result = rcc.Close("Condenser vacuum loss", T0.AddHours(2));

        Assert.Equal("RootCause.CaseWithoutEvidence", result.Error.Code);
        Assert.Equal(CaseStatus.Open, rcc.Status);      // and nothing half-applied
        Assert.Null(rcc.ConfirmedCause);
    }

    [Fact]
    public void With_evidence_on_file_the_case_closes_and_testifies()
    {
        var rcc = OpenCase();
        rcc.AttachEvidence("Hotwell level trend inverted 02:41", TagId.New(),
                           T0.AddMinutes(30));
        Assert.True(rcc.Close("Condenser vacuum loss", T0.AddHours(2)).IsSuccess);

        var evt = Assert.IsType<CaseClosedEvent>(rcc.DomainEvents[^1]);
        Assert.Equal(1, evt.EvidenceCount);
    }

    [Fact]
    public void A_closed_case_is_a_sealed_record()              // watched to fail
    {
        var rcc = OpenCase();
        rcc.AttachEvidence("...", TagId.New(), T0.AddMinutes(30));
        rcc.Close("Condenser vacuum loss", T0.AddHours(2));

        Assert.Equal("RootCause.CaseAlreadyClosed",
            rcc.AttachEvidence("late", TagId.New(), T0.AddHours(3)).Error.Code);
    }
}

$ dotnet test tests/Nexus.Domain.Tests
Passed! - Failed: 0, Passed: 11, Skipped: 0, Duration: 84 ms
```

Listing 8.4 — tests/Nexus.Domain.Tests/RootCause/RootCauseCaseTests.cs, and the run.

> **SEE IT IN NEXUS-1 · The graph behind the door**
>
> The console's **Root Cause Graph** renders exactly this cluster: the case node at center, evidence nodes linked around it, the confirmed cause highlighted when the case closes. The CLOSE button stays disabled until at least one evidence node exists — the flagship invariant, enforced here in Listing 8.3, drawn there as a grayed-out control.

HONEST BOUNDARY

Four deferrals and concessions, on the record. **"One aggregate per transaction" is asserted, not yet demonstrated:** the claim is about commits, and Volume I has nothing that commits. Volume II proves it at the IUnitOfWork, where the whole cluster saves or nothing does. **Concurrent sessions are the same deferral:** the door serializes access to one in-memory instance; two *processes* holding two copies need the optimistic-concurrency version field, owned by the persistence seam. **The evidence list is unbounded:** honest at demonstrator scale, and a real cost curve — an aggregate is loaded whole and locked whole, so a ten-thousand-row case would pay for its size on every operation. The cure is redrawing the boundary, not tuning the query; Chapter 11 discusses where the redraw would cut. **ConfirmedCause is a string:** in *From Flood to Cause* it is a node in a causal graph, and the walkthrough restores the typed reference.

CHECKPOINT · CHAPTER 8

Before moving on, you should be able to answer these without looking back:

1. Replay the 04:30 corruption, then explain — in terms of the door, not of carefulness — why Listing 8.3 makes it unwritable.

2. Evidence is an entity, yet no repository will ever exist for it. Reconcile those two facts.

3. State the three aggregate rules with one reason each, and identify where Listings 8.2–8.3 practice each one.

4. Why is *theCase.Evidence.Add(...)* not a runtime error, and which earlier chapters used the same trick of making the mistake unwritable?

5. *CaseClosedEvent* carries *EvidenceCount* but not the evidence list. Defend the choice with rule one and rule two together.

Twice now an aggregate has dropped an announcement into its mailbox and this book has looked away. The mailbox is full: opened cases, raised alarms, a closed case that ReinforcementLearning has every reason to care about. The next chapter empties it — what domain events are for, who dispatches them, when they fire relative to the transaction that is not yet real, and how contexts converse without ever holding hands.

Diamonds, Doors, and the Boundary Drawn

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
> |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 11

RootCause joins the populated contexts: the flagship aggregate, its child Evidence, two case events, and three tests including the flagship watched to fail. Suite at eleven.

UML’s three connectors, finally disambiguated

Class diagrams draw relationships with three easily-confused connectors, and the aggregate chapter is where the difference stops being pedantry. A plain line is **association**: “knows about”, no ownership implied — the mailbox line in Figure D4.1. An empty diamond is **aggregation**: a whole–part grouping where the parts outlive the whole — a Playlist and its Songs. A **filled diamond is composition**: the part’s entire lifecycle belongs to the whole — created by it, reachable only through it, dying with it. Evidence is the textbook filled diamond: born inside *AttachEvidence*, never fetched alone, meaningless without its case. DDD’s unfortunate reuse of the word “aggregate” for what UML would call a composition boundary is a historical accident this paragraph exists to defuse: **a DDD aggregate is drawn with UML composition.**

“Invariant”, defined once and plainly

An **invariant** is a statement about state that must be true at every observable moment — not a validation run at the border, but a property the object is never allowed to be caught violating, mid-operation included. “A closed case has unstruck evidence” is an invariant; “the description field is under 500 characters” is input validation wearing a suit. The distinction dictates the architecture: validations can live at any border because they judge a message once; invariants must live where *every* state change passes, which is why the aggregate root — the single door — is not a style preference but the only address at which an invariant can actually keep its promise.

The aggregate, drawn with its boundary

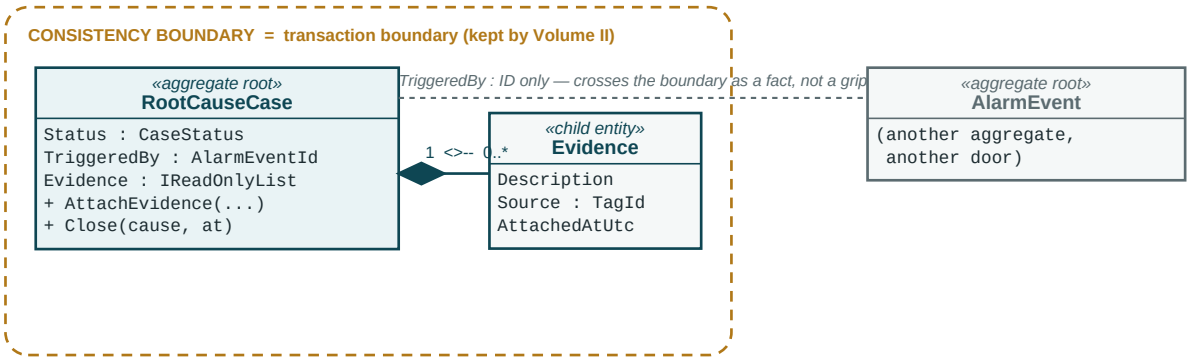


Figure D8.1 — The RootCause aggregate in UML: filled diamond = composition; the dashed frame is the line a transaction will one day trace.

The 04:30 story, replayed through the door

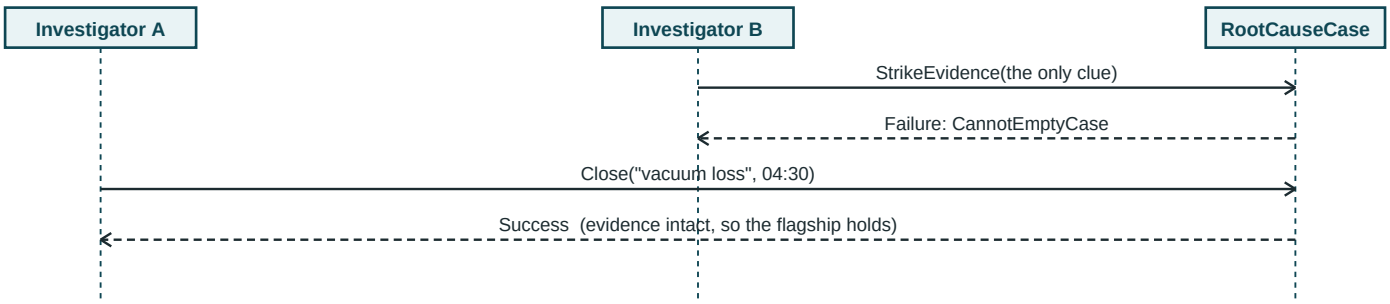


Figure D8.2 — Sequence view: two investigators, one door. (StrikeEvidence is Chapter 11's method, shown a little early.)

Compare this with page 49's naive version, where the two operations touched separate objects and interleaved into a lie. Here both arrows terminate on one lifeline, so there is no moment between check and change for the other request to occupy: the door *is* the serialization. One honest caveat the caption cannot carry: this serialization is per loaded instance — two *processes* holding two copies reopen the race, which is exactly the optimistic-concurrency debt Chapter 8's boundary box filed with Volume II. The diagram shows what the aggregate guarantees; the ledger shows what it honestly cannot, yet.

9 · Domain Events

The conversation the plant needs, and the coupling it refuses

A critical alarm is raised in AlarmManagement; a case should open in RootCause. Some code, somewhere, must connect those sentences — and the obvious connection is the wrong one. Have the alarm's code call the case's code and AlarmManagement now knows RootCause exists, knows its rules for when investigation is warranted, and must recompile when those rules change. Worse, the knowledge is a lie about authority: *whether* a critical alarm warrants a case is RootCause's decision, and the direct call quietly relocates it to the announcer. The cure inverts the sentence. AlarmManagement announces what happened — it already does; the mailbox has been filling since Chapter 6 — and RootCause, owning its own policy, decides what the announcement means to it. Announcements down, commands out.

The anatomy, now law

Every event so far obeyed four clauses by example; they are now the statute for all seventeen contexts. **Past tense** — an event is a fact, not a request: *AlarmRaisedEvent*, never *RaiseAlarmEvent*; if it can be refused, it is a command (Chapter 14), not an event. **Immutable** — facts do not develop revisions; sealed records, get-only or init-only, and a reflection test on page 60 patrols the statute mechanically. **IDs and facts, never entities** — an event crossing a boundary carrying a live aggregate would hand out back-door references to a cluster Chapter 8 just finished locking. **Minimal but sufficient** — carry what a subscriber needs to decide, not everything the publisher knows; *CaseClosedEvent* carries the evidence count, not the evidence.

The second sanctioned coupling: published language

For RootCause to react to *AlarmRaisedEvent*, it must reference the type — a cross-context reference, and a legal one. Chapter 6 sanctioned identifiers; events are the second and last exception, and the reason is what they are: a context's *Events/* folder is its published language, the deliberately public, deliberately stable vocabulary it offers the rest of the plant. Referencing a published event couples a subscriber to an announcement format — shallow, versionable, one-directional. Referencing an entity couples it to a stranger's internals. The convention's final form: **across contexts, IDs and events; nothing else, ever.**

The subscriber's brain, kept in the Domain

What RootCause does about an alarm splits cleanly in two. The *decision* — does this announcement warrant investigation? — is domain knowledge and lives here, as a policy pure enough to test with nothing. The *plumbing* — receiving the event, calling the policy, opening and saving the case — is orchestration, and belongs to the Application layer's event handlers (Chapter 16 wires them; Chapter 12 gives them their ports). Keeping the brain out of the plumbing is what lets this chapter finish inside the Domain layer:

```
namespace Nexus.Domain.RootCause;

/// <summary>
/// RootCause's own answer to AlarmManagement's announcement:
/// which alarms are worth an investigation?
/// </summary>
public static class CaseOpeningPolicy
{
    public static bool WarrantsInvestigation(AlarmRaisedEvent alarm) =>
        alarm.Severity == AlarmSeverity.Critical;
}
```

Listing 9.1 — src/Nexus.Domain/RootCause/CaseOpeningPolicy.cs, complete.

Deliberately almost insultingly simple — one severity check — and deliberately a named home rather than an inline *if* in some future handler, because this policy is scheduled to grow. *From Flood to Cause*'s first lesson is that the interesting unit of judgement is not one alarm but a *flood* — dozens of raises and clears in a correlation window — and judging a window takes state and time, which a static predicate cannot hold. That richer judge is a domain service, Chapter 10's subject; it will stand in this same file's neighborhood, and this predicate will become its first, simplest rule. The policy also settles a debt by creating a subscriber: flood analysis consumes clears as much as raises, and Chapter 6 left *Clear* silent pending exactly this justification.

```
    // inside Clear(atUtc), after the mutation:
        ClearedAtUtc = atUtc;
+       Raise(new AlarmClearedEvent(Guid.NewGuid(), atUtc, Id, UnitId));
        return Result.Success();

    // Events/AlarmEvents.cs gains:
+   public sealed record AlarmClearedEvent(Guid EventId, DateTime OccurredAtUtc,
+       AlarmEventId AlarmId, UnitId UnitId) : DomainEvent(EventId, OccurredAtUtc);
```

Listing 9.2 — AlarmEvent.cs and Events/, revised: clearance gets its announcement, now that someone is listening.

The timing question: when does the mailbox empty?

Raising an event and delivering it are different acts, and the gap between them is the most consequential timing decision in the architecture. Volume I must make it now — the mailbox contract is already public API — even though the transaction it is timed against will not exist until Volume II. Two candidates. **Dispatch before commit:** handlers run inside the publisher's transaction, so their side effects commit or roll back with it — seductive consistency, until a rollback arrives and un-happens facts that subscribers already acted on. An announced fact that can be retracted is not a fact; it is a rumor, and building on rumors is how contexts corrupt each other politely. **Dispatch after commit:** handlers see only committed truth, and each reacts in its own transaction — which is precisely rule two of Chapter 8, eventual consistency between aggregates, now with its mechanism attached.

NEXUS-1's decision, recorded: **events dispatch after successful commit, never before.** The mailbox design has been shaped for that seam since Chapter 4 — and a small mystery from its checkpoint now resolves. *Raise* is protected because only the aggregate may declare what happened to it; *ClearDomainEvents* is public because it belongs to the commit seam's side of the contract: collect the events, commit the transaction, dispatch, clear. In Volume I, the only party standing at that seam is the test — Chapter 19's handler tests play dispatcher by hand, reading mailboxes and publishing to subscribers directly, which has the pleasant side effect of making delivery order and timing explicit in every test that cares.

After-commit dispatch buys truth and pays for it with a crack: a process that commits and then dies before dispatching has made history and told no one. The crack has an industrial cure — persist the outgoing events in the same transaction as the state, deliver from that table with retries; the outbox pattern — and the cure is persistence machinery from top to bottom, which is why naming it here and building it in Volume II is the honest division of labor. What Volume I guarantees is narrower and checkable: nothing in the Domain or Application layer assumes delivery is synchronous, immediate, or even certain.

Patrolling the statute, testing the brain

Two kinds of test close the chapter. The first patrols the anatomy law by reflection over the whole assembly — add a settable property to any event, ever, and this test names the offender; the statute enforces itself from now on. The second pins the policy, all three severities in one theory.

```
public sealed class DomainEventStatuteTests
{
    private static bool IsMutable(PropertyInfo p) =>
        p.SetMethod is { } s && !s.ReturnParameter
            .GetRequiredCustomModifiers().Contains(typeof(IsExternalInit));

    [Fact]
    public void Every_domain_event_is_a_sealed_immutable_past_tense_record()
    {
        var events = typeof(AlarmRaisedEvent).Assembly.GetTypes()
            .Where(t => typeof(IDomainEvent).IsAssignableFrom(t)
                        && t is { IsAbstract: false, IsInterface: false });

        Assert.All(events, t =>
        {
            Assert.True(t.IsSealed, $"{t.Name} must be sealed");
            Assert.EndsWith("Event", t.Name);
            Assert.All(t.GetProperties(),
                p => Assert.False(IsMutable(p), $"{t.Name}.{p.Name} is mutable"));
        });
    }
}

public sealed class CaseOpeningPolicyTests
{
    [Theory]
    [InlineData(AlarmSeverity.Critical, true)]
    [InlineData(AlarmSeverity.Warning,  false)]
    [InlineData(AlarmSeverity.Advisory, false)]
    public void Only_critical_alarms_warrant_investigation_for_now(
        AlarmSeverity severity, bool expected)
    {
        var announcement = new AlarmRaisedEvent(Guid.NewGuid(),
            DateTime.MinValue, AlarmEventId.New(), UnitId.New(), severity);

        Assert.Equal(expected,
            CaseOpeningPolicy.WarrantsInvestigation(announcement));
    }
}

$ dotnet test tests/Nexus.Domain.Tests
Passed! - Failed: 0, Passed: 15, Skipped: 0, Duration: 96 ms
```

Listing 9.3 — tests/Nexus.Domain.Tests/: the self-enforcing statute, the policy pinned, and the run.

> **SEE IT IN NEXUS-1 · The conversation, animated**
>
> Trigger a critical drill on the console and watch two screens change that share no code: **Alarms & Events** gains a red row, and moments later **Root Cause Graph** gains a fresh case node citing that alarm as its trigger. The gap between the two changes is this chapter drawn as animation — announcement, policy, reaction — and the citation on the case node is *TriggeredBy*, the ID-only reference of Chapter 8, closing the loop.

HONEST BOUNDARY

Three edges of this chapter, marked. **Everything here is in-process:** "domain event" in this book means an announcement between contexts sharing one runtime; the cross-process cousin — integration events on a broker, with schemas, versioning, and consumer contracts — is a different animal with the same name, and Volume II introduces it properly rather than letting the vocabulary blur. **The dual-write crack is named, not fixed:** commit-then-crash loses announcements until the outbox arrives in Volume II; Volume I's guarantee is only that no code here assumes reliable delivery. **The statute test is a doctrine exception, acknowledged:** Chapter 3 declined architecture tests for namespace isolation, and page 60 ships one for event anatomy. The distinction: the statute is a mechanical property of types (sealed, immutable, named), checkable without modeling the dependency graph — cheap to enforce, cheap to understand. Where the line sits between "convention held by discipline" and "convention held by reflection" is a judgement, and this book draws it in the open.

CHECKPOINT · CHAPTER 9

Before moving on, you should be able to answer these without looking back:

1. The direct call from alarm to case is wrong twice — once about coupling, once about authority. Give both halves.

2. Recite the four-clause statute, and for each clause, name the listing (any chapter) where it was already being obeyed.

3. State the final cross-context convention, and explain why referencing a published event is shallow coupling where referencing an entity is deep.

4. Argue after-commit dispatch from the rumor problem, name the crack it opens, and name the cure and its owner.

5. Why did *AlarmClearedEvent* wait until this chapter — and what does that waiting rule prevent, at seventeen-context scale?

The policy on page 58 confessed its own ceiling: one alarm, one predicate, no memory. Flood detection needs to watch a window; causal ranking needs to weigh a graph; neither rule belongs to any single entity, and neither fits in a static method. Logic that is genuinely of the domain but native to no object gets its own citizen — the domain service — and the next chapter builds two of them.

Publish/Subscribe, and the Direction of Knowledge

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
> |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 15

The Events/ folders are now doctrine: the statute test patrols every event in the assembly, CaseOpeningPolicy holds RootCause's judgement, and AlarmClearedEvent exists because a subscriber finally does.

Three ways for A to make B do something

A **method call** is maximal knowledge: A holds a reference to B, names B's method, and blocks for the answer — right inside a boundary, ruinous across one. A **command** (Chapter 14) is addressed intent: A knows *what* it wants done and that a handler exists, but not who; the request can be refused. An **event** is the far end of the spectrum: A announces a fact about itself and knows nothing else — not who listens, not whether anyone does, not what they will do about it. The design skill this chapter trains is choosing the *minimum knowledge that still does the job*, and the figure below shows why the alarm/case connection sits at the far end.

Publish/subscribe, mechanically

Strip the vocabulary and pub/sub is the classic Observer pattern grown up: publishers append announcements to a channel; a dispatcher, later, hands each announcement to every handler whose *type signature* says it cares. Nobody holds a list of recipients in their own code — the subscription *is* the type (*I handle AlarmRaisedEvent*), discovered by scanning, exactly like Chapter 13's "a command's type is its address" but one-to-many and fire-and-observe. The profound consequence for a seventeen-context plant: **adding the eighteenth reaction to an alarm requires touching zero existing files.** The publisher's code is closed; the system's behavior stays open — the open/closed principle, delivered by a folder named Events.

The same connection, wired two ways

Figure D9.1 — One connection, two wirings. On the left, knowledge and authority flow the wrong way; on the right, only facts move.

The whole journey of one announcement

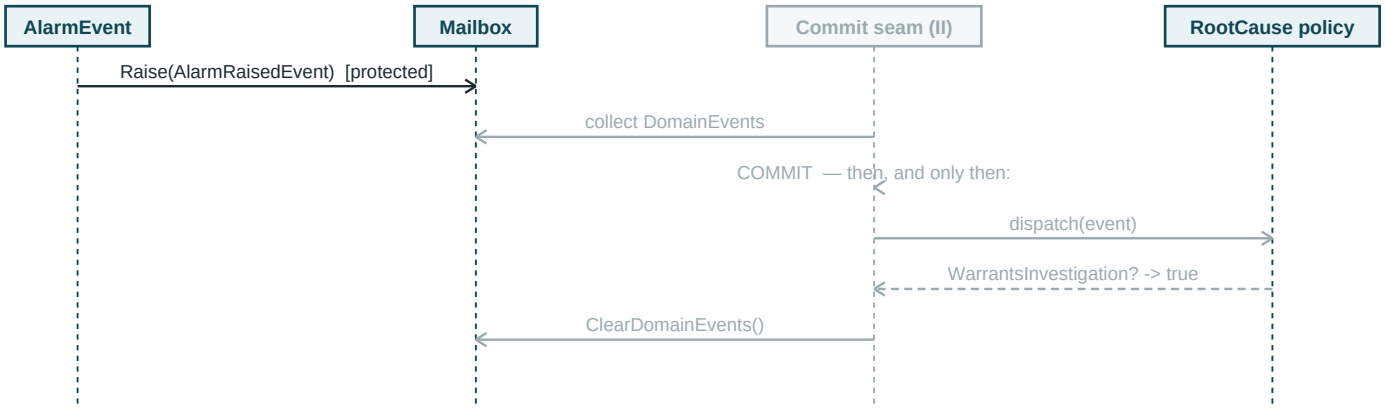


Figure D9.2 — Sequence of an announcement's life. Gray lifelines are the machinery Chapters 12–16 and Volume II supply.

Read the gray lifeline's three arrows as one ritual — collect, commit, dispatch, clear — and the after-commit decision becomes visual: the dispatch arrow is *below* the commit self-call, never above it. In Volume I the test plays that gray column by hand; in Volume II the unit of work inherits the ritual verbatim, which is why the port's documentation already spells it in this order.

10 · Domain Services

Logic that is of the domain but native to no object

Chapter 9's policy hit its confessed ceiling: judging a *flood* means judging a window of many alarms, and the question “whose method is that?” has no good answer. Put *DetectFlood* on *AlarmEvent* and one alarm is somehow responsible for judging its hundred neighbors. Invent a *FloodMonitor* entity to hold the method and you have built an object-shaped excuse — no identity worth tracking, no lifecycle, no state that is not really configuration; a noun manufactured to house a verb. DDD's answer is to stop forcing the noun: some domain knowledge is genuinely an *operation*, and it gets its own kind of citizen. The test, worth memorizing: **if assigning the logic to any entity or value object distorts that object, it is a domain service.**

The four rules of the citizen

Stateless between calls — a service may carry configuration (fixed at construction, read-only forever) but never accumulates state from one judgement to the next; the same inputs yield the same verdict on Tuesday and in a test. **Domain-pure** — no ports, no I/O, no clock, no repository; a domain service computes, and anything it needs to compute *on* arrives as parameters. This is Chapter 5's doctrine paying out: the Application handler does the fetching and injecting of the world, then passes *values* in. **Named as an operation in the ubiquitous language** — *FloodDetectionService*, *CausalRankingService*; the name is the verb the experts use. **Verdicts are values** — a judgement worth returning is worth a type of its own, not a bare bool that discards the evidence for the verdict.

One consequence of domain-purity deserves its own sentence before the code: these services are the most testable objects in the entire system. No fakes, no fixtures, no time machinery — construct, call, assert. If a domain service is hard to test, it has smuggled in a dependency and stopped being a domain service.

The windowed judge

```
namespace Nexus.Domain.AlarmManagement;

public sealed record FloodThresholds
{
    private FloodThresholds(int c, TimeSpan w) { AlarmCount = c;  Window = w; }
    public int AlarmCount { get; }
    public TimeSpan Window { get; }

    public static readonly FloodThresholds Default =
        new(alarmCount: 10, window: TimeSpan.FromMinutes(10));

    public static Result<FloodThresholds> Create(int alarmCount, TimeSpan window) =>
        alarmCount > 0 && window > TimeSpan.Zero
            ? Result.Success(new FloodThresholds(alarmCount, window))
            : Result.Failure<FloodThresholds>(AlarmErrors.InvalidThresholds);
}
// the verdict: constructed only by the judge (see licensing note below)
public sealed record FloodVerdict
{
    internal FloodVerdict(bool flood, int count, DateTime startUtc, DateTime endUtc)
    { IsFlood = flood;  AlarmsInWindow = count;
      WindowStartUtc = startUtc;  WindowEndUtc = endUtc; }
    public bool IsFlood { get; }
    public int AlarmsInWindow { get; }
    public DateTime WindowStartUtc { get; }
    public DateTime WindowEndUtc { get; }
}
public sealed class FloodDetectionService
{
    private readonly FloodThresholds _thresholds;

    public FloodDetectionService(FloodThresholds thresholds) => _thresholds = thresholds;

    public FloodVerdict Judge(IReadOnlyList<DateTime> raiseTimesUtc, DateTime asOfUtc)
    {
        var windowStart = asOfUtc - _thresholds.Window;
        var inWindow = raiseTimesUtc.Count(t => t > windowStart && t <= asOfUtc);

        return new FloodVerdict(inWindow >= _thresholds.AlarmCount,
                                inWindow, windowStart, asOfUtc);
    }
}
```

Listing 10.1 — src/Nexus.Domain/AlarmManagement/FloodDetection.cs, complete.

Values in, value out. *Judge* receives raise *times* — not alarms, not a repository, not a clock; whoever calls has already done the fetching and supplies *asOfUtc* per the no-UtcNow law. The verdict carries its evidence (count and window), and its *internal* constructor licenses a lighter value-object form than Chapter 7's: when the sole author of a type is domain code in the same assembly and every field is set in one place, the guarded public factory would guard against nobody. The license is exactly that narrow — inputs from beyond the layer still pay full fare.

The borderline case, decided out loud

Ranking candidate causes looks, for a moment, like it belongs on *RootCauseCase* — the case holds the evidence, after all. The test settles it: ranking weighs *priors*, knowledge from the causal graph about how plausible each failure mode is before any evidence arrives, and the case does not own that knowledge and should not — a case that carried the plant's entire causal model would be Chapter 8's unbounded aggregate nightmare wearing a clever disguise. The case owns its evidence; the graph owns the priors; the *operation* that weighs one against the other belongs to neither. Service.

```
namespace Nexus.Domain.RootCause;

public sealed record CauseCandidate
{
    private CauseCandidate(string cause, double priorWeight, int supportingEvidence)
    {
        Cause = cause;  PriorWeight = priorWeight;  SupportingEvidence = supportingEvidence;
    }

    public string Cause { get; }
    public double PriorWeight { get; }
    public int SupportingEvidence { get; }

    public static Result<CauseCandidate> Create(string cause, double prior, int support) =>
        !string.IsNullOrWhiteSpace(cause) && prior > 0 && support >= 0
            ? Result.Success(new CauseCandidate(cause, prior, support))
            : Result.Failure<CauseCandidate>(RootCauseErrors.InvalidCandidate);
}

public sealed record RankedCause { /* Cause, Score; light form, service-authored */ }

public sealed class CausalRankingService
{
    public Result<IReadOnlyList<RankedCause>> Rank(IReadOnlyList<CauseCandidate> candidates)
    {
        if (candidates.Count == 0)
            return Result.Failure<IReadOnlyList<RankedCause>>(
                RootCauseErrors.NothingToRank);

        IReadOnlyList<RankedCause> ranked = candidates
            .Select(c => new RankedCause(c.Cause,
                        c.PriorWeight * (1 + c.SupportingEvidence)))
            .OrderByDescending(r => r.Score)
            .ThenBy(r => r.Cause, StringComparer.Ordinal)   // deterministic ties
            .ToList();

        return Result.Success(ranked);
    }
}
```

Listing 10.2 — src/Nexus.Domain/RootCause/CausalRankingService.cs, complete.

Note the fare difference inside one listing: *CauseCandidate* arrives from outside the layer and pays for the full guarded pattern; *RankedCause* is service-authored and rides the license. And note the tie-break: *ThenBy Ordinal* is not pedantry — a judgement that returns a different order for the same facts on different runs is a judgement no test can pin and no investigator can trust. Determinism is a domain requirement here, not a testing convenience.

Who composes the judges

Chapter 5's ledger holds: the Domain project still publishes no registrations. The services *exist* in the Domain; *composing* them is the Application layer's business, and *AddApplication* grows its promised first growth ring — singletons, legally, because both are stateless and their only field is read-only configuration:

```
        services.AddValidatorsFromAssembly(typeof(DependencyInjection).Assembly);

+       // domain services: stateless judges, composed here, newed in tests
+       services.AddSingleton(new FloodDetectionService(FloodThresholds.Default));
+       services.AddSingleton<CausalRankingService>();

        return services;
```

Listing 10.3 — DependencyInjection.cs, revised as promised in Chapter 5.

The judges, pinned

```
public sealed class FloodDetectionServiceTests
{
    private static readonly DateTime T0 = new(2026, 7, 2, 4, 0, 0, DateTimeKind.Utc);
    private readonly FloodDetectionService _judge = new(FloodThresholds.Default);

    private static IReadOnlyList<DateTime> Raises(int n, TimeSpan apart) =>
        [.. Enumerable.Range(0, n).Select(i => T0 - i * apart)];

    [Fact]
    public void Ten_raises_in_ten_minutes_is_a_flood_at_the_threshold_exactly()
    {
        var verdict = _judge.Judge(Raises(10, TimeSpan.FromSeconds(30)), T0);
        Assert.True(verdict.IsFlood);
        Assert.Equal(10, verdict.AlarmsInWindow);
    }

    [Fact]
    public void A_raise_sitting_exactly_on_the_window_edge_is_outside_it()  // pinned
    {
        var edge = T0 - FloodThresholds.Default.Window;          // t == windowStart
        var verdict = _judge.Judge([edge], T0);
        Assert.Equal(0, verdict.AlarmsInWindow);                 // strict >, by design
    }
}
```

Listing 10.4 — tests/Nexus.Domain.Tests/: no fakes, no fixtures — construct, call, assert.

```
public sealed class CausalRankingServiceTests
{
    [Fact]
    public void Ranking_nothing_is_refused_by_name()            // watched to fail
    {
        var result = new CausalRankingService().Rank([]);
        Assert.Equal("RootCause.NothingToRank", result.Error.Code);
    }

    [Fact]
    public void Equal_scores_rank_in_the_same_order_every_run()  // determinism pinned
    {
        var svc = new CausalRankingService();
        var tied = new[] { "Valve stiction", "Sensor drift" }
            .Select(c => CauseCandidate.Create(c, prior: 0.5, support: 2).Value)
            .ToArray();

        Assert.Equal(svc.Rank(tied).Value.Select(r => r.Cause),
                     svc.Rank([.. tied.Reverse()]).Value.Select(r => r.Cause));
    }
}

$ dotnet test tests/Nexus.Domain.Tests
Passed! - Failed: 0, Passed: 19, Skipped: 0, Duration: 108 ms
```

Listing 10.5 — CausalRankingServiceTests, and the run.

> **SEE IT IN NEXUS-1 · Both judges, on screen**
>
> Run a flood drill on the console and the **Alarms & Events** header flips to a FLOOD banner the moment the window fills — Listing 10.1's verdict as a red ribbon, with the in-window count displayed beside it. Then open the **Root Cause Graph**: the candidate causes down the side panel are sorted by exactly the kind of score Listing 10.2 computes, top suspect first. Two services, two screens, and neither screen knows the other exists — which is the whole of Part II in one sentence.

HONEST BOUNDARY

Two teaching cuts, priced. **The ranking is a placeholder:** *From Flood to Cause* ranks by propagating evidence through the causal graph itself — edges, depths, and time-decay, not a flat prior-times-support product. The service's *signature* is the durable artifact here (candidates in, deterministic ranking out); Chapter 11's RootCause walkthrough upgrades the scoring behind it without a caller noticing. **The flood judge sees what it is handed:** its correctness depends on the caller supplying the complete raise history for the window, and nothing in the type system enforces that completeness — the query that guarantees it is Chapter 15's read-model business, and the gap between “correct function” and “correctly fed function” is exactly where the Application layer earns its keep.

CHECKPOINT · CHAPTER 10

Before moving on, you should be able to answer these without looking back:

1. State the distortion test, then run it on *DetectFlood*-as-entity-method and on the invented *FloodMonitor* — one sentence of verdict each.

2. *FloodDetectionService* holds a field yet counts as stateless. Reconcile, and say what that field may never do.

3. Recite the light-form license exactly, and explain why *CauseCandidate* pays full fare in the same file where *RankedCause* rides free.

4. Why is the ordinal tie-break a domain requirement rather than test hygiene? Answer with the investigator, not the test runner.

5. The judges are registered as singletons in *AddApplication*. Defend both halves: why singleton is legal, and why the registration lives in Application while Chapter 5's “Domain publishes nothing” stays true.

Six kinds of citizen now exist — entities, value objects, aggregates, events, policies, services — each shown once, in its cleanest case. What remains for Part II is proof at scale: the next chapter walks three full bounded contexts end to end — AlarmManagement completed, RootCause deepened to its causal graph, and ReinforcementLearning built from nothing — using no tool this book has not already placed on the bench.

Pure Functions With a Business Card

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
> |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 19

Two judges now live in the Domain — FloodDetectionService and CausalRankingService — registered by Application, newed by tests, and owning nothing but read-only configuration.

“Stateless” and “pure”, precisely

Two words carried this chapter and both deserve exact definitions. A method is **pure** when its output depends on nothing but its inputs and it changes nothing anywhere: no clock, no I/O, no mutation of arguments, no logging on the side. Call it a thousand times with the same inputs and you get a thousand identical answers and an untouched world. A service is **stateless** when nothing from one call survives into the next — which is compatible with having fields, provided they are *configuration*: set once at construction, read-only forever, identical for every caller. *FloodDetectionService* threads that needle exactly: *_thresholds* is a fact about the judge, not a memory of past judgements. The test for whether a field is legal is one question — **could two threads share this instance forever without noticing each other?** Yes: configuration. No: smuggled state, and the singleton registration is now a bug.

Service or helper class? The business card test

Every codebase accumulates static “helpers”, and newcomers reasonably ask how a domain service differs from a *MathUtils*. The difference is who would recognize the name. A domain service carries a **business card**: *FloodDetectionService*, *CausalRankingService* — nouns a control-room operator or an investigator would nod at, performing one operation the ubiquitous language already had a verb for. A helper carries a shrug: *StringExtensions*, useful everywhere, meaningful nowhere. Helpers are fine — the kernel’s *Ensure* is one — but they live outside the domain conversation. If the domain expert would not recognize the class name, it is not a domain service, whatever folder it sits in.

The judge’s diet, drawn

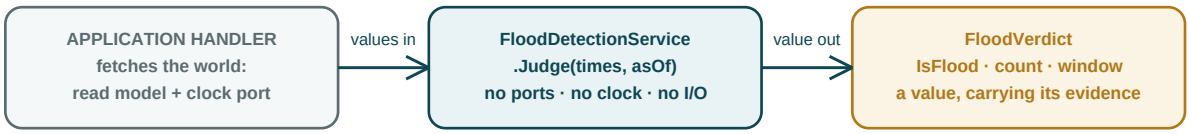


Figure D10.1 — The dataflow contract of every domain service: the world stops at the door; only values cross it.

The window, on a clock you can argue with

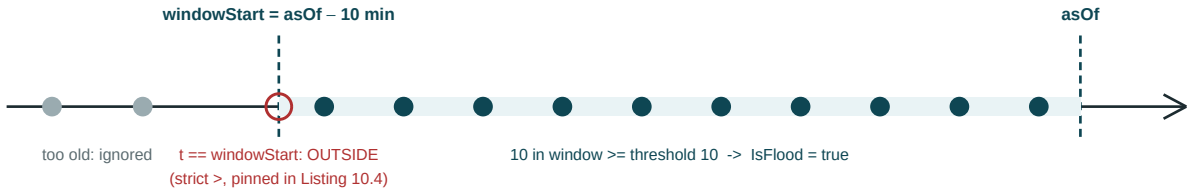


Figure D10.2 — The flood window as an interval: half-open (windowStart, asOf], edge exclusive by design and by test.

The open circle is the whole point of drawing this: interval boundaries are where windowed logic silently disagrees with itself, and “is the edge in or out?” is not a taste question once two implementations must agree (this service today, a SQL projection in Volume II tomorrow). The book’s answer — half-open, edge outside, pinned by a test that puts a raise exactly on the line — is one of those decisions that costs a sentence now or an incident report later. When you meet any windowed rule in the wild, draw this figure for it before writing code; the drawing is cheaper than the disagreement.

11 · Context Walkthroughs

Proof at scale, and a ledger of debts

Every pattern so far was shown once, in its cleanest case — which proves elegance, not adequacy. This chapter is the adequacy test: three bounded contexts taken end to end — AlarmManagement completed, RootCause deepened to its causal graph, ReinforcementLearning built from nothing — under one constraint that is the whole point: **no tool appears here that Chapters 4 through 10 did not already place on the bench.** If the walkthroughs need a new invention, Part II failed; if they read as routine, Part II worked. The chapter also settles accounts. Six chapters made promises payable here, and the calibration habit says a promise unlisted is a promise half-kept — so the ledger comes first:

Ch. 6 boundary	The fuller alarm lifecycle: shelving and re-alarm restored	*§11.1, Listing 11.1*
Ch. 8, p. 52	StrikeEvidence, refusing from the invariant's other side	*§11.2, Listing 11.2*
Ch. 8 boundary	ConfirmedCause: teaching-cut string -> typed CauseNodeId	*§11.3, Listing 11.4*
Ch. 10 boundary	Ranking upgraded to graph-aware scoring, callers unchanged	*§11.3, Listing 11.3*
Ch. 1, Table 1.1	The Policy and QTable aggregates of From Trial to Policy	*§11.4, Listings 11.5–6*
Ch. 10 boundary	The flood judge correctly fed	*deferred again, by name — Ch. 15's read model*

Table 11.1 — The debts ledger. One entry is deferred again rather than quietly dropped; that, too, is the habit.

A note on form: the walkthroughs write in diffs wherever a chapter already printed the class, and in full wherever a type is new — the reader's picture of AlarmEvent should evolve the way the file does, not be reprinted the way a textbook pads.

§11.1 · AlarmManagement, completed

Two transitions were amputated from Chapter 6's teaching cut, and both exist because operators are humans in rooms, not state machines. **Shelving** is disciplined snoozing: an acknowledged nuisance alarm may be suppressed *by a named operator, until a named time* — never anonymously, never forever; an unbounded shelf is how alarm systems rot. **Re-alarm** is the plant reasserting itself: a condition that returns while its alarm is shelved or cleared re-raises it — and, crucially, *wipes the acknowledgement*, because yesterday's "I saw it" is not consent to today's recurrence.

```
public enum AlarmStatus { Raised = 1, Acknowledged = 2, Cleared = 3 }
public enum AlarmStatus { Raised = 1, Acknowledged = 2, Cleared = 3, Shelved = 4 }

    public OperatorId? ShelvedBy { get; private set; }
    public DateTime? ShelvedUntilUtc { get; private set; }

    public Result Shelve(OperatorId by, DateTime untilUtc, DateTime atUtc)
    {
        if (Status != AlarmStatus.Acknowledged)
            return Result.Failure(AlarmErrors.OnlyAcknowledgedShelves);
        if (untilUtc <= atUtc)
            return Result.Failure(AlarmErrors.ShelfMustEnd);

        Status = AlarmStatus.Shelved;
        ShelvedBy = by;  ShelvedUntilUtc = untilUtc;
        Raise(new AlarmShelvedEvent(Guid.NewGuid(), atUtc, Id, by, untilUtc));
        return Result.Success();
    }

    public Result ReAlarm(DateTime atUtc)   // the condition has returned
    {
        if (Status is not (AlarmStatus.Shelved or AlarmStatus.Cleared))
            return Result.Failure(AlarmErrors.NothingToReAlarm);

        Status = AlarmStatus.Raised;
        Acknowledgement = null;             // recurrence demands fresh eyes
        Raise(new AlarmReRaisedEvent(Guid.NewGuid(), atUtc, Id, UnitId, Severity));
        return Result.Success();
    }
```

Listing 11.1 — AlarmEvent.cs, revised: the lifecycle grows two transitions and one new refusal each.

Two loose ends, tied deliberately rather than hidden. *ShelvedBy* and *ShelvedUntilUtc* are Chapter 7's smell resurfacing on purpose — curing them with a *Shelving* value object, plus the missing *Unshelve*, is **Exercise 11.1**, the book's first, with the solution in Appendix B. And nothing here *enforces* the shelf's expiry: watching clocks is a scheduler's job, and schedulers are runtime — Volume III owns the timer that calls *ReAlarm* when a shelf lapses. The domain's job ends at making the transition legal and the deadline recorded.

§11.2 · RootCause I: correcting the record without erasing it

Chapter 8 made evidence immutable and promised the correction door. The design question is what "correct" may mean in an investigation, and the answer is borrowed from every paper trail that has ever survived an audit: **you strike through; you do not erase.** Struck evidence stays in the record, visibly struck, with its reason — so *StrikeEvidence* marks rather than removes. And one refusal completes the flagship from its other side. The flagship said a case cannot *close* without evidence; its mirror says a case that has evidence cannot be *emptied*: **a case may start empty, but may not be made empty.** An investigation whose entire basis is struck out is not a corrected case — it is a case someone is trying to make disappear, and the aggregate refuses to be the instrument.

```
    // Evidence gains the annotation; nothing is ever deleted:
    public bool Struck { get; private set; }
    public string? StruckReason { get; private set; }
    internal void Strike(string reason) { Struck = true;  StruckReason = reason; }

    // RootCauseCase gains the correction door:
    public Result StrikeEvidence(EvidenceId id, string reason)
    {
        if (Status == CaseStatus.Closed)
            return Result.Failure(RootCauseErrors.CaseAlreadyClosed);

        var target = _evidence.FirstOrDefault(e => e.Id == id && !e.Struck);
        if (target is null)
            return Result.Failure(RootCauseErrors.NoSuchEvidence);

        if (_evidence.Count(e => !e.Struck) == 1)
            return Result.Failure(RootCauseErrors.CannotEmptyCase);   // the mirror

        target.Strike(reason);
        return Result.Success();
    }

    // and inside Close(...), the flagship learns to see through strikes:
        if (_evidence.Count == 0)
        if (_evidence.Count(e => !e.Struck) == 0)
            return Result.Failure(RootCauseErrors.CaseWithoutEvidence);
```

Listing 11.2 — Evidence.cs and RootCauseCase.cs, revised: the strike-through, and the flagship deepened.

Note where *Strike* lives: an *internal* method on Evidence, called only by the root — the child mutates, but only through the cluster's one door, which is Chapter 8's whole sermon in a single call chain. And note the asymmetry defended in the refusal: birth and emptying are different acts. A newly opened case with no evidence is an investigation that has not started; a stripped case is one being unmade. The type system now knows the difference.

§11.3 · RootCause II: the causal graph arrives

From Flood to Cause models the plant's failure knowledge as a graph: nodes are failure modes, each with a prior plausibility and a parent pointing toward deeper causes — symptoms shallow, root modes deep. Volume I carries the graph as its own small aggregate, and its arrival pays two debts at once. Candidates are now *manufactured by the graph*, with depth woven into the prior — a deep cause must earn its rank through support, not inherit it from plausibility alone — so Chapter 10's ranking gets smarter while its signature, and every caller, stands still. And the confirmed cause becomes a typed citizen.

```
namespace Nexus.Domain.RootCause;

public readonly record struct CauseNodeId(Guid Value) : IEntityId;

public sealed class CauseNode : Entity<CauseNodeId>
{
    internal CauseNode(CauseNodeId id, string name, double prior,
                       CauseNodeId? parent, TagId? signal) : base(id)
    { Name = name;  Prior = prior;  Parent = parent;  Signal = signal; }

    public string Name { get; }
    public double Prior { get; }
    public CauseNodeId? Parent { get; }     // toward deeper failure modes
    public TagId? Signal { get; }           // the instrument that testifies for it
}

public sealed class CausalGraph : Entity<CausalGraphId>
{
    private readonly Dictionary<CauseNodeId, CauseNode> _nodes = [];
    public UnitId UnitId { get; }
    // factory + AddNode: Ch.8 patterns exactly; printed in Appendix B

    public int DepthOf(CauseNodeId id)
    {
        var depth = 0;
        for (var n = _nodes[id]; n.Parent is { } p; n = _nodes[p]) depth++;
        return depth;
    }

    public IReadOnlyList<CauseCandidate> CandidatesFor(
        IReadOnlyList<Evidence> evidence, double depthDecay = 0.8) =>
        [.. _nodes.Values.Select(n => CauseCandidate.Create(
                n.Name,
                prior:   n.Prior * Math.Pow(depthDecay, DepthOf(n.Id)),
                support: evidence.Count(e => !e.Struck && e.Source == n.Signal)
            ).Value)];   // .Value is safe: the graph is the candidates' sole author
}
```

Listing 11.3 — src/Nexus.Domain/RootCause/CausalGraph.cs (construction elided to Appendix B, marked).

```
    public string? ConfirmedCause { get; private set; }
    public CauseNodeId? ConfirmedCause { get; private set; }

    public Result Close(string confirmedCause, DateTime atUtc)
    public Result Close(CauseNodeId confirmedCause, DateTime atUtc)
        // body unchanged; CaseClosedEvent now carries the node id
```

Listing 11.4 — RootCauseCase.cs, revised: the confirmed cause joins the type system.

§11.4 · ReinforcementLearning, from nothing

The third walkthrough starts from an empty folder, which is its point: a context this book has never touched, built with only the bench. The ubiquitous language comes from *From Trial to Policy*: the plant learns by *episodes*, each producing *targets* for state–action pairs; learning accumulates in a **Q-table**; and when learning is judged good enough, a snapshot is promoted into a **policy** — the deployable artifact operators actually consult. The central modelling decision is that these are *two aggregates, not one*. A Q-table is hot: it changes with every episode and its consistency is per-update. A policy is frozen: drafted from a table, evaluated, activated, retired — an audit trail with a lifecycle, pointing at its source by ID like any respectable stranger. Fusing them would weld a write-hot log to an approval workflow, and Chapter 8's sizing rule already priced that mistake.

```
namespace Nexus.Domain.ReinforcementLearning;

public readonly record struct StateAction(string State, string Action);

public sealed record LearningRate
{
    private LearningRate(double v) => Value = v;
    public double Value { get; }

    public static Result<LearningRate> Create(double v) =>
        v > 0 && v <= 1
            ? Result.Success(new LearningRate(v))
            : Result.Failure<LearningRate>(RlErrors.InvalidLearningRate);
}

public sealed class QTable : Entity<QTableId>
{
    private readonly Dictionary<StateAction, double> _q = [];

    private QTable(QTableId id, UnitId unitId) : base(id) => UnitId = unitId;

    public UnitId UnitId { get; }
    public int UpdateCount { get; private set; }

    public static QTable StartLearning(UnitId unitId) => new(QTableId.New(), unitId);

    public void Learn(StateAction sa, double target, LearningRate alpha)
    {
        var current = _q.GetValueOrDefault(sa);
        _q[sa] = current + alpha.Value * (target - current);  // step toward the target
        UpdateCount++;
    }

    public double ValueOf(StateAction sa) => _q.GetValueOrDefault(sa);
}
```

Listing 11.5 — src/Nexus.Domain/ReinforcementLearning/QTable.cs and friends, complete.

Learn returns void, and the absence is a statement: with alpha pre-validated at the boundary and unknown pairs legally starting at zero, no business rule *can* fail here — and per Chapter 4's doctrine, a method that cannot refuse should not pretend it might. The update itself is the exponential step toward a target that *From Trial to Policy* derives; where the *targets* come from is priced on the last page.

The deployable artifact — and the invariant rhyme

```
public enum PolicyStatus { Draft = 1, Evaluated = 2, Active = 3, Retired = 4 }

public static class RlErrors
{
    public static readonly Error InvalidLearningRate = new(
        "ReinforcementLearning.InvalidLearningRate", "Alpha must be in (0, 1].");
    public static readonly Error NotDraft = new(
        "ReinforcementLearning.NotDraft", "Only a draft policy can be evaluated.");
    public static readonly Error NotEvaluated = new(
        "ReinforcementLearning.NotEvaluated", "A policy cannot activate without evaluation.");
}

public sealed class Policy : Entity<PolicyId>
{
    private Policy(PolicyId id, UnitId unitId, QTableId source) : base(id)
    { UnitId = unitId;  SourceTable = source;  Status = PolicyStatus.Draft; }

    public UnitId UnitId { get; }
    public QTableId SourceTable { get; }        // frozen learning: ID only
    public PolicyStatus Status { get; private set; }
    public double? EvaluationScore { get; private set; }

    public static Policy Draft(UnitId unitId, QTableId source) =>
        new(PolicyId.New(), unitId, source);

    public Result RecordEvaluation(double score)
    {
        if (Status != PolicyStatus.Draft)
            return Result.Failure(RlErrors.NotDraft);

        Status = PolicyStatus.Evaluated;  EvaluationScore = score;
        return Result.Success();
    }

    public Result Activate(DateTime atUtc)
    {
        if (Status != PolicyStatus.Evaluated)
            return Result.Failure(RlErrors.NotEvaluated);      // the rhyme

        Status = PolicyStatus.Active;
        Raise(new PolicyActivatedEvent(Guid.NewGuid(), atUtc, Id, UnitId));
        return Result.Success();
    }
}

public sealed record PolicyActivatedEvent(Guid EventId, DateTime OccurredAtUtc,
    PolicyId PolicyId, UnitId UnitId) : DomainEvent(EventId, OccurredAtUtc);
```

Listing 11.6 — src/Nexus.Domain/ReinforcementLearning/Policy.cs, complete.

The comment on the refusal says *the rhyme*, and the rhyme is the chapter's real finding. **A case cannot close without evidence; a policy cannot activate without evaluation.** Two contexts that share not one type share a shape: an artifact of consequence, a gate of due diligence, a named refusal standing at the gate. When patterns start rhyming across contexts unprompted, the vocabulary has stopped being technique and become the way the system thinks — which is the strongest evidence Part II could file for its own adequacy.

Four proofs, one per promise

```
public sealed class WalkthroughTests
{
    private static readonly DateTime T0 = new(2026, 7, 2, 5, 0, 0, DateTimeKind.Utc);

    [Fact]
    public void A_shelved_alarm_re_alarms_and_demands_fresh_eyes()
    {
        var a = AlarmEvent.Raise(UnitId.New(), TagId.New(),
                                 AlarmSeverity.Warning, "level drift", T0);
        a.Acknowledge(OperatorId.New(), T0.AddMinutes(1));
        a.Shelve(OperatorId.New(), untilUtc: T0.AddHours(8), atUtc: T0.AddMinutes(2));

        Assert.True(a.ReAlarm(T0.AddHours(1)).IsSuccess);
        Assert.Equal(AlarmStatus.Raised, a.Status);
        Assert.Null(a.Acknowledgement);                    // the slate is clean
    }

    [Fact]
    public void A_case_may_start_empty_but_may_not_be_emptied()   // the mirror, failing
    {
        var rcc = RootCauseCase.Open(UnitId.New(), AlarmEventId.New(), T0);
        rcc.AttachEvidence("the only clue", TagId.New(), T0.AddMinutes(5));

        var strike = rcc.StrikeEvidence(rcc.Evidence[0].Id, "transcription typo");
        Assert.Equal("RootCause.CannotEmptyCase", strike.Error.Code);
        Assert.False(rcc.Evidence[0].Struck);              // and nothing half-applied
    }

    [Fact]
    public void A_policy_cannot_activate_without_evaluation()     // the rhyme, failing
    {
        var p = Policy.Draft(UnitId.New(), QTableId.New());
        Assert.Equal("ReinforcementLearning.NotEvaluated",
                     p.Activate(T0).Error.Code);
    }

    [Fact]
    public void Learning_steps_toward_the_target_by_alpha()
    {
        var q = QTable.StartLearning(UnitId.New());
        var sa = new StateAction("HighXenon", "LowerRods");
        var alpha = LearningRate.Create(0.5).Value;

        q.Learn(sa, target: 1.0, alpha);
        Assert.Equal(0.50, q.ValueOf(sa));                 // halfway there
        q.Learn(sa, target: 1.0, alpha);
        Assert.Equal(0.75, q.ValueOf(sa));                 // and half of the rest
    }
}

$ dotnet test tests/Nexus.Domain.Tests
Passed! - Failed: 0, Passed: 26, Skipped: 0, Duration: 131 ms
```

Listing 11.7 — tests/Nexus.Domain.Tests/Walkthroughs/, selected: each walkthrough's sharpest edge, and the run.

SEE IT IN NEXUS-1 · Three walkthroughs, three screens

Everything this chapter built has a face on the console. Shelved alarms wear a muted badge with their expiry on **Alarms & Events**; the **Root Cause Graph** draws struck evidence with a literal strikethrough and dims deep nodes exactly the way depth decay discounts them; and the **Optimization (RL)** screen shows the Q-value heat map beside policy cards whose ACTIVATE button lights up only once an evaluation score is present — the rhyme, rendered twice on two different screens.

HONEST BOUNDARY

Three cuts, priced. **Three contexts stand for seventeen:** the claim this chapter actually proves is that the bench suffices for three very different shapes — lifecycle-heavy, graph-heavy, learning-heavy; the remaining fourteen are asserted to follow the same patterns, and Appendix D maps them, but no walkthrough exists for them in this volume. **The RL here is the artifact, not the algorithm:** where targets come from — rewards, exploration, episode generation, convergence — is the entire subject of *From Trial to Policy* and lives in the engines, not in this domain model; Volume I models the custody chain of learning, not the learning. **Two numbers are confessed demo constants:** the 0.8 depth decay and the evaluation gate's lack of a score threshold (any recorded score permits activation — a real deployment would demand a floor, and adding one is Exercise 11.2).

CHECKPOINT · CHAPTER 11

Before moving on, you should be able to answer these without looking back:

1. Why does *ReAlarm* wipe the acknowledgement, and which value-object refactor made that wipe a single honest assignment?

2. State the flagship and its mirror as one principle about investigations, and explain why birth-empty and made-empty are different acts to the type system.

3. How did the ranking get smarter without a caller changing? Name what moved, and what stood still.

4. Defend two aggregates for RL against the fused alternative, using Chapter 8's sizing rule and the audit trail.

5. *Learn* returns void where every other mutation in the book returns *Result*. Why is the void the doctrinally correct signature here?

Part II, audited

Part II promised a domain layer with business rules living inside the objects they govern, provable with nothing installed but the SDK: delivered — twenty-six green tests, every invariant watched to fail, and *Nexus.Domain.csproj* still holding zero package references, exactly as Chapter 3 committed. It promised the cross-context convention in its final form: kept — IDs and events, nothing else, now practiced across five contexts. It promised its debts a ledger: paid, with one honest re-deferral (the flood judge's feeding, to Chapter 15) and two exercises opened rather than papered over. The standing concessions travel forward unchanged — context boundaries by convention, concurrency at the persistence seam, dispatch at a commit that does not yet exist. The heart is built and beating in vitro. What it cannot yet do is answer anyone: no request reaches it, no query reads it, no transaction saves it. Part III builds the layer that speaks for the domain to the world — beginning with the ports: the interfaces where the core, for the first time, names what it needs from outside.

The Machines, Completed

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
> |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 26

Five of seventeen context folders are now populated — AlarmManagement, Instrumentation, RootCause, CorePlatform (identifiers), ReinforcementLearning — and Part II closes with the Domain suite at twenty-six.

How to read a diff listing

Chapter 11 writes mostly in **diffs**, and the notation deserves one plain paragraph. A red line prefixed – is code that existed and is now deleted; a green + line is its replacement or addition; unmarked lines are untouched context shown so you can find the spot. Diffs are how professionals actually read change — reviews, pull requests, and history are all diffs — so the book uses them wherever a class you have already read in full evolves: your mental model updates the way the file does, and reprinting forty unchanged lines would only hide the two that matter. If a diff ever confuses you, Appendix B holds every file whole.

Guards in brackets: one more piece of UML

State-machine arrows can carry a **guard** — a condition in square brackets that must be true for the transition to fire: *Shelve(by, until, at) [until > at]*. Guards are how a diagram says “this arrow exists, but conditionally” without inventing extra states, and they map one-to-one onto the early *if*-checks of Listing 11.1: a failed guard is a refusal with a name (*ShelfMustEnd*), state untouched. The completed machine below uses one; spot it, then find its two lines in the listing. That round-trip — diagram element to code lines and back — is the literacy this deep dive exists to install.

The rhyme, as a skill

“A case cannot close without evidence; a policy cannot activate without evaluation” is more than a pleasing sentence — it is a reusable shape: **artifact of consequence, gate of due diligence, named refusal at the gate.** Train yourself to spot the shape and you can design contexts you have never seen: a Maintenance work order that cannot start without a permit, a Compliance report that cannot file without sign-off — same machine, new nouns. Pattern literacy at this level is what “the bench suffices” actually means.

Figure D6.1, redrawn complete — as promised

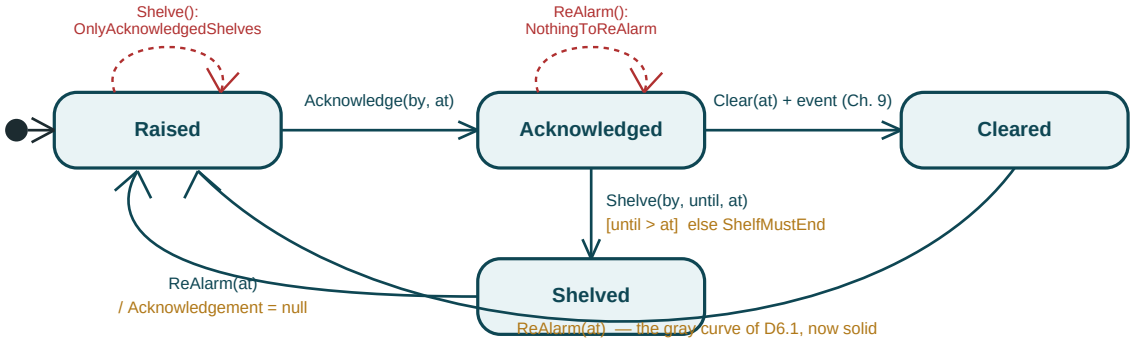


Figure D11.1 — The AlarmEvent machine, complete: four states, one guard in brackets, one action after the slash, refusals in red.

The custody chain of learning

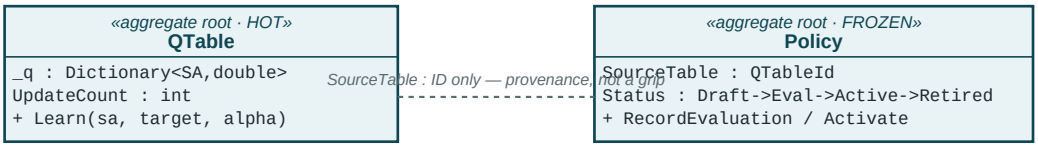


Figure D11.2 — Two aggregates, two tempos: the hot table mutates per episode; the frozen policy carries an audit trail and points back by ID.

One figure, one design law: **things that change at different speeds belong behind different doors.** Fuse these two boxes and every episode's write contends with an approval workflow's locks; split them, and each aggregate stays small, loads cheap, and versions on its own clock — Chapter 8's sizing rule, visible as whitespace between two rectangles.

PART III

The Application Layer: The Orchestrator

The heart beats; now it learns to answer. Six chapters build the layer that speaks for the domain: the ports where the core names what it needs from a world it refuses to know, the commands and queries that carve every use case into a writing side and a reading side, the handlers that orchestrate without deciding, and the pipeline that guards every door with the same checks. Still no database, still no HTTP — and by the end, an entire plant's use cases running against thin air, on purpose.

CHAPTERS 12–17 · PORTS · CQRS · COMMANDS · QUERIES · BEHAVIORS · CASE STUDY

12 · Ports Before Adapters

Naming the need without meeting the supplier

The Application layer's first act is to state its demands. It will need to load aggregates, commit business acts, and know what time it is — three needs that only the outside world can supply, and that the Dependency Rule forbids it from reaching outward to take. The resolution is the oldest trick in the hexagon: **the consumer writes the interface; the supplier, later, signs it.** These consumer-owned interfaces are the *ports*; Volume II's implementations are the *adapters*; and the direction of ownership is the entire point. A port declared in *Nexus.Application.Abstractions* is shaped by what use cases need and nothing else — no leaked DbContext idioms, no HTTP smells — because on the day it is written, no supplier exists to lobby for its own convenience.

First, the freeze gets audited

The repository port must traffic in aggregate roots only — Chapter 8's rule — which requires the type system to know which entities *are* roots, and no such marker exists. The obvious home is the SharedKernel, and the SharedKernel is frozen: Chapter 4 committed to no new public kernel type for the rest of the trilogy, and this is the first moment the commitment costs something. It holds. “Aggregate root” fails the kernel's own admission rule anyway — it is saturated with domain meaning — so the marker lives at the root of the Domain project, a cross-context modeling concept among the contexts it describes:

```
// src/Nexus.Domain/IAggregateRoot.cs -- new:
namespace Nexus.Domain;

public interface IAggregateRoot { }   // a promise: 'ports may traffic in me'

// the five roots take the tag; children and value objects pointedly do not:
public sealed class AlarmEvent : Entity<AlarmEventId>
public sealed class AlarmEvent : Entity<AlarmEventId>, IAggregateRoot
public sealed class RootCauseCase : Entity<RootCauseCaseId>
public sealed class RootCauseCase : Entity<RootCauseCaseId>, IAggregateRoot
// likewise CausalGraph, QTable, Policy -- and NOT Evidence, NOT CauseNode
```

Listing 12.1 — The marker, Domain-owned by conviction and by freeze, and the five roots taking the tag.

An empty interface doing real work: it converts “repositories load roots only” from a sentence in Chapter 8 into a *where*-clause the compiler enforces. Fetching an *Evidence* through a port stops being a review comment and becomes CS0311.

The three ports, in full

```
namespace Nexus.Application.Abstractions;

/// <summary>The core's demand for persistence, stated from the consumer's
/// side. Traffic is aggregate roots only; children ride inside their cluster.
/// </summary>
public interface IRepository<TRoot, TId>
    where TRoot : Entity<TId>, IAggregateRoot
    where TId : notnull
{
    Task<TRoot?> GetAsync(TId id, CancellationToken ct = default);
    Task AddAsync(TRoot root, CancellationToken ct = default);
    void Remove(TRoot root);
    // no Update, no queries -- on purpose: 'What the ports refuse to say', p. 88
}

/// <summary>One business act, one commit. Implementations dispatch each
/// touched aggregate's domain events AFTER a successful commit (Chapter 9's
/// decision, now contract language), then clear the mailboxes.</summary>
public interface IUnitOfWork
{
    Task CommitAsync(CancellationToken ct = default);
}

/// <summary>The clock, demoted to a dependency: the only lawful source of
/// 'now' above the Domain layer.</summary>
public interface IDateTimeProvider
{
    DateTime UtcNow { get; }
}
```

Listing 12.2 — src/Nexus.Application/Abstractions/, complete. The seam the whole book has been promising.

Three signatures deserve their sentence. The repository is generic over the *typed* ID — Chapter 4's identifiers survive the seam intact, so Volume II's adapter cannot quietly regress to naked Guids. Everything that will touch I/O is *async from birth* with a *CancellationToken* in every signature: Volume I's fakes complete synchronously, but the port is Volume II's law, and retrofitting async onto a sync seam is the kind of surgery this book exists to prevent. And the after-commit dispatch decision now lives in the contract's XML documentation — the port does not just permit Chapter 9's timing, it *requires* it of every implementor, including the fakes on the next page.

The named fakes take their real shape

Chapter 5's fixture was written before the ports existed, and one of its guesses did not survive contact with the real seam — recorded as a correction, not silently rewritten:

```
        // every port declared in Chapter 12 meets a named fake here:
        services.AddSingleton(typeof(IRepository<>), typeof(InMemoryRepository<>));
        services.AddSingleton(typeof(IRepository<,>), typeof(InMemoryRepository<,>));
```

Listing 12.3 — ApplicationFixture.cs, corrected: the port has two type parameters, not one.

```
namespace Nexus.Application.Tests.Fakes;

public sealed class InMemoryRepository<TRoot, TId> : IRepository<TRoot, TId>
    where TRoot : Entity<TId>, IAggregateRoot  where TId : notnull
{
    private readonly Dictionary<TId, TRoot> _store = [];

    public Task<TRoot?> GetAsync(TId id, CancellationToken ct = default) =>
        Task.FromResult(_store.GetValueOrDefault(id));

    public Task AddAsync(TRoot root, CancellationToken ct = default)
    {
        _store[root.Id] = root;
        return Task.CompletedTask;
    }

    public void Remove(TRoot root) => _store.Remove(root.Id);
}

public sealed class FixedDateTimeProvider(DateTime fixedUtc) : IDateTimeProvider
{
    public DateTime UtcNow => fixedUtc;    // testable time: always the same o'clock
}

public sealed class InMemoryUnitOfWork : IUnitOfWork
{
    public int Commits { get; private set; }

    public Task CommitAsync(CancellationToken ct = default)
    {
        Commits++;
        return Task.CompletedTask;
    }
}
```

Listing 12.4 — tests/Nexus.Application.Tests/Fakes/, complete. Fakes are named fakes, printed in full.

Twenty lines of fake for three ports is itself a design review passed: ports shaped by consumer need are cheap to counterfeit, and a port whose fake takes an afternoon is a port that has smuggled in supplier convenience. *InMemoryUnitOfWork.Commits* is the one deliberate luxury — Chapter 14's tests will assert not just that state changed but that the business act was *committed exactly once*, and the counter is how they see it.

The seam, exercised once

```
namespace Nexus.Application.Tests;

public sealed class PortContractTests
{
    [Fact]
    public async Task A_root_added_is_the_root_returned()
    {
        var repo = new InMemoryRepository<Policy, PolicyId>();
        var policy = Policy.Draft(UnitId.New(), QTableId.New());

        await repo.AddAsync(policy);
        var loaded = await repo.GetAsync(policy.Id);

        Assert.Same(policy, loaded);   // reference-identical: fakes don't rehydrate
    }
}

$ dotnet test tests/Nexus.Application.Tests
Passed! - Failed: 0, Passed: 1   (Domain suite: 26, unchanged)
```

Listing 12.5 — tests/Nexus.Application.Tests/PortContractTests.cs, and the first run of the second suite.

The comment on the assertion is a small honest flag with a long reach: the fake returns the very instance it was handed, but Volume II's adapter will *rehydrate* — same identity, different object. Code written against this port may rely on *Equals* (Chapter 4 compares type and Id, and rehydration passes) but never on *ReferenceEquals* — the one behavioral gap between fake and adapter, named now so it ambushes no one in Volume II.

> **TRY IT**
>
> Prove the seam is one-directional. Open any Domain entity and add *using Nexus.Application.Abstractions;* with a field of type *IRepository<RootCauseCase, RootCauseCaseId>* — the domain trying to load itself. CS0246 again: Nexus.Domain holds no reference to Nexus.Application, and the arrow that would let an aggregate reach for a repository does not exist to be pointed. The core names its needs; it never serves them.

> **HONEST BOUNDARY**
>
> Two contested calls, argued rather than assumed. **Port placement:** a respectable DDD school declares repository interfaces in the Domain layer — "repository" is, after all, domain vocabulary. NEXUS-1 puts the ports in Application because no domain object may ever call one (entities are newed and rehydrated, never self-loading), so a Domain-resident port would be vocabulary without a speaker — but the other school trades that purity for keeping the whole model's language in one project, and neither choice is free. **The marker is structure, not semantics:** nothing stops a future developer tagging *Evidence* with IAggregateRoot and unlocking the forbidden port; the tag encodes the decision, not the reasoning. A statute-style reflection test could patrol it (only tagged types exposed by ports, no tagged type held by another root) — writing one is **Exercise 12.1**, with the Chapter 9 precedent as the pattern.

What the ports refuse to say

The repository's absences are load-bearing, and each points at a chapter. **No Update:** a loaded root is mutated through its own door and saved by the unit of work committing the business act; how change is *detected* is adapter machinery (EF's tracker, in Volume II), and a port that said *Update(root)* would be dictating the adapter's homework. **No queries:** no *GetAll*, no predicates, no paging. Loading a cluster to change it and reading data to display it are different jobs with different shapes, and jamming the second into the repository is how repositories grow forty methods; reads get their own citizens in Chapter 15. **No SaveChanges on the repository:** the transaction belongs to the unit of work, spans the one aggregate Chapter 8 allows, and carries the event-dispatch contract — three responsibilities that would dissolve if every repository could commit on its own initiative.

> **CHECKPOINT · CHAPTER 12**
>
> Before moving on, you should be able to answer these without looking back:
>
> **1.** State the ownership inversion in one sentence, and name the specific corruption it prevents on the day an adapter is finally written.
>
> **2.** The kernel freeze was audited and held. Reconstruct both halves of the argument that put IAggregateRoot in Domain instead.
>
> **3.** Why is the repository generic over the typed ID, and what regression does that make impossible for Volume II?
>
> **4.** Name the one behavioral gap between the in-memory fake and the future adapter, and the equality design that makes it survivable.
>
> **5.** Give the three refusals of the repository port and the chapter or volume each one points at.

The core can now be loaded, saved, and told the time. What it cannot yet do is receive a request — there is no shape for “an operator wants to acknowledge alarm X” to arrive in, no dispatcher to route it, and no rule about who is allowed to change state versus merely look at it. The next chapter introduces the split that organizes everything that remains: commands that write, queries that read, and the CQRS discipline that keeps them from becoming each other.

Interfaces, Sockets, and Signatures of the Future

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
> |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 27

The Abstractions folder earns its highlight: three ports declared, three named fakes signing them in the test project, and the Application suite opens with its first green.

What an interface literally is

An **interface** is a contract with no body: a list of member signatures — names, parameters, return types — and nothing else. It cannot be constructed, holds no state, and does no work; its entire power is the sentence *class X : IRepository<…>*, which is X **signing** the contract and the compiler auditing the signature: implement every member exactly as declared or fail to build. From that one mechanism comes the whole chapter: code written against the interface (*await repo.GetAsync(id)*) compiles today with no implementation in sight, runs tomorrow against whichever signatory is plugged in, and never contains a clue about which one it got. The interface is the socket; implementations are plugs; and the wall does not care what is plugged in.

Task, await, and the token — the future tense of C#

Task<TRoot?> is a **promise**: not the root itself but a receipt for one that will exist when slow work (a network hop, a disk read) finishes. *await* redeems the receipt without freezing the thread while waiting — the method pauses, the thread goes elsewhere, execution resumes when the value lands. Volume I's fakes finish instantly, so every await here redeems immediately; the signatures are written for Volume II's genuinely slow adapters, because — the chapter's phrase — retrofitting async onto a sync seam is surgery. The *CancellationToken* is the caller's hand on the cord: a signal threaded through every async call meaning "if I give up, you may stop"; each method passes it inward and slow implementations check it. Ignore both keywords' mechanics if you must, but keep the reading: **async signatures are the port admitting, honestly, that the world is slow.**

The seam, drawn as hardware

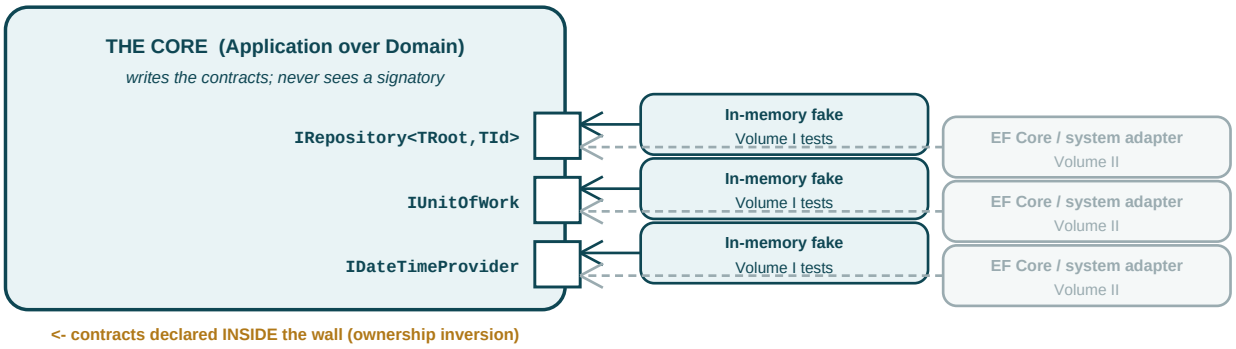


Figure D12.1 — Ports as sockets on the core's wall. Arrows point inward: plugs conform to sockets, never the reverse.

The same seam, in strict UML

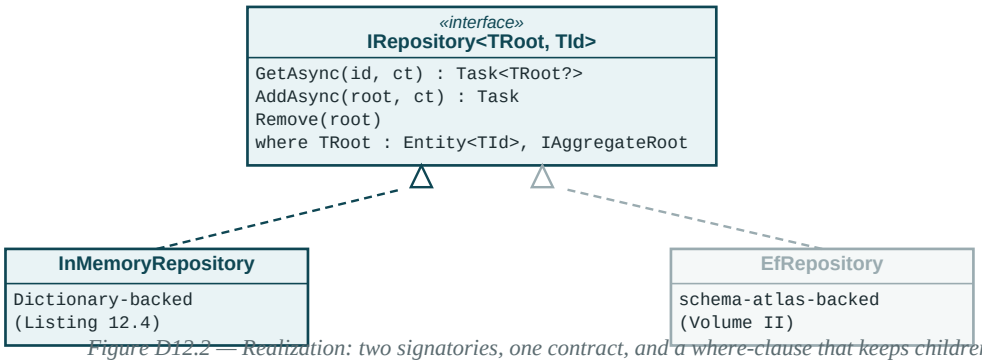


Figure D12.2 — Realization: two signatories, one contract, and a where-clause that keeps children out of the traffic.

Hold both figures against Listing 19.2 when you reach it: the contract test class is these dashed triangles made executable — every law a signatory must keep, run against whichever plug is in the socket. The gray box's three-line subclass is already drafted there, two volumes early.

13 · Introduction to CQRS

Two jobs wearing one model's clothes

Every use case remaining in this book does one of two things: it changes the plant's recorded state, or it looks at it. The insight of CQS — command–query separation — is that these are not two flavors of the same job but different jobs with opposite virtues. A **command** wants the full aggregate, its invariants armed, its transaction ready; it traffics in clusters and answers only success or a named refusal. A **query** wants none of that — no invariants fire when you look — it wants flat, screen-shaped data, cheaply, from as many tables as the screen needs. CQRS scales the initial to *Responsibility Segregation*: not just separate methods but separate *models*, each free to be good at its own job.

Why the split earns its keep here specifically

NEXUS-1's asymmetry is extreme and structural. The console redraws dashboards continuously — reads outnumber writes by orders of magnitude — and a single overview screen joins alarm state, case status, and policy badges across contexts, a shape no aggregate should ever have. Meanwhile the writes are precious and guarded: Chapter 8 spent a whole chapter making sure changing a case loads exactly one cluster through exactly one door. Serve both jobs with one model and each corrupts the other — aggregates sprout navigation properties to feed screens, or screens issue seventeen aggregate loads to draw one table. And the scoping, stated before the acronym inflates: **this is CQRS as two code paths, not two systems.** One store, one process, no event sourcing, no separate read database; Volume II maps both paths onto the same schema atlas. The segregation here is of *models and responsibilities*, which is the part that pays; the exotic infrastructure is a different book's gamble.

Naming the traffic

Commands are imperative sentences addressed to the plant — *AcknowledgeAlarmCommand*, *CloseCaseCommand* — and if a request can be refused, it is a command, the exact converse of Chapter 9's event test. Queries are questions — *GetOpenCasesQuery*, *GetAlarmHistoryQuery* — and may be repeated, cached, or abandoned without the plant noticing. The two vocabularies never mix: a command that returns a report or a query that tidies up state as a side hustle has crossed the segregation, and the contracts on the next page are written so the crossing has no type to ride on.

The contracts, in full

```
namespace Nexus.Application.Abstractions;

// the write side: commands mutate, and answer only through Chapter 4's channel
public interface ICommand : IRequest<Result> { }

public interface ICommand<TResponse> : IRequest<Result<TResponse>> { }

public interface ICommandHandler<TCommand>
    : IRequestHandler<TCommand, Result>
    where TCommand : ICommand { }

public interface ICommandHandler<TCommand, TResponse>
    : IRequestHandler<TCommand, Result<TResponse>>
    where TCommand : ICommand<TResponse> { }

// the read side: queries answer, and touch nothing
public interface IQuery<TResponse> : IRequest<Result<TResponse>> { }

public interface IQueryHandler<TQuery, TResponse>
    : IRequestHandler<TQuery, Result<TResponse>>
    where TQuery : IQuery<TResponse> { }
```

Listing 13.1 — src/Nexus.Application/Abstractions/Messaging.cs, complete. Thin, and thin on purpose.

Everything returns *Result*, and strict CQS raises an eyebrow: in the pure doctrine a command returns nothing at all. NEXUS-1 dissents twice, with reasons on the record. A command returns *Result* because refusal is this book's first-class citizen — “RootCause.CaseWithoutEvidence” must travel to whoever asked, and smuggling it through exceptions was rejected in Chapter 4. And a creating command may return *Result<TId>* because a client that just opened a case needs the case's identity for its very next breath, and forcing a follow-up query to learn what the command already knew is purity billing the user. What remains forbidden is returning *read data*: a command may confess an identity, never render a screen.

Note also what these interfaces sit on: *IRequest* and *IRequestHandler* are MediatR's types, which means the trial promised in Chapters 3 and 5 can be postponed no longer.

MediatR on trial, as promised

The charge, from Chapter 3's ledger: a third-party package sits inside an inner layer, routing every use case in the system. The defense begins by showing exactly what acquittal by removal would cost — here is the hand-rolled alternative, printed to be priced, not to be used:

```
public interface IDispatcher
{
    Task<TResponse> Send<TResponse>(IRequest<TResponse> request,
                                    CancellationToken ct = default);
}

public sealed class Dispatcher(IServiceProvider services) : IDispatcher
{
    public Task<TResponse> Send<TResponse>(IRequest<TResponse> request,
                                           CancellationToken ct = default)
    {
        var handlerType = typeof(IRequestHandler<,>)
            .MakeGenericType(request.GetType(), typeof(TResponse));

        dynamic handler = services.GetRequiredService(handlerType);
        return handler.Handle((dynamic)request, ct);
    }
}
```

Listing 13.2 — The thought experiment: a homegrown dispatcher (interface names ours, for the exercise).

Eighteen lines, one afternoon — and that is precisely the finding. The dispatcher was never the purchase. What MediatR is actually selling arrives in Chapter 16: the **pipeline** — open-generic behaviors wrapping every handler with validation, logging seams, and whatever cross-cutting discipline the trilogy adds later, composed by registration order and battle-tested across a decade of production systems. Hand-rolling *that* — behavior chaining, open-generic resolution, the reflective plumbing done fast and done right — is not an afternoon; it is a small framework with one maintainer, us. The verdict, recorded: **MediatR stays, for the pipeline, with the exit priced.** Every handler in this book implements *our* interfaces from Listing 13.1; MediatR's names appear in exactly one file. If the verdict ever reverses — and its licensing model's change in 2025 is exactly the kind of event that reopens such trials — the migration is Listing 13.2 grown a pipeline, plus a mechanical rename, not an archaeology dig through seventeen contexts.

The pipeline routes by type alone — proven once

One smoke test closes the wiring question before Chapter 14 builds real traffic: a diagnostic command, its handler, and the assertion that the fixture's provider routes the one to the other with no registration naming either. The fixture gains one honest line — test-local diagnostics live in the test assembly, which the production scan rightly does not cover:

```
        services.AddApplication();                    // the real registrations
        services.AddMediatR(c =>                      // plus test-local diagnostics
            c.RegisterServicesFromAssembly(typeof(EchoCommand).Assembly));
```

Listing 13.3 — ApplicationFixture.cs, one line grown for test-local handlers.

```
namespace Nexus.Application.Tests;

public sealed record EchoCommand(string Text) : ICommand<string>;

public sealed class EchoHandler : ICommandHandler<EchoCommand, string>
{
    public Task<Result<string>> Handle(EchoCommand cmd, CancellationToken ct) =>
        Task.FromResult(Result.Success(cmd.Text.ToUpperInvariant()));
}

public sealed class DispatchSmokeTests(ApplicationFixture fx)
    : IClassFixture<ApplicationFixture>
{
    [Fact]
    public async Task The_pipeline_routes_by_type_alone()
    {
        var sender = fx.Provider.GetRequiredService<ISender>();

        var result = await sender.Send(new EchoCommand("scram"));

        Assert.Equal("SCRAM", result.Value);
    }
}

$ dotnet test tests/Nexus.Application.Tests
Passed! - Failed: 0, Passed: 2   (Domain suite: 26, unchanged)
```

Listing 13.4 — tests/Nexus.Application.Tests/DispatchSmokeTests.cs, and the run.

Small as it is, the test pins the property everything after this relies on: **a command's type is its address.** Nothing anywhere maps EchoCommand to EchoHandler; the closed generic *ICommandHandler<EchoCommand, string>* is the routing table. Chapter 14 sends real commands down this exact wire.

SEE IT IN NEXUS-1 · The console, re-read as traffic

Re-open any console screen with this chapter's eyes and the UI dissolves into the two vocabularies. Every panel, table, gauge, and graph is a query's answer; every button that changes anything — ACK, SHELVE, CLOSE CASE, ACTIVATE POLICY — is a command with a refusal it must be prepared to display. The top-bar UNIT selector is neither: it re-parameterizes every query on screen, changing what is asked without asking the plant to change — the segregation, visible in a single click.

HONEST BOUNDARY

Three concessions on the record. **The name overshoots the practice:** much of the literature's CQRS — separate stores, projections, event sourcing — is deliberately absent, and a reader hired onto a system with those things will find this book's foundation compatible but not sufficient. **Strict CQS is genuinely violated:** Result-returning commands and identity-returning creators are dissents this chapter argued, not oversights; the purist position costs a round trip per creation and moves refusals to exceptions, and teams have chosen it with open eyes. **The verdict on MediatR is contingent, and says so:** it holds while the pipeline's value exceeds the dependency's cost, both of which move — the licensing change of 2025 moved one of them already. The wrap is what converts that contingency from a threat into a line item.

CHECKPOINT · CHAPTER 13

Before moving on, you should be able to answer these without looking back:

1. Give the command test and the query test, and explain how the command test is Chapter 9's event test reflected.

2. Argue the split from NEXUS-1's asymmetry: what would aggregates grow, and what would screens pay, if one model served both jobs?

3. State both dissents from strict CQS and the price of the purist alternative for each.

4. What was MediatR actually convicted of providing, and why is Listing 13.2 evidence for the defense rather than the prosecution?

5. “A command's type is its address.” What test pinned it, and what single file would change if the trial's verdict ever reversed?

The vocabulary exists and the wire is proven. The next chapter puts real traffic on it: the first production commands — acknowledging an alarm, closing a case — with handlers that orchestrate ports and aggregates without deciding a single business rule themselves, and the test discipline that keeps them that honest.

One Bus, Two Lanes, Nested Rings

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
> |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 28

The messaging contracts exist and the wire is proven: EchoCommand found EchoHandler with no registration naming either. Every later use case rides exactly this dispatch.

MediatR in four honest sentences

MediatR is an in-process message router. You hand *ISender* an object; it inspects the object's type; it looks up, in the DI container, the one handler whose interface names that exact type; it invokes it and returns the answer. Around that lookup it offers **pipeline behaviors** — open-generic wrappers that run before and after every handler — which Chapter 13's trial identified as the actual purchase. Nothing about it is magic and nothing about it is HTTP: it is method dispatch by type, indirected through the container, in the same process and on the same stack.

Marker interfaces: types whose job is to be a type

ICommand adds no members — so what is it for? Three jobs, all compile-time. **Routing:** the closed generic *ICommandHandler<CloseCaseCommand>* is the routing-table entry; the marker is what makes the generic writable. **Constraint:** *where TCommand : ICommand* lets the pipeline say "write-side traffic only" in a where-clause. **Declaration of intent:** a record implementing ICommand has told every reader its category before one property is read — the same trick as IAggregateRoot in Chapter 12, and the same lesson: **in a typed language, the cheapest documentation is a type.** The Result-typed responses thread the whole pipeline for one reason worth naming now: because every response *is* a Result, Chapter 16's behavior can manufacture a refusal of the right shape without ever reaching a handler — the constraint is the pipeline's license to speak.

The segregation, as road layout

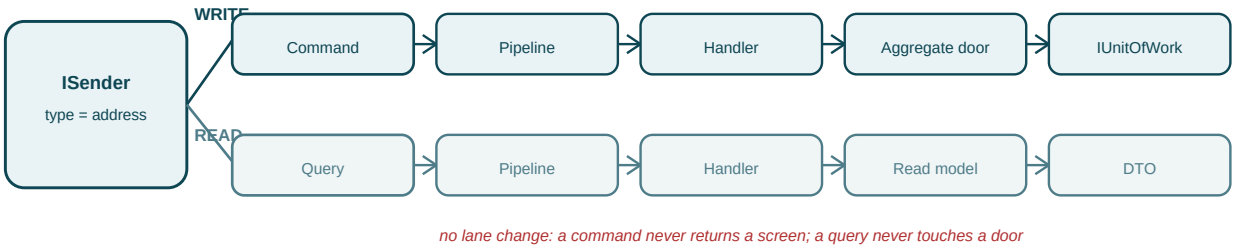


Figure D13.1 — One entrance, two lanes. The fork happens at the type; everything downstream is lane-specific.

The pipeline, as nesting

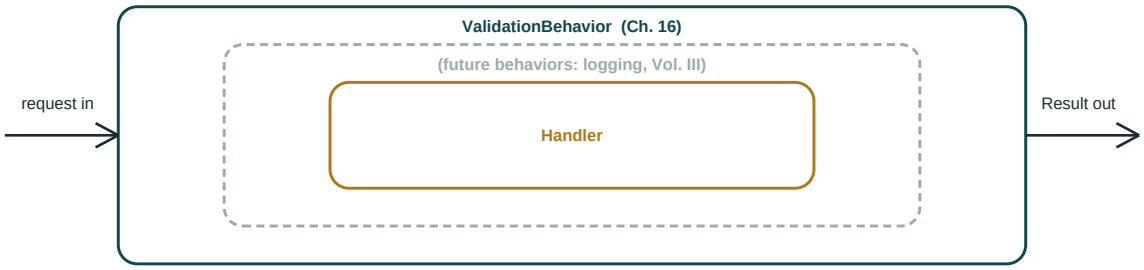


Figure D13.2 — Behaviors nest like rings: each may pass inward, or answer on the spot. A request that dies in the outer ring never wakes the handler.

Two figures, one route map for the rest of the volume: Chapter 14 builds the WRITE lane's last three boxes, Chapter 15 the READ lane's, and Chapter 16 arms the outer ring both lanes share. When Listing 16.5 later asserts "no NotFound, therefore the repository was never consulted," it is asserting Figure D13.2's geometry: the request died one ring out.

14 · Commands and Handlers

The handler's oath: orchestrate, never decide

A command handler has one rhythm and one oath. The rhythm: **load, act, commit** — fetch the aggregate through a port, invoke the business act through its door, commit the unit of work if and only if the domain said yes. The oath: **the handler decides nothing.** Every rule this book has built lives behind an aggregate's door, and the handler's job is to carry requests to that door and answers back — a courier with a security clearance, not a deputy judge. The litmus is mechanical: *the only ifs permitted in a handler are absence checks and result forwarding.* The moment a handler compares a Status or counts an evidence list, a rule has escaped the domain and is now enforced in a place with no invariants and no watched-to-fail test.

Absence is Application vocabulary

One refusal genuinely belongs to this layer. “No such alarm” is not a domain rule — an aggregate cannot refuse to exist; the case of the missing case has no door to be checked at. Absence is discovered by whoever holds the repository, and so the Application layer owns its first and only error family:

```
namespace Nexus.Application.Abstractions;

public static class ApplicationErrors
{
    public static Error NotFound(string what, Guid id) =>
        new($"{what}.NotFound", $"No {what} exists with id {id:N}.");
}

// src/Nexus.Application/AlarmManagement/Commands/:
public sealed record AcknowledgeAlarmCommand(AlarmEventId AlarmId, OperatorId By)
    : ICommand;

// src/Nexus.Application/RootCause/Commands/:
public sealed record OpenCaseCommand(UnitId UnitId, AlarmEventId TriggeredBy)
    : ICommand<RootCauseCaseId>;

public sealed record CloseCaseCommand(RootCauseCaseId CaseId, CauseNodeId ConfirmedCause)
    : ICommand;
```

Listing 14.1 — The first production commands, and the Application layer's one error of its own.

Commands are records of intent, nothing more: typed IDs and values, no entities, no behavior — the write-side siblings of Chapter 9's events, imperative where events are past tense. Note what *AcknowledgeAlarmCommand* carries and does not verify: *By* is a claim of identity, not a proof; who may actually acknowledge is priced on the last page.

The rhythm, in full

```
namespace Nexus.Application.AlarmManagement.Commands;

public sealed class AcknowledgeAlarmHandler(
    IRepository<AlarmEvent, AlarmEventId> alarms,
    IUnitOfWork uow,
    IDateTimeProvider clock) : ICommandHandler<AcknowledgeAlarmCommand>
{
    public async Task<Result> Handle(AcknowledgeAlarmCommand cmd, CancellationToken ct)
    {
        var alarm = await alarms.GetAsync(cmd.AlarmId, ct);           // load
        if (alarm is null)
            return Result.Failure(
                ApplicationErrors.NotFound("AlarmEvent", cmd.AlarmId.Value));

        var acknowledged = alarm.Acknowledge(cmd.By, clock.UtcNow);   // act
        if (acknowledged.IsFailure)
            return acknowledged;         // the domain has spoken: forward verbatim

        await uow.CommitAsync(ct);       // commit: one business act, one commit
        return Result.Success();
    }
}
```

Listing 14.2 — AcknowledgeAlarmHandler.cs, complete. Load, act, commit — and not one decision.

Twenty-two lines carrying four chapters. *clock.UtcNow* is Chapter 6's no-UtcNow law finally consuming its port — time enters the domain here, as a value, from the one lawful source. The verbatim forward is a rule of its own: a handler never rewraps, translates, or "improves" a domain refusal, because *AlarmManagement.NotRaised* is a published name that Chapter 6's tests pinned and Volume II's problem-details mapping will consume — editing it in transit would be the courier rewriting the letter. And the commit sits *after* the act and *behind* the success check: a refused command must cost the plant nothing, a claim the tests on page 101 make numeric. Both constructor injection promises from Chapter 5 land here too — the signature is the complete dependency manifest, and the primary constructor keeps even the ceremony thin.

A creator, and a closer

```
namespace Nexus.Application.RootCause.Commands;

public sealed class OpenCaseHandler(
    IRepository<RootCauseCase, RootCauseCaseId> cases,
    IUnitOfWork uow,
    IDateTimeProvider clock) : ICommandHandler<OpenCaseCommand, RootCauseCaseId>
{
    public async Task<Result<RootCauseCaseId>> Handle(
        OpenCaseCommand cmd, CancellationToken ct)
    {
        var rcc = RootCauseCase.Open(cmd.UnitId, cmd.TriggeredBy, clock.UtcNow);

        await cases.AddAsync(rcc, ct);
        await uow.CommitAsync(ct);

        return Result.Success(rcc.Id);   // the creator confesses the identity
    }
}

public sealed class CloseCaseHandler(
    IRepository<RootCauseCase, RootCauseCaseId> cases,
    IUnitOfWork uow,
    IDateTimeProvider clock) : ICommandHandler<CloseCaseCommand>
{
    public async Task<Result> Handle(CloseCaseCommand cmd, CancellationToken ct)
    {
        var rcc = await cases.GetAsync(cmd.CaseId, ct);
        if (rcc is null)
            return Result.Failure(
                ApplicationErrors.NotFound("RootCauseCase", cmd.CaseId.Value));

        var closed = rcc.Close(cmd.ConfirmedCause, clock.UtcNow);
        if (closed.IsFailure)
            return closed;

        await uow.CommitAsync(ct);
        return Result.Success();
    }
}
```

Listing 14.3 — OpenCaseHandler.cs and CloseCaseHandler.cs, complete.

OpenCaseHandler contains no *if* at all, and the absence is doctrine, not luck: Chapter 8's factory cannot fail, so there is no refusal to forward — the handler-side twin of *Learn* returning void in Chapter 11. It also shows the second dissent of Chapter 13 earning its keep: the caller gets the new case's identity in the command's own answer, no follow-up query, no guessing. *CloseCaseHandler*, meanwhile, is Listing 14.2 with the nouns changed — and that sameness is the chapter's quiet claim: once the rhythm is right, the write side of seventeen contexts is this shape, repeated.

Three paths, one counter

Handlers are tested the way Chapter 5 promised: *new*-ed by hand with the named fakes — no fixture, no container, no pipeline. Three paths cover the space: the legal act (state changed, committed exactly once), the domain refusal (name intact, and — the counter's payoff — *zero* commits), and the absence (the Application layer speaking in its own voice).

```
public sealed class CloseCaseHandlerTests
{
    private static readonly DateTime T0 = new(2026, 7, 2, 6, 0, 0, DateTimeKind.Utc);

    private readonly InMemoryRepository<RootCauseCase, RootCauseCaseId> _cases = new();
    private readonly InMemoryUnitOfWork _uow = new();

    private CloseCaseHandler Handler() => new(_cases, _uow, new FixedDateTimeProvider(T0));

    [Fact]
    public async Task A_legal_close_changes_state_and_commits_exactly_once()
    {
        var rcc = RootCauseCase.Open(UnitId.New(), AlarmEventId.New(), T0);
        rcc.AttachEvidence("hotwell trend", TagId.New(), T0);
        await _cases.AddAsync(rcc);

        var result = await Handler().Handle(
            new CloseCaseCommand(rcc.Id, new CauseNodeId(Guid.NewGuid())), default);

        Assert.True(result.IsSuccess);
        Assert.Equal(CaseStatus.Closed, rcc.Status);
        Assert.Equal(1, _uow.Commits);
    }

    [Fact]
    public async Task A_domain_refusal_arrives_verbatim_and_commits_nothing()  // watched
    {
        var rcc = RootCauseCase.Open(UnitId.New(), AlarmEventId.New(), T0);  // no evidence
        await _cases.AddAsync(rcc);

        var result = await Handler().Handle(
            new CloseCaseCommand(rcc.Id, new CauseNodeId(Guid.NewGuid())), default);

        Assert.Equal("RootCause.CaseWithoutEvidence", result.Error.Code);
        Assert.Equal(0, _uow.Commits);        // a refusal costs the plant nothing
    }

    [Fact]
    public async Task A_missing_case_is_the_application_layer_speaking()
    {
        var result = await Handler().Handle(
            new CloseCaseCommand(RootCauseCaseId.New(),
                                 new CauseNodeId(Guid.NewGuid())), default);

        Assert.Equal("RootCauseCase.NotFound", result.Error.Code);
    }
}

$ dotnet test tests/Nexus.Application.Tests
Passed! - Failed: 0, Passed: 5   (Domain suite: 26, unchanged)
```

Listing 14.4 — tests/Nexus.Application.Tests/RootCause/CloseCaseHandlerTests.cs, and the run

HONEST BOUNDARY

Three absences, named before anyone mistakes them for oversights. **No authorization:** *By* travels on faith; whether that operator may acknowledge that alarm is the Security context's question and the API layer's enforcement, both Volume II — the command carries the claim so the check has something to check. **No idempotency:** send *CloseCaseCommand* twice and the domain's own sealed-record refusal absorbs it benignly, but send *OpenCaseCommand* twice and two cases exist; retry semantics and deduplication are delivery-infrastructure concerns, deferred with the outbox to Volume II. **Commit is still a counter:** "exactly once" currently means the fake incremented; that the same line becomes a real transaction boundary wrapping the aggregate is precisely the promise the ports carry into Volume II, where the same tests re-run against the adapter.

CHECKPOINT · CHAPTER 14

Before moving on, you should be able to answer these without looking back:

1. Recite the rhythm and the oath, and state the mechanical litmus that detects an escaped rule.

2. Why is NotFound the Application layer's error and not the domain's? Answer with the door metaphor.

3. Defend the verbatim forward: who are the two consumers of "RootCause.CaseWithoutEvidence" that a rewrapping handler would betray?

4. *OpenCaseHandler* has no ifs. Connect that absence to two earlier design decisions, one per layer.

5. What exactly does *Assert.Equal(0, _uow.Commits)* prove, and what is the same assertion scheduled to prove in Volume II?

The write side is a solved shape. But every command here was tested by conjuring aggregates into a fake repository — the console cannot do that. Screens need to *ask*: which alarms are raised right now, which cases are open, what does the overview owe the operator — questions whose answers span contexts and fit no aggregate. The next chapter builds the read side: queries, the DTOs they answer with, and the read-model port that finally gives Chapter 10's flood judge its promised honest feeding.

The Rhythm, Frame by Frame

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
> |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 31

Three production handlers exist and the Application suite holds five: the write lane of Figure D13.1 now runs end to end against the named fakes.

Reading a handler top to bottom, with the litmus in hand

A well-formed handler reads like a checklist because it is one. Line one: **load** — an await on a port, followed by the absence check (if null, the Application speaks its one error). Middle: **act** — exactly one call through the aggregate's door, followed by the forward check (if the domain refused, relay the refusal verbatim and stop). Last: **commit**, then Success. Anything else you find — a status comparison, a count, a calculation — is Chapter 14's litmus firing: a business rule has escaped the domain and is squatting in orchestration. Train the eye on the two permitted if-shapes (*is null*, *IsFailure*) until any third shape looks wrong on sight; that reflex is most of what code review checks in this layer.

Why the refusal must travel untouched

The temptation in every handler is to "improve" a domain refusal — wrap it, translate it, add context. The book forbids it because the refusal's *code* is a published name with subscribers: Chapter 6's tests assert it verbatim, Chapter 18's inventory audits by it, and Volume II's API will map it — unchanged — onto an HTTP problem-details response a client can switch on. A handler that rewraps is a courier editing a letter between two parties who both keep copies: the edit cannot help and can only desynchronize. The sequence diagrams overleaf make the rule visible — watch the dashed answers cross the handler's lifeline without changing text.

Scenario one: the legal act

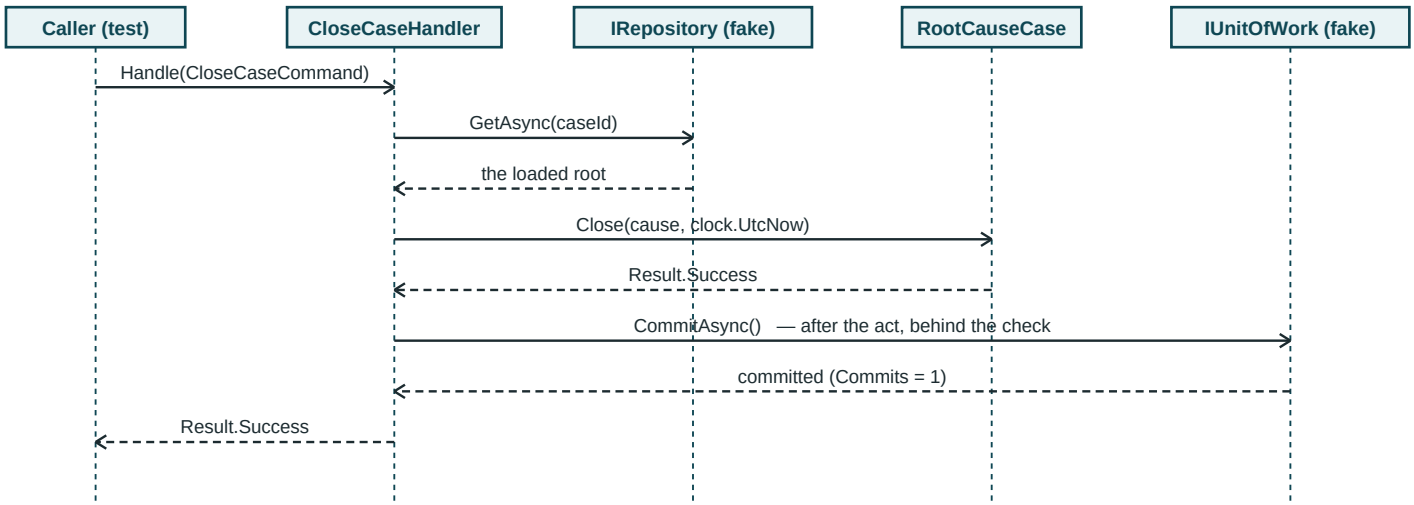


Figure D14.1 — Load, act, commit: the success path of Listing 14.3, one commit, in order.

Scenario two: the refusal, costing nothing

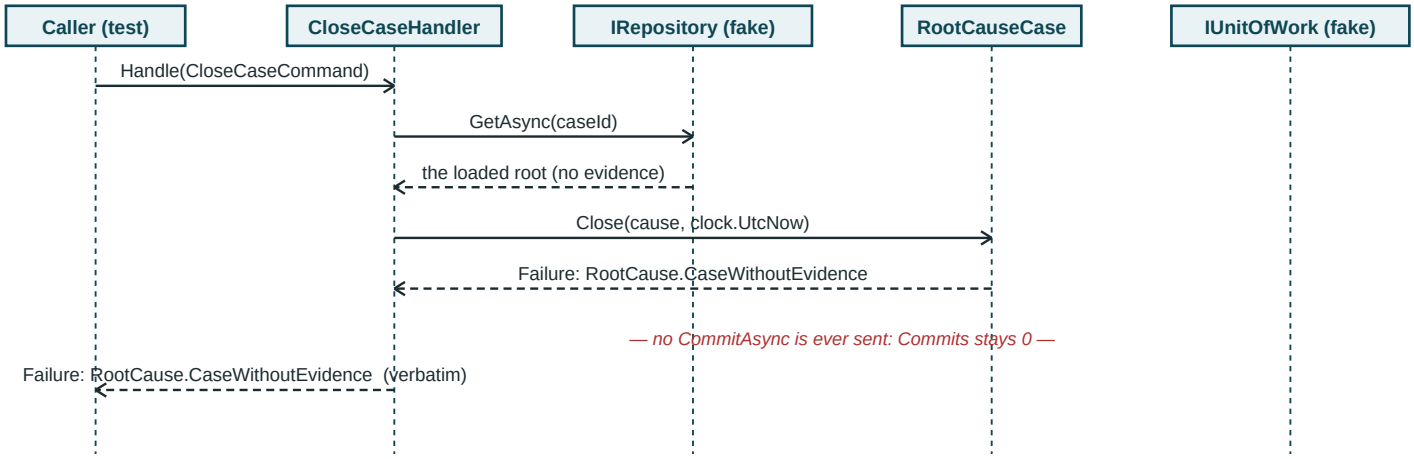


Figure D14.2 — The same wire when the flagship refuses: the answer relays verbatim, and the commit arrow does not exist.

Set the two figures side by side and Listing 14.4's three tests become three tracings: the success test walks D14.1 and counts one commit; the watched-to-fail walks D14.2 and asserts both the unchanged text on the last arrow and the red note in the middle; the not-found test is D14.2 cut short at the third arrow, with the Application speaking instead of the domain. When a handler test fails in your own work, draw its scenario in this notation before touching code — the arrow that differs from the drawing is, almost always, the bug.

15 · Queries, DTOs, and Read Models

The read side owes the doors nothing

A screen asking “which cases are open in Unit 2?” does not want a cluster — it wants five columns, cheap. Serving it through repositories would load whole aggregates, arm their invariants, and drag their children into memory, all to render a table that mutates nothing: paying the invariant tax to window-shop. The read side therefore bypasses the aggregates entirely. Its citizens are three: the **DTO** — a flat, screen-shaped, sealed record with no behavior and an honest suffix; the **read-model port** — an Application-owned interface that answers in DTOs, Chapter 12's ownership inversion applied to reading; and the **query handler** — usually a single forwarding line, because with no rules to guard there is nothing to orchestrate.

A clarification the convention has been waiting for

One DTO below joins alarm counts, case counts, and a policy name — three contexts in one shape, which looks like a felony against Chapter 9's “IDs and events, nothing else, ever.” It is not, and the reason sharpens what the convention was always for: **the cross-context rule protects writers.** It exists so no context can reach into another's model and mutate, couple, or depend on its internals. A read model does none of that — it is a projection of *committed* data, owned by the Application layer, mutating nothing, coupling to published outcomes rather than living objects. Writers converse through IDs and events; readers survey the committed whole. Both sentences are the same boundary, seen from its two sides.

```
namespace Nexus.Application.RootCause.Queries;

public sealed record OpenCaseSummaryDto(
    RootCauseCaseId CaseId,
    UnitId UnitId,
    DateTime OpenedAtUtc,
    int UnstruckEvidenceCount,
    string TopRankedCause);

namespace Nexus.Application.CorePlatform.Queries;

public sealed record UnitOverviewDto(
    UnitId UnitId,
    int RaisedAlarms,
    int OpenCases,
    string? ActivePolicyName);   // three contexts, one shape, zero aggregates loaded
```

Listing 15.1 — The read side's vocabulary: flat, screen-shaped, honestly suffixed.

The ports of the read side — and a debt's mechanism

Chapter 10 left the flood judge with a named gap: a correct function is not a correctly *fed* function, and nothing guaranteed the judge sees every raise in its window. The guarantee lives here, as contract language on the read-model port — completeness is what the interface *means*, written where every implementor must read it:

```
namespace Nexus.Application.Abstractions;

public interface IAlarmReadModel
{
    /// <summary>ALL raise instants in (sinceUtc, now]; completeness is the
    /// contract -- the flood verdict is only as honest as this list.</summary>
    Task<IReadOnlyList<DateTime>> RaiseTimesSinceAsync(
        UnitId unitId, DateTime sinceUtc, CancellationToken ct = default);
}

public interface ICaseReadModel
{
    Task<IReadOnlyList<OpenCaseSummaryDto>> OpenCasesAsync(
        UnitId unitId, CancellationToken ct = default);
}

public interface IOverviewReadModel
{
    Task<UnitOverviewDto> OverviewAsync(UnitId unitId, CancellationToken ct = default);
}
```

Listing 15.2 — src/Nexus.Application/Abstractions/ReadModels.cs, complete.

Three ports, three grains: a value list for a domain service's diet, a per-context summary for a table, a cross-context join for the overview — each shaped by exactly one consumer's need, which is what consumer ownership is for. Two disciplines ride along. **No unit of work in sight:** a query handler injecting *IUnitOfWork* is a design alarm with one meaning — someone is about to mutate inside a read. And **a freshness fact worth stating:** because this is CQRS-as-two-paths over one store, a read here is as fresh as the last commit; the eventual consistency of Chapter 9 lives *between aggregates*, never between a write and the read that follows it. The exotic staleness of two-store CQRS was declined in Chapter 13, and this is the dividend.

Two queries: a forwarder, and the debt paid

```
namespace Nexus.Application.RootCause.Queries;

public sealed record GetOpenCasesQuery(UnitId UnitId)
    : IQuery<IReadOnlyList<OpenCaseSummaryDto>>;

public sealed class GetOpenCasesHandler(ICaseReadModel cases)
    : IQueryHandler<GetOpenCasesQuery, IReadOnlyList<OpenCaseSummaryDto>>
{
    public async Task<Result<IReadOnlyList<OpenCaseSummaryDto>>> Handle(
        GetOpenCasesQuery q, CancellationToken ct) =>
        Result.Success(await cases.OpenCasesAsync(q.UnitId, ct));
}

namespace Nexus.Application.AlarmManagement.Queries;

public sealed record GetFloodStatusQuery(UnitId UnitId) : IQuery<FloodVerdict>;

public sealed class GetFloodStatusHandler(
    IAlarmReadModel alarms,
    FloodDetectionService judge,
    IDateTimeProvider clock) : IQueryHandler<GetFloodStatusQuery, FloodVerdict>
{
    public async Task<Result<FloodVerdict>> Handle(
        GetFloodStatusQuery q, CancellationToken ct)
    {
        var now = clock.UtcNow;

        var times = await alarms.RaiseTimesSinceAsync(
            q.UnitId, now - FloodThresholds.Default.Window, ct);

        return Result.Success(judge.Judge(times, now));
        // fetch the world, pass values in -- Chapter 5, kept to the letter
    }
}
```

Listing 15.3 — The read side's handlers: one line where nothing needs orchestrating, and Chapter 5's doctrine verbatim where it does.

The second handler is the sentence this book wrote in Chapter 5 finally executing unchanged: *the Application handler receives the service and passes values into the domain.* The read model fetches the world; the injected judge — Chapter 10's singleton — receives times and a now, and computes; nothing in the Domain layer learned what a port is. The window arithmetic sits in the handler because it is *feeding*, not judging: deciding what the judge sees is orchestration, deciding what it means is domain, and the line between them is exactly the line between these two objects.

The debt, formally settled

The ledger entry from Table 11.1 — “the flood judge correctly fed, deferred again” — closes here with a contract, a seedable fake, and two tests: one proving ten honest raises reach the judge and return a flood, one proving the feeding respects the unit boundary, so no plant floods on its neighbor's noise.

```
public sealed class InMemoryAlarmReadModel : IAlarmReadModel
{
    private readonly List<(UnitId Unit, DateTime At)> _raises = [];

    public void Seed(UnitId unit, params DateTime[] at) =>
        _raises.AddRange(at.Select(t => (unit, t)));
    public Task<IReadOnlyList<DateTime>> RaiseTimesSinceAsync(
        UnitId unitId, DateTime sinceUtc, CancellationToken ct = default) =>
        Task.FromResult<IReadOnlyList<DateTime>>(
            [.. _raises.Where(r => r.Unit == unitId && r.At > sinceUtc)
                       .Select(r => r.At)]);
}

public sealed class GetFloodStatusHandlerTests
{
    private static readonly DateTime T0 = new(2026, 7, 2, 7, 0, 0, DateTimeKind.Utc);

    private static GetFloodStatusHandler Handler(IAlarmReadModel reads) => new(
        reads, new FloodDetectionService(FloodThresholds.Default),
        new FixedDateTimeProvider(T0));

    [Fact]
    public async Task Ten_recent_raises_reach_the_judge_and_return_a_flood()
    {
        var unit = UnitId.New();
        var reads = new InMemoryAlarmReadModel();
        reads.Seed(unit, [.. Enumerable.Range(0, 10)
                              .Select(i => T0.AddSeconds(-30 * i))]);

        var verdict = await Handler(reads)
            .Handle(new GetFloodStatusQuery(unit), default);

        Assert.True(verdict.Value.IsFlood);
        Assert.Equal(10, verdict.Value.AlarmsInWindow);
    }

    [Fact]
    public async Task Another_units_raises_do_not_flood_this_one()   // scope pinned
    {
        var reads = new InMemoryAlarmReadModel();
        reads.Seed(UnitId.New(), [.. Enumerable.Range(0, 20)
                                     .Select(i => T0.AddSeconds(-i))]);
        var verdict = await Handler(reads)
            .Handle(new GetFloodStatusQuery(UnitId.New()), default);
        Assert.False(verdict.Value.IsFlood);
    }
}

$ dotnet test tests/Nexus.Application.Tests
Passed! - Failed: 0, Passed: 7   (Domain suite: 26, unchanged)
```

Listing 15.4 — tests/Nexus.Application.Tests/AlarmManagement/, complete, and the run.

SEE IT IN NEXUS-1 · The overview is a DTO with pixels

The console's home screen — alarm badge, open-case count, active-policy chip per unit — is *UnitOverviewDto* rendered, one query per unit card. Flip the UNIT selector and watch every card re-ask its question with a new parameter: the read side's whole economy in a click. And the FLOOD banner of Chapter 10 now has its full supply chain on paper — read model to handler to judge to ribbon — with the completeness contract as the link the pixels silently trust.

HONEST BOUNDARY

Three seams, marked. **Every read model here is a port with only a test fake behind it:** the production adapters — SQL projections against the schema atlas, the natural home of a five-column summary — are Volume II's opening act, and the completeness contract will be theirs to honor against real indexes. **Domain types cross into answers:** *FloodVerdict* and typed IDs travel inside query responses, legal while everything shares one process; the API boundary of Volume II is where they get mapped to primitives, and pretending otherwise now would be inventing a serialization problem this volume does not have. **TopRankedCause is a display string:** the DTO carries the name, not the *CauseNodeId*, because screens print names — but a client that needs to *act* on the ranking would need the ID, and adding it is the read model's cheapest future change.

CHECKPOINT · CHAPTER 15

Before moving on, you should be able to answer these without looking back:

1. What is the invariant tax, and why does the read side lawfully evade it?

2. Restate the cross-context convention as its two-sided form, and explain why the overview DTO commits no felony.

3. Where does the flood window arithmetic live, and why there rather than one object in either direction?

4. Why is a query handler injecting IUnitOfWork a one-meaning alarm?

5. State the freshness fact of one-store CQRS, and which chapter's eventual consistency it does not contradict.

Both sides of the segregation now carry traffic — and every handler trusts its command blindly. Nothing yet checks that an AlarmEventId is not empty, that a cause name is not whitespace, that the request deserves a handler's time at all; and nothing wraps the traffic in the cross-cutting discipline MediatR was retained to enable. The next chapter buys what the trial paid for: validation, and the pipeline behaviors that guard every door with the same checks.

Shapes for Jobs

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
> |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 33

The read side exists: three DTOs, three read-model ports, two query handlers — and the twice-deferred flood-feeding debt is formally settled with two tests.

DTO: the least clever object in the book, on purpose

A **data transfer object** is pure shape: named fields, no behavior, no rules, no opinions. Where every other citizen in this book earns its keep by *refusing* things, a DTO's virtue is that it cannot refuse anything — it is a tray, built to match one screen's appetite exactly, carried from a read model to a renderer and thrown away. Newcomers sometimes feel DTOs are duplication ("the case already has these fields!"); the figure below is the answer. The aggregate and the summary hold overlapping *data* but are opposite *shapes*, optimized for opposite jobs — and forcing one shape to do both jobs is how systems end up with aggregates full of UI conveniences and screens paying transaction prices.

Projection: the verb behind every read model

The read side's core verb is **project**: take rich, guarded, committed state and cast its flat shadow — a count instead of a collection, a name instead of a graph node, a boolean instead of a lifecycle. Projection loses information *deliberately*; that is what makes it cheap and safe. Nothing guarded is bypassed, because nothing is changed: a shadow cannot violate the invariants of the object casting it. This is also why the read side needs no watched-to-fail tests of business rules — there are none here to watch — and why its honest risks are different ones: completeness (Chapter 15's contract language) and freshness (settled by the one-store decision).

Same data, two shapes

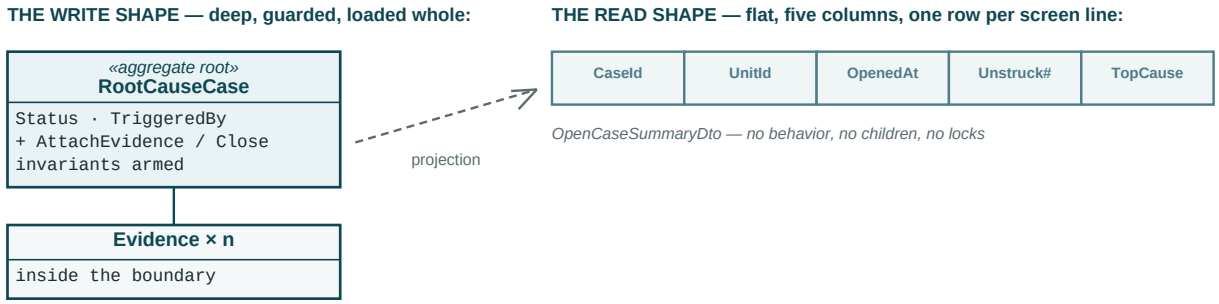


Figure D15.1 — One truth, two shapes: the cluster serves change; its flat shadow serves the screen. The dashed arrow only ever points right.

A query's whole life, with one deliberate absence

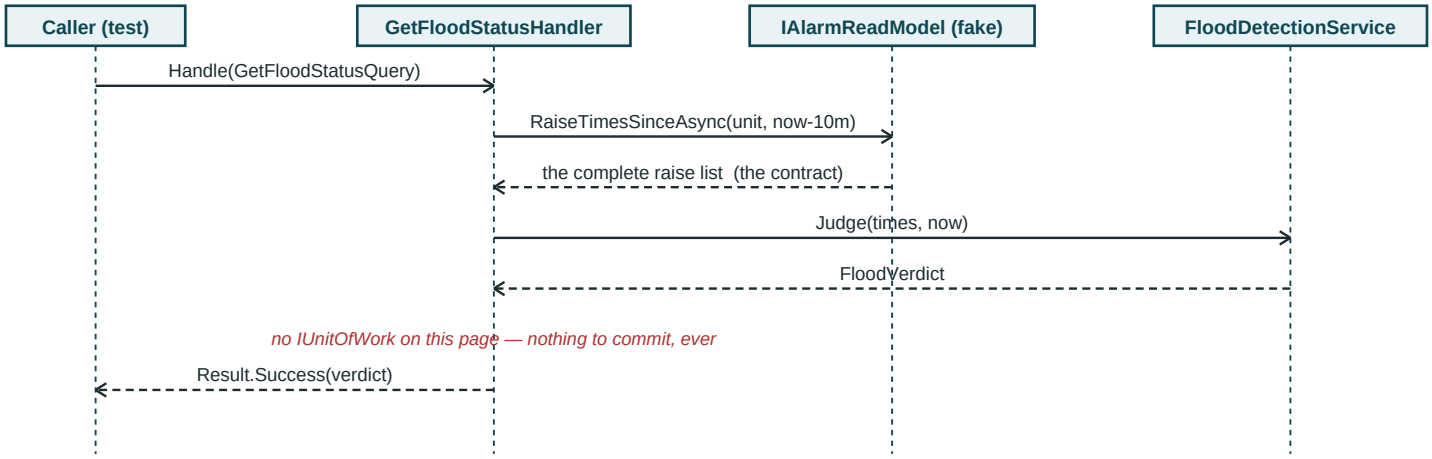


Figure D15.2 — The flood query end to end. Find what is missing from the cast list; that absence is the read side's constitution.

Compare the cast here with Figure D14.1's: the unit of work — present in every write scenario — is not merely idle but *uninvited*, and the red note makes the absence part of the record. The middle arrows are also Chapter 5's doctrine in miniature one more time: the handler fetches the world (arrow 2), then passes values into a judge that has never heard of ports (arrow 4). If you can explain why arrows 2 and 4 must not swap owners, the Application layer has no secrets left for you.

16 · Validation and Pipeline Behaviors

Validators inspect the letter; aggregates judge the act

Two kinds of “no” guard a command, and confusing them undoes chapters of work. **Validation** checks the *message*: is this request well-formed — IDs present, strings non-blank, numbers in sane ranges — before anyone spends a repository call on it? **Invariants** check the *world*: given the aggregate's actual state, may this act happen? The first needs no data and belongs in front of the handler; the second needs the cluster and lives behind its door, where Part II put it. The corollary is a hard rule: **a validator never restates a domain rule.** A validator that checks evidence counts has copied the flagship into a place with no aggregate, no lock, and no watched-to-fail test — a stale duplicate that will disagree with the original under concurrency and win the argument by answering first.

```
namespace Nexus.Application.RootCause.Commands;

public sealed class CloseCaseCommandValidator : AbstractValidator<CloseCaseCommand>
{
    public CloseCaseCommandValidator()
    {
        RuleFor(c => c.CaseId.Value).NotEmpty();
        RuleFor(c => c.ConfirmedCause.Value).NotEmpty();
        // and pointedly NOT: RuleFor(c => ...evidence...)  -- that is the world's rule
    }
}
```

Listing 16.1 — CloseCaseCommandValidator.cs, complete. Message checks only; the flagship stays home.

Validators are discovered by the assembly scan Chapter 5 registered and never invoked by hand — which raises the real question of this chapter: *who calls them?* Nothing in any handler mentions validation, and nothing will. The pipeline does — the purchase the MediatR trial upheld, collected on the next page.

The behavior: every door, the same guard

```
namespace Nexus.Application.Behaviors;

public sealed class ValidationBehavior<TRequest, TResponse>(
    IEnumerable<IValidator<TRequest>> validators)
    : IPipelineBehavior<TRequest, TResponse>
    where TRequest : IRequest<TResponse>
    where TResponse : Result
{
    public async Task<TResponse> Handle(TRequest request,
        RequestHandlerDelegate<TResponse> next, CancellationToken ct)
    {
        if (!validators.Any())
            return await next();                    // nothing registered: pass through

        var failure = validators
            .Select(v => v.Validate(request))
            .SelectMany(r => r.Errors)
            .FirstOrDefault();

        if (failure is null)
            return await next();                    // well-formed: pass through

        return Failure(new Error(
            $"Validation.{failure.PropertyName}", failure.ErrorMessage));
    }

    // the one reflective wart in the book, quarantined and commented:
    // Result has no type parameter; Result<T> needs its T summoned at runtime.
    private static TResponse Failure(Error e) =>
        typeof(TResponse) == typeof(Result)
            ? (TResponse)Result.Failure(e)
            : (TResponse)typeof(Result)
                .GetMethod(nameof(Result.Failure), 1, [typeof(Error)])!
                .MakeGenericMethod(typeof(TResponse).GetGenericArguments()[0])
                .Invoke(null, [e])!;
}
```

Listing 16.2 — src/Nexus.Application/Behaviors/ValidationBehavior.cs, complete.

Read the shape before the wart. This one open-generic class wraps *every* command and query in the system — present and future, seventeen contexts' worth — with the same guard, registered once; that is the pipeline dividend Listing 13.2's homegrown dispatcher could not pay. Malformed requests die here, before any handler runs, any repository is asked, or any refusal is manufactured — and they die through the ordinary channel, as a *Result* whose code says *Validation.<Property>*, not as an exception dressed up as flow control. The wart is real and priced: constructing *Result<T>* for an unknown T takes one reflective factory call, isolated in one private method with its excuse written above it. A codebase allergic to even that can generate the closed types at compile time; this one takes the eight honest lines.

The other purchase: Chapter 9's plumbing arrives

The kernel's *IDomainEvent* knows nothing of MediatR — that was the point — so a bridge carries announcements into the pipeline: a wrapper notification, a dispatcher that wraps and publishes, and the reaction handler Chapter 9 designed the brain for. Together they are the plumbing that chapter deferred, and not one line of Domain code changes to receive them:

```
namespace Nexus.Application.Events;

public sealed record DomainEventNotification<TEvent>(TEvent DomainEvent)
    : INotification where TEvent : IDomainEvent;

public interface IDomainEventDispatcher
{
    Task DispatchAsync(IEnumerable<IDomainEvent> events, CancellationToken ct = default);
}

public sealed class DomainEventDispatcher(IPublisher publisher) : IDomainEventDispatcher
{
    public async Task DispatchAsync(IEnumerable<IDomainEvent> events,
                                    CancellationToken ct = default)
    {
        foreach (var e in events)
        {
            var notification = Activator.CreateInstance(
                typeof(DomainEventNotification<>).MakeGenericType(e.GetType()), e)!;

            await publisher.Publish(notification, ct);
        }
    }
}

// the reaction Chapter 9 promised: RootCause reacting to AlarmManagement's news
public sealed class OpenCaseOnCriticalAlarm(ISender sender)
    : INotificationHandler<DomainEventNotification<AlarmRaisedEvent>>
{
    public async Task Handle(DomainEventNotification<AlarmRaisedEvent> n,
                             CancellationToken ct)
    {
        if (!CaseOpeningPolicy.WarrantsInvestigation(n.DomainEvent))
            return;                       // the brain said no; the plumbing obeys

        await sender.Send(new OpenCaseCommand(
            n.DomainEvent.UnitId, n.DomainEvent.AlarmId), ct);
    }
}
```

Listing 16.3 — src/Nexus.Application/Events/, complete: the bridge, the dispatcher, the first reaction.

Three details carry the design. The reaction's *if* does not violate Chapter 14's litmus — it forwards a *domain verdict*; the brain decided, the plumbing obeys, which is Chapter 9's split executing. The reaction *sends a command* rather than touching a repository: the case gets opened through the same guarded write path as every other case, validation and all — reactions are clients, not insiders. And the dispatcher stands exactly where Chapter 9 placed the seam: in Volume I the tests hand it mailboxes; in Volume II the unit-of-work adapter calls this same class after commit, honoring the contract Listing 12.2 wrote into the port's documentation.

AddApplication, printed again, grown — as promised

```
        services.AddSingleton(new FloodDetectionService(FloodThresholds.Default));
        services.AddSingleton<CausalRankingService>();

        // pipeline behaviors (validation, logging seam) are added in Chapter 16,
        // in this same method -- it is printed again there, grown.
        services.AddTransient(
            typeof(IPipelineBehavior<,>), typeof(ValidationBehavior<,>));

        services.AddTransient<IDomainEventDispatcher, DomainEventDispatcher>();

        return services;
```

Listing 16.4 — DependencyInjection.cs: the growth ring Chapter 5 reserved, filled.

One open-generic registration arms the guard at every door; one transient gives the commit seam its dispatcher. And the first proof runs through the fixture — the full pipeline path, because the behavior only exists there:

```
public sealed class ValidationBehaviorTests(ApplicationFixture fx)
    : IClassFixture<ApplicationFixture>
{
    [Fact]
    public async Task A_malformed_command_never_reaches_its_handler()  // watched
    {
        var sender = fx.Provider.GetRequiredService<ISender>();

        var result = await sender.Send(new CloseCaseCommand(
            new RootCauseCaseId(Guid.Empty),           // the malformation
            new CauseNodeId(Guid.NewGuid())));

        Assert.True(result.IsFailure);
        Assert.Equal("Validation.CaseId.Value", result.Error.Code);
        // no NotFound: the repository was never consulted
    }
}
```

Listing 16.5 — ValidationBehaviorTests.cs: the malformed letter dies at the guard.

The last assertion's comment is the test's real finding: a *NotFound* here would mean the handler ran and asked the repository about *Guid.Empty*; the *Validation* code proves the request died at the guard, one layer earlier and one repository call cheaper — ordering made observable through nothing but error names.

The conversation, end to end

The chapter closes with the test Chapter 9 could only describe as a console animation: an alarm raised in one context, its announcement dispatched, the reaction firing, and a case existing in another context — with no line of code in either context aware of the other beyond an ID and a published event:

```
public sealed class ConversationTests(ApplicationFixture fx)
    : IClassFixture<ApplicationFixture>
{
    [Fact]
    public async Task A_critical_alarm_opens_a_case_two_contexts_away()
    {
        var dispatcher = fx.Provider.GetRequiredService<IDomainEventDispatcher>();
        var cases = fx.Provider
            .GetRequiredService<IRepository<RootCauseCase, RootCauseCaseId>>();

        // AlarmManagement announces (the test plays the commit seam, per Ch. 9):
        var alarm = AlarmEvent.Raise(UnitId.New(), TagId.New(),
            AlarmSeverity.Critical, "RCS pressure high", DateTime.UtcNow);

        await dispatcher.DispatchAsync(alarm.DomainEvents);
        alarm.ClearDomainEvents();          // collect, commit, dispatch, clear

        // RootCause has reacted -- through the full guarded write path:
        var opened = ((InMemoryRepository<RootCauseCase, RootCauseCaseId>)cases)
            .Single();
        Assert.Equal(alarm.Id, opened.TriggeredBy);
    }
}

// one line grows the fake for the assertion above:
// public TRoot Single() => _store.Values.Single();

$ dotnet test tests/Nexus.Application.Tests
Passed! - Failed: 0, Passed: 9   (Domain suite: 26, unchanged)
```

Listing 16.6 — ConversationTests.cs: announcement, policy, reaction — the whole of Chapter 9, executable.

Count what fired between the two asserts: the dispatcher wrapped and published; the reaction consulted the brain; the brain said Critical warrants a case; the reaction sent *OpenCaseCommand*; the pipeline validated it; the handler created, added, and committed. Six chapters of machinery, one method call wide — and every piece individually pinned by an earlier test, which is why this one can afford to assert so little.

HONEST BOUNDARY

Four edges, marked. **Behavior order is registration order:** the pipeline composes in the sequence behaviors are registered, an implicit contract that grows fragile as behaviors multiply — one exists today, so the fragility is theoretical, and Volume III's logging behavior is where the ordering question gets a real answer. **First failure only:** the behavior surfaces one validation error where a form-driven API wants them all; widening the Error type to carry a list is **Exercise 16.1**, and Volume II's problem-details mapping is its customer. **Reactions share the publisher's fate:** MediatR publishes notifications sequentially in-process, so a throwing reaction aborts its siblings — failure isolation between subscribers is outbox-and-retry territory, deferred with the rest of delivery. **And the dispatcher still has no caller in production:** the test plays the commit seam, exactly as Chapter 9 scheduled; the day Volume II's unit of work takes over the call is the day the after-commit contract stops being documentation and starts being code.

CHECKPOINT · CHAPTER 16

Before moving on, you should be able to answer these without looking back:

1. State the letter/act distinction, and explain the concurrency argument for why a validator restating the flagship would eventually lie.

2. What exactly did the pipeline buy that Listing 13.2's dispatcher could not pay for? Point at the line in Listing 16.4 that collects it.

3. Defend the reflective wart: why does it exist, where is it confined, and what would eliminating it cost?

4. The reaction handler sends a command instead of using its repository directly. Name two guards that choice keeps in force.

5. In Listing 16.5, why is the *absence* of NotFound the finding? What does it prove about ordering?

Every piece now exists and every piece is pinned — separately. What Part III has not yet shown is the pieces as one system under narrative load: a single operational story, from a sensor misbehaving to a case closed and a policy consulted, traced through every layer this book has built. That vertical slice is the next chapter's whole job, and Part III's closing argument.

Cross-Cutting, Open Generics, and One Look in the Mirror

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
> |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 35

The pipeline is armed and the bridge is built: every command and query now passes the validation ring, and Chapter 9's conversation runs as an executable test.

Cross-cutting concerns: the rules that belong to every door

Some requirements attach to no single use case because they attach to all of them: validate the message, log the passage, time the handler, retry on transient failure. Implemented naively, each becomes a stanza copy-pasted into every handler — seventeen contexts of drift waiting to happen. The pipeline's answer is structural: write the concern **once**, as a ring every request passes through, and the handlers stay pure rhythm. That is why Chapter 14's handlers contain no validation calls and never will: the concern is not theirs, and the architecture has a place for concerns that are nobody's in particular.

Open generics: one class, every shape

ValidationBehavior<TRequest, TResponse> is registered with its type parameters left **open** — *typeof(IPipelineBehavior<,>)*, blanks unfilled — and the container closes them per request: a CloseCaseCommand arriving manufactures a *ValidationBehavior<CloseCaseCommand, Result>* on the spot. One registration line therefore guards every message type that exists *and every one not yet written*: Chapter 17's brand-new AttachEvidenceCommand was validated the moment it compiled, by code that had never heard of it. When the book says the pipeline was "the actual purchase," this multiplying trick is the receipt.

Reflection, and this book's two uses of the mirror

Reflection is C# reading its own types at runtime — asking an object what class it is, what properties it has, invoking a method found by name. It is powerful and slow and opaque, so the book's policy is visible restraint: reflection appears exactly twice, both times quarantined and captioned — the statute test of Chapter 9 (the mirror pointed at every event, enforcing anatomy) and the Failure wart of Chapter 16 (the mirror summoning Result<T> for an unknown T). Notice both uses share a shape: reflection at the *edges*, auditing or adapting types — never in domain logic, where a rule you can only find with a mirror is a rule nobody can read.

Dying one ring out

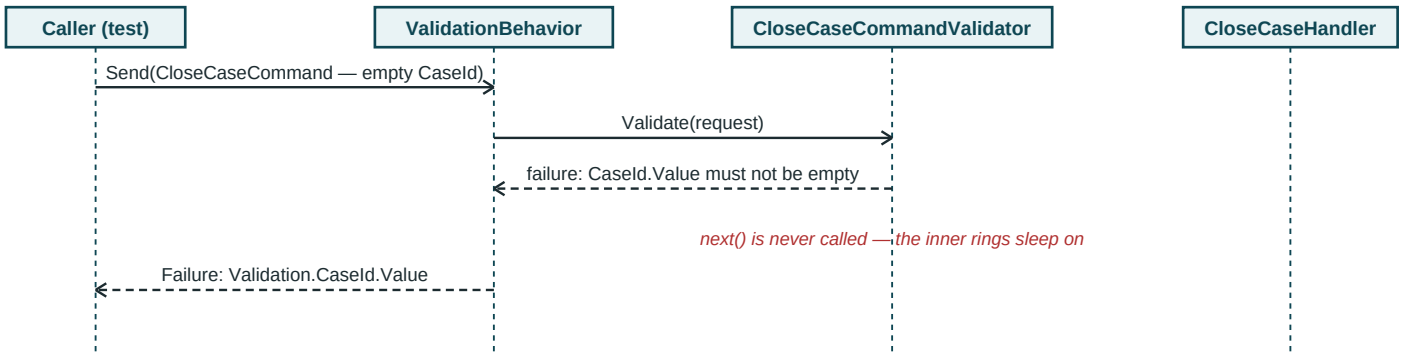


Figure D16.1 — Listing 16.5 as geometry: the malformed command never wakes the handler; ordering proven by which voice answers.

The conversation, wired at last

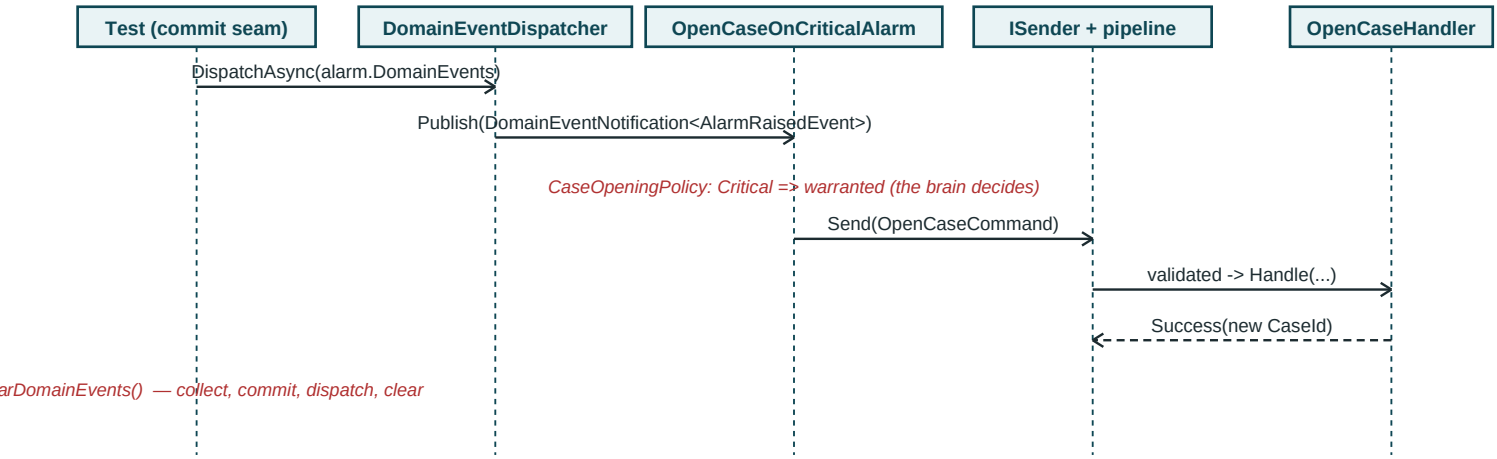


Figure D16.2 — Listing 16.6's spine: from announcement to a case in another context, through the full guarded write path.

Two figures, one shared moral about *voices*. In D16.1 the caller learns the outcome's origin from its error code alone — *Validation…* means the outer ring spoke, *…NotFound* would have meant the handler did — which is how a well-named system makes its own ordering observable without a debugger. In D16.2, count who holds opinions: exactly one note carries a decision (the policy's), and every other arrow is carriage. Reactions are clients; brains stay in the Domain; and the write path is the same guarded lane whether a human or an announcement is driving — which is the entire reason the reaction sends a command instead of touching a repository.

17 · Orchestration Case Study

One night, every layer

Every piece is pinned; nothing yet proves the pieces as a *system*. This chapter is that proof, and its method is narrative: the night of July 2nd, replayed as one executable story. A condenser instrument misbehaves in Unit 2 and the board lights up — twelve raises in eight minutes, a flood by any threshold. One of the raises is Critical, and two contexts away a case opens that no one asked for. An investigator works it: evidence attached, one entry struck as a transcription error, candidates ranked off the causal graph. The case closes on the top suspect. And the plant learns: the episode feeds a Q-table, a policy is drafted from the frozen result, evaluated, and activated for the next such night. Five acts, five contexts brushed, every one through the machinery of the last eleven chapters — under this chapter's one rule: **if the story needs a piece that does not exist, that is a finding, not a footnote.**

Finding one, before the curtain: a missing command

The rule bites immediately. Act III attaches evidence through the guarded write path, and no *AttachEvidenceCommand* exists — Chapter 14 built three commands and stopped. The repair is the point: Chapter 14 claimed the write side of seventeen contexts is "this shape, repeated," and here is the claim cashed — a complete production command, written in minutes because every decision was already made:

```
namespace Nexus.Application.RootCause.Commands;

public sealed record AttachEvidenceCommand(
    RootCauseCaseId CaseId, string Description, TagId Source) : ICommand;

public sealed class AttachEvidenceHandler(
    IRepository<RootCauseCase, RootCauseCaseId> cases,
    IUnitOfWork uow,
    IDateTimeProvider clock) : ICommandHandler<AttachEvidenceCommand>
{
    public async Task<Result> Handle(AttachEvidenceCommand cmd, CancellationToken ct)
    {
        var rcc = await cases.GetAsync(cmd.CaseId, ct);
        if (rcc is null)
            return Result.Failure(
                ApplicationErrors.NotFound("RootCauseCase", cmd.CaseId.Value));

        var attached = rcc.AttachEvidence(cmd.Description, cmd.Source, clock.UtcNow);
        if (attached.IsFailure)
            return attached;

        await uow.CommitAsync(ct);
        return Result.Success();
    }
}
```

Listing 17.1 — AttachEvidenceCommand.cs, complete. (Validator: two NotEmpty rules in Listing 16.1's exact shape; printed in Appendix B.)

Load, act, commit; two permitted ifs; nothing decided. Its sibling — *StrikeEvidenceCommand*, for Act III's correction — is deliberately *not* written here: it is **Exercise 17.1**, because a reader who cannot produce it from Listing 17.1 by substitution has a question this book should hear about. The scenario marks the spot where it would run.

Acts I–II: the flood, and the case nobody requested

One housekeeping diff rides along before the curtain: Chapter 12's fake clock gained a constructor, and *ValidateOnBuild* flagged the fixture's registration that same build — the wiring guard doing precisely its job. The ledger prints the correction here, where a story first depends on the clock:

```
        services.AddSingleton<IDateTimeProvider, FixedDateTimeProvider>();
        services.AddSingleton<IDateTimeProvider>(new FixedDateTimeProvider(
            new DateTime(2026, 7, 2, 0, 0, 0, DateTimeKind.Utc)));
```

Listing 17.2 — ApplicationFixture.cs, housekeeping: a fake with a constructor needs an instance registration.

```
public sealed class TheNightOfJuly2nd(ApplicationFixture fx)
    : IClassFixture<ApplicationFixture>
{
    private static readonly DateTime T0 = new(2026, 7, 2, 2, 14, 0, DateTimeKind.Utc);
    private static readonly UnitId Unit2 = UnitId.New();

    [Fact]
    public async Task The_whole_night_end_to_end()
    {
        // ---- ACT I: twelve raises in eight minutes; the judge convicts ----
        var reads = new InMemoryAlarmReadModel();
        reads.Seed(Unit2, [.. Enumerable.Range(0, 12)
                              .Select(i => T0.AddSeconds(-40 * i))]);

        var flood = await new GetFloodStatusHandler(reads,
                new FloodDetectionService(FloodThresholds.Default),
                new FixedDateTimeProvider(T0))
            .Handle(new GetFloodStatusQuery(Unit2), default);

        Assert.True(flood.Value.IsFlood);
        Assert.Equal(12, flood.Value.AlarmsInWindow);

        // ---- ACT II: one raise is Critical; RootCause reacts unbidden ----
        var dispatcher = fx.Provider.GetRequiredService<IDomainEventDispatcher>();

        var critical = AlarmEvent.Raise(Unit2, TagId.New(),
            AlarmSeverity.Critical, "Condenser vacuum low-low", T0);

        await dispatcher.DispatchAsync(critical.DomainEvents);
        critical.ClearDomainEvents();       // collect, commit, dispatch, clear

        var cases = (InMemoryRepository<RootCauseCase, RootCauseCaseId>)
            fx.Provider.GetRequiredService<IRepository<RootCauseCase, RootCauseCaseId>>();
        var rcc = cases.Single();
        Assert.Equal(critical.Id, rcc.TriggeredBy);
```

Listing 17.3 — TheNightOfJuly2nd.cs, Acts I–II: the judge convicts a flood; the conversation opens a case.

Act I runs outside the container on purpose — a handler *new*-ed with its fakes, Chapter 5's cheapest promise — while Act II runs through it, because the reaction chain is container wiring by nature. The story uses each door the way the architecture intends, which is itself part of what it demonstrates.

Acts III–IV: the investigation, and a flagged seam felt

```
        // ---- ACT III: the investigator works the case ----
        var sender = fx.Provider.GetRequiredService<ISender>();

        await sender.Send(new AttachEvidenceCommand(rcc.Id,
            "Hotwell level trend inverted 02:41", TagId.New()));
        await sender.Send(new AttachEvidenceCommand(rcc.Id,
            "Air ejector flow spike 02:43", TagId.New()));

        //   (a third entry is attached and struck as a transcription error;
        //    StrikeEvidenceCommand is Exercise 17.1 -- the domain door already
        //    proved the refusal logic in Listing 11.7)

        // the plant's failure atlas, seeded (factory printed in Appendix B):
        var graph = CausalGraph.Chart(Unit2);
        var vacuum = graph.AddNode("Condenser vacuum loss", prior: 0.30,
                                   parent: null, signal: rcc.Evidence[0].Source);
        graph.AddNode("Sensor drift",  prior: 0.45, parent: null, signal: null);
        graph.AddNode("Tube fouling",  prior: 0.60, parent: vacuum, signal: null);

        var ranked = new CausalRankingService()
            .Rank(graph.CandidatesFor(rcc.Evidence)).Value;

        Assert.Equal("Condenser vacuum loss", ranked[0].Cause);
        //   evidence outvoted a lazier prior; depth taxed the deeper node

        // ---- ACT IV: close on the top suspect ----
        //   the seam Chapter 15 flagged, now felt: RankedCause carries the
        //   display name, so the story keeps its own name->id map (Ex. 17.2):
        var closed = await sender.Send(new CloseCaseCommand(rcc.Id, vacuum));

        Assert.True(closed.IsSuccess);
        Assert.Equal(CaseStatus.Closed, rcc.Status);
        var testimony = Assert.IsType<CaseClosedEvent>(rcc.DomainEvents[^1]);
        Assert.Equal(2, testimony.EvidenceCount);
```

Listing 17.4 — TheNightOfJuly2nd.cs, Acts III–IV: evidence, ranking, closure.

Two findings surface exactly where earlier chapters said they would. The ranking assert is Chapter 11's scoring under real narrative load: *Sensor drift* arrived with the fattest prior and lost, because two pieces of evidence testified for the vacuum node and depth decay taxed *Tube fouling* for its distance — priors propose, evidence disposes. And Act IV steps directly onto the seam Chapter 15's boundary box flagged: the ranked answer names the cause but cannot identify it, so the story holds the *CauseNodeId* it happened to keep from seeding — a workaround wearing its exercise number. Widening *RankedCause* to carry the ID is **Exercise 17.2**, and Volume II's API — whose clients keep no seeding variables — is the customer that will insist.

Act V: the plant learns

```
        // ---- ACT V: the episode feeds the plant's judgement ----
        var q = QTable.StartLearning(Unit2);
        var lesson = new StateAction("CondenserVacuumLowLow", "ReduceLoadEarly");
        var alpha = LearningRate.Create(0.5).Value;

        q.Learn(lesson, target: 1.0, alpha);      // three passes over the episode
        q.Learn(lesson, target: 1.0, alpha);
        q.Learn(lesson, target: 1.0, alpha);
        Assert.Equal(0.875, q.ValueOf(lesson));   // converging, visibly

        var policy = Policy.Draft(Unit2, q.Id);   // frozen custody: ID only
        Assert.True(policy.RecordEvaluation(score: 0.87).IsSuccess);
        Assert.True(policy.Activate(T0.AddHours(6)).IsSuccess);

        var activated = Assert.IsType<PolicyActivatedEvent>(policy.DomainEvents[^1]);
        Assert.Equal(Unit2, activated.UnitId);
        //   next July 2nd, the Optimization screen has an answer ready
    }
}

$ dotnet test tests/Nexus.Application.Tests
Passed! - Failed: 0, Passed: 11   (Domain suite: 26, unchanged)

$ dotnet test    # the whole solution, for the record
Passed! - Failed: 0, Passed: 37, Skipped: 0, Duration: 412 ms
```

Listing 17.5 — TheNightOfJuly2nd.cs, Act V and curtain: the episode becomes a policy.

Count what one test traversed: a domain service judged a window; a domain event crossed a context; a policy object consulted; a reaction sent a command; a pipeline validated it; four handlers loaded, acted, and committed; a child entity collection grew behind its door; a graph manufactured candidates; a ranking held deterministic; an aggregate refused nothing because nothing illegal was asked — and two aggregates in a fifth context recorded the plant getting smarter. Every layer of the book, one story wide. The test asserts generously and still trusts more than it checks — it can afford to, because all thirty-six tests beneath it each pinned one piece alone. That division — stories broad, pieces deep — is Part IV's entire testing philosophy, previewed.

HONEST BOUNDARY

What the night does not prove, stated plainly. **The story is choreographed:** the test seeds its own flood, raises its own alarm, and plays the commit seam by hand; nothing arrived over a wire, survived a process restart, or raced a concurrent investigator — the narrative exercises the layers' *composition*, not the runtime's hazards, which are Volumes II and III by design. **Acts II and V do not touch:** the flood verdict informs no one and the policy activation notifies a mailbox the test reads directly; wiring verdicts to reactions and activations to the DigitalTwin is deferred until those consumers exist — Chapter 9's waiting rule, still in force. **And the RL remains custody, not cognition:** three hand-fed passes toward a hand-picked target demonstrate the artifact chain; where targets truly come from stays with *From Trial to Policy*, as Chapter 11 priced.

CHECKPOINT · CHAPTER 17

Before moving on, you should be able to answer these without looking back:

1. State the chapter's one rule, and name both findings it produced — one repaired on the page, one felt and deferred with a number.

2. Why does Act I run outside the container while Act II runs through it? Answer per the architecture, not convenience.

3. Reconstruct why *Sensor drift* lost despite the fattest prior — two mechanisms, one sentence each.

4. The scenario asserts generously and trusts more than it checks. Why is that affordable, and what testing philosophy does it preview?

5. Name two things the choreographed night deliberately cannot prove, and the volume that owns each.

Part III, audited

Part III promised an Application layer that orchestrates without deciding: delivered — five production handlers whose only ifs are absence checks and forwarded verdicts, a litmus any reader can re-run against any listing. It promised the seam: delivered as three ports plus three read models, consumer-owned, with the after-commit contract written into their documentation and a named fake behind each — and the honest count is that *every adapter is still a fake*, which is not a shortfall but the volume's thesis: the core provable against thin air. It promised CQRS as two paths: delivered, with both dissents from purity argued in the open and the exotic variants declined by name. It promised the pipeline would justify MediatR's tenancy: collected, in one open-generic guard at every door and one bridge that let Chapter 9's conversation finally run. The ledger carries forward: three exercises open (16.1, 17.1, 17.2), the dispatcher awaiting its production caller, authorization and idempotency filed with Volume II, and one reflective wart under quarantine. Thirty-seven tests, zero packages in the Domain, and a plant that can be asked, told, and taught — by a test suite. What no one has yet done is turn on the lights the way an auditor would: Part IV proves the core is not merely working but *watched* — the testing discipline itself, made systematic.

The next chapter begins that reckoning where the value is highest: unit testing the Domain — the anatomy of the twenty-six tests already green, the patterns that made them cheap, and the discipline that keeps a growing plant that way.

The Night, on One Page

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
> |- tests/Nexus.Application.Tests
```

Tests green: 37

Part III closes: five handlers, the pipeline, the bridge, and one scenario test that walks the whole plant. Thirty-seven green; every adapter still a fake, on purpose.

Scenario tests, and the pyramid they sit on

A scenario test like TheNightOfJuly2nd is deliberately different testing: broad, narrative, and — the phrase the chapter used — trusting more than it checks. That trust is not laxity; it is **division of labor**, and the classic picture for it is the test pyramid below. The wide base holds many small, fast, merciless tests, each pinning one rule in isolation; the narrow top holds few broad stories proving the pinned pieces compose. Invert the pyramid — many stories, few piece tests — and every failure becomes an investigation ("something, somewhere, in this forty-line narrative") instead of a diagnosis. Volume I's pyramid stops where the fakes stop; the gray layers are the floors the next volumes pour on top.

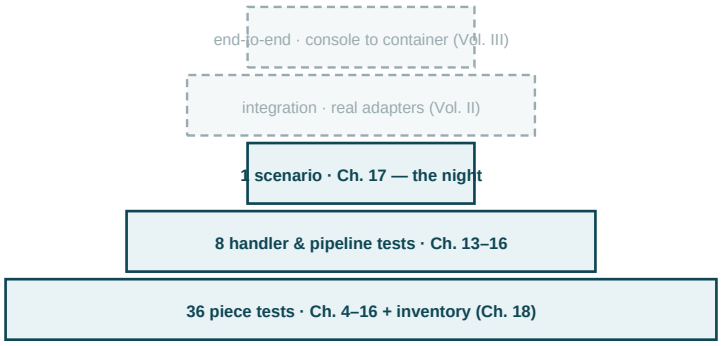


Figure D17.1 — Volume I's test pyramid, with the floors the trilogy still owes drawn in gray above it.

One reading habit for any scenario test, this book's included: before following the code, list what it does *not* assert and ask where that claim is pinned instead. Every act of the night trusts a dozen facts — the flagship refuses, the pipeline validates, the tie-break is stable — and each has an address lower in the pyramid. A trust without an address is the only bug a scenario test can truly have.

Five acts, seven lifelines, one figure

The capstone figure: Listing 17.3–5 compressed into one sequence, act boundaries as red notes. Every arrow below has appeared in an earlier deep-dive figure with more room; this page is where they meet.

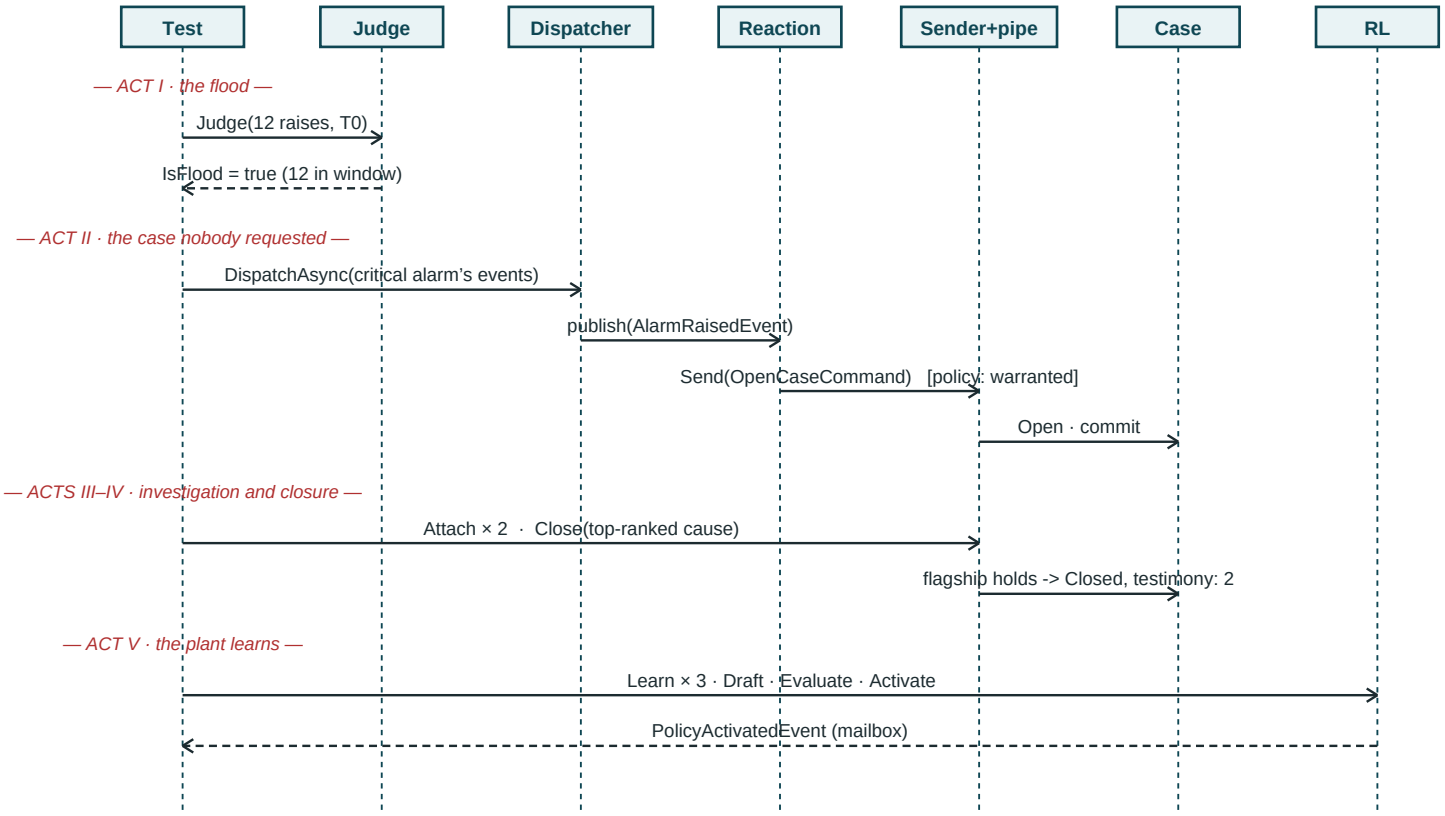


Figure D17.2 — The Night of July 2nd as one UML sequence. Acts I–V, left to right and top to bottom, exactly as the test runs them.

Two features of the figure repay a slow look. The Test lifeline touches every act — it is the choreographer the honest-boundary box confessed — while the domain lifelines never touch *each other* directly: Case and RL share a page and not one arrow, and the alarm reaches the Case only through three intermediaries whose entire job is carrying. That is seventeen chapters of architecture visible as white space. And the one bracketed guard, on the Reaction's arrow, marks the single decision the whole middle of the diagram makes — everything else is rhythm. When Volume II replaces the Test lifeline with a real API and a real commit seam, this figure is the one that gains lifelines without moving an arrow.

PART IV

Proving the Core

Thirty-seven tests are green, and green is not the same as watched. Two chapters turn the discipline that grew test by test into a system: an inventory that knows every invariant by name and can say which crimes have never been committed on purpose, builders that assemble legal states through the domain's own doors, and the handler-testing rhythm that makes the Application layer as cheap to pin as the Domain. The volume ends the way it promised to be judged: audited.

CHAPTERS 18–19 · THE INVARIANT INVENTORY · BUILDERS · HANDLER TESTS

18 · Unit Testing the Domain

The purity law, and its dividend

A Domain test may touch nothing but the Domain and the assertion library — no container, no fakes, no time machinery, no *async*, no setup files. This is not asceticism; it is a detector. Every dependency a test needs is a dependency the code under test has, so the moment a Domain test reaches for a mock, **the Domain has sprung a leak** and the test just found it — cheaper than any architecture review. The dividend compounds: twenty-six tests run in a tenth of a second, fail with the offending line in the name, and will run unchanged in Volume II and III against the identical classes, because nothing they touch is going to change. The inner ring's independence, which Part I argued and Part II built, is here converted into its cash value: **the cheapest tests in the solution guard its most important rules.**

Naming is specification

The suite's second discipline has been hiding in plain sight since Listing 4.6: every test name is a sentence in the ubiquitous language, present tense, no “should”, no method names. Run the discovery command and the payoff renders — the Domain suite *is* the specification, readable by the domain experts who wrote the rules in *From Domain to Twin* without a line of C# between them and the claims:

```
$ dotnet test tests/Nexus.Domain.Tests --list-tests

  A_case_cannot_close_without_evidence
  A_case_may_start_empty_but_may_not_be_emptied
  A_closed_case_is_a_sealed_record
  A_raised_alarm_acknowledges_exactly_once
  A_shelved_alarm_re_alarms_and_demands_fresh_eyes
  A_policy_cannot_activate_without_evaluation
  A_reading_that_is_not_a_number_is_not_a_reading
  Bars_refuse_comparison_with_degrees
  Equal_scores_rank_in_the_same_order_every_run
  Every_domain_event_is_a_sealed_immutable_past_tense_record
  ...
```

Listing 18.1 — dotnet test --list-tests (excerpt): the specification the suite has been writing all along.

A test that cannot be named as such a sentence is testing implementation, not rule — the naming convention doubles as a linter for the tests themselves.

Watched-to-fail, formalized: the pair rule and the two asserts

The habit the book has practiced since Chapter 4 now becomes method. **The pair rule:** every named refusal in the codebase owes the suite a test that commits its exact crime — an invariant demonstrated only holding is a claim, not a guarantee. **The two-assert discipline:** the violation test asserts the refusal *by its published name* (the code is the contract) and asserts *nothing half-applied* — status unchanged, mailbox silent, collections intact — because a refusal that mutates on the way out is the 04:30 corruption wearing a seatbelt. Method invites audit: if every refusal owes a test, the debts can be tabulated. Here is the inventory, and it is not decorative:

A case cannot close without evidence	`RootCause.CaseWithoutEvidence`	**8.4**
A case may not be made empty	`RootCause.CannotEmptyCase`	**11.7**
A closed case is sealed — attach path	`RootCause.CaseAlreadyClosed`	**8.4**
A closed case is sealed — close path	`RootCause.CaseAlreadyClosed`	—
Striking targets existing, unstruck evidence	`RootCause.NoSuchEvidence`	—
Only a raised alarm acknowledges	`AlarmManagement.NotRaised`	**6.5**
Only an acknowledged alarm clears	`AlarmManagement.NotAcknowledged`	—
Only an acknowledged alarm shelves	`AlarmManagement.OnlyAcknowledgedShelves`	—
A shelf must end in the future	`AlarmManagement.ShelfMustEnd`	—
Re-alarm needs a shelved or cleared alarm	`AlarmManagement.NothingToReAlarm`	—
A policy cannot activate unevaluated	`ReinforcementLearning.NotEvaluated`	**11.7**
Only a draft policy evaluates	`ReinforcementLearning.NotDraft`	—
Readings are finite; units must match	`Instrumentation.NotFinite / UnitMismatch`	**7.6**
Guarded factories refuse bad input	`InvalidLearningRate / Thresholds / Candidate`	—

Table 18.1 — The invariant inventory: rule, published refusal, and the listing where it was watched to fail. Red dashes are debts.

Ten refusals have never been watched. Most entered during Chapter 11's walkthroughs, where the pace favored the new over the exhaustive — precisely the drift the inventory exists to catch. The next page pays.

Paying the inventory

Ten crimes, ten tests, each in the two-assert discipline. Three representative repairs are printed — the sealed record refusing to close twice, the unacknowledged clear, the non-draft evaluation — and the remaining seven follow the identical shape in Appendix B, the elision marked as the rules allow:

```
public sealed class InventoryRepairs
{
    private static readonly DateTime T0 = new(2026, 7, 2, 8, 0, 0, DateTimeKind.Utc);

    [Fact]
    public void A_closed_case_refuses_to_close_again()
    {
        var rcc = new CaseBuilder().Closed().Build();          // Listing 18.3
        var again = rcc.Close(new CauseNodeId(Guid.NewGuid()), T0.AddHours(2));

        Assert.Equal("RootCause.CaseAlreadyClosed", again.Error.Code);
        Assert.Single(rcc.DomainEvents.OfType<CaseClosedEvent>());   // one closing, ever
    }

    [Fact]
    public void An_unacknowledged_alarm_refuses_to_clear()
    {
        var alarm = AlarmEvent.Raise(UnitId.New(), TagId.New(),
            AlarmSeverity.Warning, "drift", T0);

        var cleared = alarm.Clear(T0.AddMinutes(1));

        Assert.Equal("AlarmManagement.NotAcknowledged", cleared.Error.Code);
        Assert.Equal(AlarmStatus.Raised, alarm.Status);        // nothing half-applied
    }

    [Fact]
    public void An_evaluated_policy_refuses_a_second_evaluation()
    {
        var p = Policy.Draft(UnitId.New(), QTableId.New());
        p.RecordEvaluation(0.90);

        var again = p.RecordEvaluation(0.10);                  // the grade-shopper

        Assert.Equal("ReinforcementLearning.NotDraft", again.Error.Code);
        Assert.Equal(0.90, p.EvaluationScore);                 // the first grade stands
    }
}

$ dotnet test tests/Nexus.Domain.Tests
Passed! - Failed: 0, Passed: 36, Skipped: 0, Duration: 118 ms
```

Listing 18.2 — tests/Nexus.Domain.Tests/InventoryRepairs.cs (three of ten; seven more in Appendix B), and the run.

Note what the third test's second assert quietly protects: without it, a refusal that overwrote the score on the way out would pass the name check and still let a policy shop for grades. The two-assert discipline exists for exactly the failures the first assert cannot see. Domain suite: thirty-six; whole solution: forty-seven.

Builders: legal states, assembled through the doors

Listing 18.2 used a builder before defining it — a closed case takes four lines of arrange (open, attach, close, and a node id), and the inventory repairs needed one three times. The rule of three triggers the test-data builder, with one law that makes or breaks it: **a builder reaches every state through the domain's own doors.** No reflection, no private setters pried open, no deserialization tricks — because a builder that cheats can assemble states the plant cannot reach, and a test against an unreachable state tests nothing, green forever, guarding nobody.

```
namespace Nexus.Domain.Tests.Builders;

public sealed class CaseBuilder
{
    private static readonly DateTime T0 = new(2026, 7, 2, 3, 0, 0, DateTimeKind.Utc);

    private int _evidence;
    private bool _closed;

    public CaseBuilder WithEvidence(int count = 1)
    {
        _evidence = count;
        return this;
    }

    public CaseBuilder Closed()
    {
        _closed = true;
        _evidence = Math.Max(1, _evidence);   // a legal close needs evidence: the
        return this;                          // builder knows the flagship too
    }

    public RootCauseCase Build()
    {
        var rcc = RootCauseCase.Open(UnitId.New(), AlarmEventId.New(), T0);

        for (var i = 0; i < _evidence; i++)
            rcc.AttachEvidence($"evidence {i}", TagId.New(), T0.AddMinutes(i));

        if (_closed)
            rcc.Close(new CauseNodeId(Guid.NewGuid()), T0.AddHours(1));

        return rcc;
    }
}
```

Listing 18.3 — tests/Nexus.Domain.Tests/Builders/CaseBuilder.cs, complete.

The comment inside *Closed()* is the design in one line: the builder encodes the *preconditions* of legal states, so a test asking for a closed case gets one that could actually exist. And because every step is a public door, the builder is itself under test by transitivity — if Chapter 8's rules ever change, the builder breaks loudly at the same commit, instead of silently manufacturing yesterday's plant.

HONEST BOUNDARY

Three rungs above this chapter, named and declined for now. **No coverage percentage is measured, on purpose:** the inventory audits *rules*, not lines — a codebase can hit 95% coverage with every invariant unwatched, and the number's comfort is exactly its danger; line coverage arrives in Volume III as a smoke detector, never as the goal. **No property-based testing:** the ranking's determinism and the Q-table's convergence are natural property-test candidates (any input list, same output order; any target sequence, monotone approach), and a FsCheck-style suite is the honest next rung — **Exercise 18.1** opens it. **No mutation testing:** the sharpest known audit of a suite's teeth — mutate the code, expect a failure — is runtime-heavy machinery that belongs with Volume III's quality gates, where it is scheduled.

CHECKPOINT · CHAPTER 18

Before moving on, you should be able to answer these without looking back:

1. State the purity law and explain how it works as a leak detector rather than a style rule.

2. What does the naming convention lint, and what would a test that cannot be named as a rule-sentence be testing instead?

3. Recite the pair rule and the two-assert discipline, and give the grade-shopping example of what the second assert alone catches.

4. Why did most inventory debts date from Chapter 11, and what does that say about when audits earn their keep?

5. State the builder's one law, and explain “under test by transitivity”.

The Domain is watched. One suite remains: the handlers — where the pieces under test have dependencies by design, and the discipline shifts from purity to *honest fakery*: what a fake may simulate, what it must refuse to, and how the Application suite stays cheap while proving orchestration. The final chapter of the volume.

Anatomy of a Test, Anatomy of an Audit

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
> |- tests/Nexus.Domain.Tests
  |- tests/Nexus.Application.Tests
```

Tests green: 47

The Domain suite grows to thirty-six: the inventory found ten unwatched refusals and the pair rule collected them. CaseBuilder now assembles legal states through the doors.

AAA: the grammar every test in this book speaks

Every test since Listing 4.6 has had the same three-part grammar, worth naming once: **Arrange** — build the world the rule needs (an open case, one clue attached); **Act** — the single operation under judgement (Close, twice); **Assert** — the checkable claims about what happened. The blank lines in the listings are not styling; they are the grammar's punctuation, and a test whose three parts cannot be found by whitespace alone is usually testing two things at once. The xUnit attributes map onto the grammar's outside: *[Fact]* marks one story with fixed characters; *[Theory]* with *[InlineData]* marks one story told several times with the cast swapped — Chapter 9's three-severity policy test is the book's canonical theory, one grammar, three readings.

Why "coverage" and "watched" are different numbers

Line coverage answers "did execution pass through here?"; the inventory answers "has this *rule* been made to refuse, on purpose, and did we check the refusal's name and its cleanliness?" A test that merely *reaches* the flagship's if-statement on the happy path lights the coverage line green while proving nothing about the refusal — which is how a suite can score 95% and guard 0% of what matters. The chapter's refusal to measure coverage is therefore not austerity but unit hygiene: **count in rules, because rules are what the plant runs on.** The two figures overleaf draw the counting machine.

The pair rule, drawn

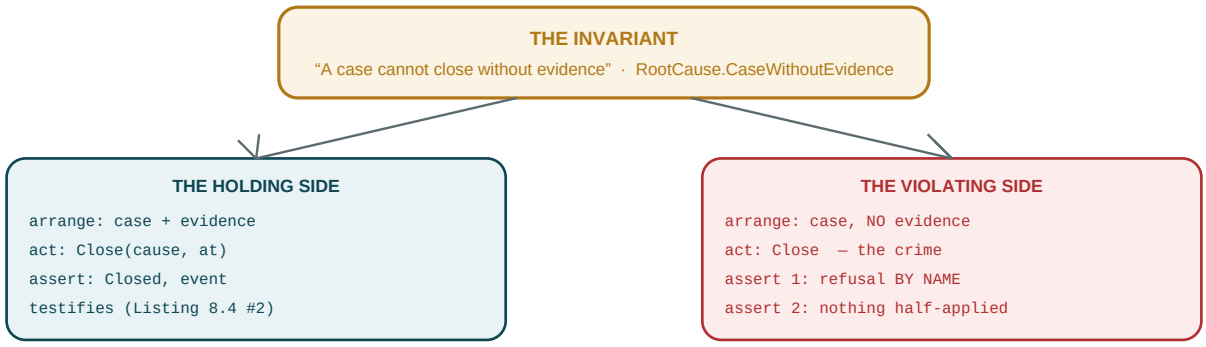


Figure D18.1 — One rule, two owed tests. The violating side carries two asserts because a refusal that mutates is corruption in a seatbelt.

The audit, as a loop that never closes

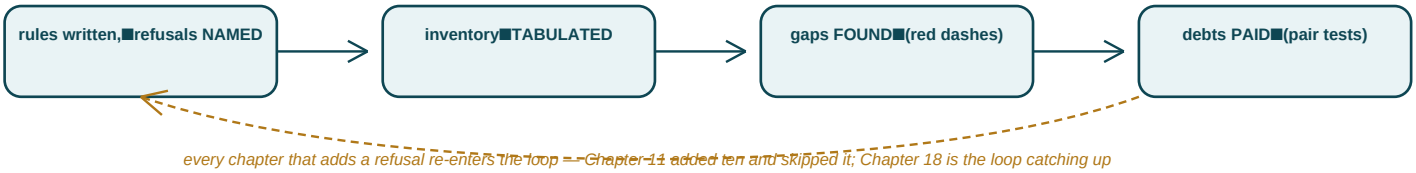


Figure D18.2 — The inventory as process, not table: Table 18.1 is one photograph of a loop the codebase must keep running.

The loop is the transferable asset. Any codebase with named refusals can run it — grep the error catalog, tabulate against the suite, pay the dashes — and the cadence matters more than the tooling: audits catch drift precisely because drift accumulates *between* them, which is why the deepest debt pooled in the fastest-moving chapter. Schedule the loop where speed lives.

19 · Testing Application Handlers

Honest fakery: the fake is the contract, executable

Chapter 18's purity law cannot apply here — handlers have dependencies by design — so the discipline shifts from abstinence to honesty. **A fake may simulate exactly what its port's contract promises, and nothing more.** The in-memory repository returns what was added because *GetAsync*'s documentation says so; the unit of work counts commits because counting is observation, not behavior. What a fake must refuse is generosity: a fake repository that validates, a fake unit of work that auto-commits, a fake clock that advances helpfully — each is smarter than its contract, and **a fake smarter than its contract tests a system that does not exist.** Green against it proves nothing about the system that does.

The decision tree, in one sentence

Two ways to arrange a handler test have appeared since Chapter 13, and the rule that chooses between them is short enough to memorize: **new the handler when testing the handler; use the container when testing the wiring.** The three-path canon of Chapter 14 — success with exactly one commit, domain refusal verbatim with zero commits, absence in the Application's voice — is hand-*new*-ed territory: the unit under test is one class's orchestration, and the container would only blur whose behavior failed. The fixture path exists for what only the container exhibits: pipeline ordering (Listing 16.5), reaction chains (Listing 16.6), resolution itself (Listing 13.4). A fixture test that could have been a hand-newed test is paying container rent for a room it never enters.

What handlers owe when the world fails

One question the three paths never asked: what does a handler do when a *port* fails — not a refusal, a real infrastructure exception mid-commit? The doctrine is Chapter 4's two channels, applied one last time: an infrastructure failure is not a business outcome, so it travels the exception channel, and **the handler does not catch it.** A try/catch in a handler is a smell with two readings — either it is converting infrastructure pain into a fake business refusal (lying to the caller), or it is attempting resilience (retry, fallback), which is pipeline and host machinery owned by Volumes II and III. The handler's whole duty on the worst day is to fail loudly and touch nothing else. The next page pins that duty — and discovers a scar worth reading twice.

A fake built to fail, and the scar it exposes

```
public sealed class ThrowingUnitOfWork : IUnitOfWork
{
    public Task CommitAsync(CancellationToken ct = default) =>
        throw new InvalidOperationException("simulated infrastructure failure");
}

public sealed class HandlerFailureTests
{
    private static readonly DateTime T0 = new(2026, 7, 2, 9, 0, 0, DateTimeKind.Utc);

    [Fact]
    public async Task An_infrastructure_failure_is_not_the_handlers_to_catch()
    {
        var cases = new InMemoryRepository<RootCauseCase, RootCauseCaseId>();
        var rcc = RootCauseCase.Open(UnitId.New(), AlarmEventId.New(), T0);
        rcc.AttachEvidence("clue", TagId.New(), T0);
        await cases.AddAsync(rcc);

        var handler = new CloseCaseHandler(cases, new ThrowingUnitOfWork(),
                                           new FixedDateTimeProvider(T0));

        await Assert.ThrowsAsync<InvalidOperationException>(() =>
            handler.Handle(new CloseCaseCommand(rcc.Id,
                new CauseNodeId(Guid.NewGuid())), default));

        // the scar, asserted on purpose -- read the paragraph below:
        Assert.Equal(CaseStatus.Closed, rcc.Status);
    }
}
```

Listing 19.1 — tests/Nexus.Application.Tests/Fakes/ThrowingUnitOfWork.cs and its test.

The last assert looks like a bug being celebrated, and it is the most honest line in the chapter. The act succeeded *in memory* before the commit died: the loaded instance is Closed, its mailbox holds a *CaseClosedEvent*, and no store anywhere agrees. Volume II's transaction will keep the *database* consistent under exactly this failure — rollback is its job — but no transaction reaches into the heap to un-mutate an object, and that is not a flaw to fix but a law to know: **a loaded aggregate never survives a failed commit.** The operation is dead, the instance is discarded, the retry reloads. The rule is bequeathed to Volume II by name, where an EF Core change tracker will make ignoring it genuinely expensive — and this fake, sixteen lines of deliberate failure, is what surfaced it two volumes early. That is honest fakery paying rent.

Contract tests: the law both the fake and the adapter must sign

If the fake's virtue is fidelity to the port's contract, the contract deserves to be executable — once, abstractly, against *whoever claims to implement the port*. The pattern is an abstract test class whose subclasses supply only the implementation; Volume I instantiates it with the fake, and Volume II's opening act is a second three-line subclass with the real adapter, inheriting every law verbatim:

```
public abstract class RepositoryContract<TRoot, TId>
    where TRoot : Entity<TId>, IAggregateRoot  where TId : notnull
{
    protected abstract IRepository<TRoot, TId> NewRepository();
    protected abstract TRoot NewRoot();

    [Fact]
    public async Task What_is_added_is_what_is_returned()
    {
        var repo = NewRepository();
        var root = NewRoot();

        await repo.AddAsync(root);

        Assert.Equal(root, await repo.GetAsync(root.Id));
        // Equal, not Same: a law an adapter that rehydrates can still sign
    }

    [Fact]
    public async Task A_missing_id_returns_null_not_an_exception()
    {
        Assert.Null(await NewRepository().GetAsync(NewRoot().Id));
    }

    [Fact]
    public async Task What_is_removed_is_forgotten()
    {
        var repo = NewRepository();
        var root = NewRoot();
        await repo.AddAsync(root);

        repo.Remove(root);

        Assert.Null(await repo.GetAsync(root.Id));
    }
}
```

Listing 19.2 — tests/Nexus.Application.Tests/Contracts/RepositoryContract.cs, complete.

The comment on the first assert retires a debt quietly incurred in Chapter 12: *PortContractTests* asserted *Assert.Same*, which only a fake can pass — fake-only knowledge dressed as a contract. It is deliberately *not* ported here, because **a contract must be one every implementor can sign**, and Chapter 4's equality — type and Id, rehydration-proof — is exactly the signature an adapter can honor. The old test class is deleted; its one lesson survives as this comment.

The signature, and the census

```
public sealed class InMemoryRepositoryContract
    : RepositoryContract<Policy, PolicyId>
{
    protected override IRepository<Policy, PolicyId> NewRepository() =>
        new InMemoryRepository<Policy, PolicyId>();

    protected override Policy NewRoot() =>
        Policy.Draft(UnitId.New(), QTableId.New());
}

// Volume II, chapter one, verbatim by design:
//   public sealed class EfRepositoryContract : RepositoryContract<Policy, PolicyId>
//   { ...two overrides, one real adapter, the same three laws... }

$ dotnet test
Passed! - Failed: 0, Passed: 50, Skipped: 0, Duration: 583 ms
```

Listing 19.3 — The fake signs the contract; Volume II's first diff is written in advance. And the final run.

Nexus.Domain.Tests	36	~0.12 s	Nothing but the SDK and the assertion library
Nexus.Application.Tests	14	~0.46 s	Named fakes only; container for wiring tests alone
Total — the whole solution	50	~0.6 s	Zero databases, zero HTTP, zero configuration files

Table 19.1 — The census at the volume's close: fifty tests, six hundred milliseconds, and not one piece of infrastructure.

Part IV, audited

Part IV promised to convert discipline into system: delivered — the inventory found ten unwatched refusals and the pair rule collected them; the builder's one law made legal states cheap without ever picking a lock; honest fakery found its edge cases, including one (the *Assert.Same* retirement) where this book's own earlier test had quietly over-promised. It promised the suites would be Volume II's inheritance, not just Volume I's proof: delivered as an executable contract with the adapter's subclass already drafted in a comment, and one law — the failed-commit scar — bequeathed by name. The forward ledger closes where the volume does: exercises 11.1, 11.2, 12.1, 16.1, 17.1, 17.2, and 18.1 open for the reader; mutation testing, coverage-as-smoke-detector, and the unhandled-exception policy filed with Volume III; everything else the trilogy owes is owed by Volume II, in writing, at the seam.

HONEST BOUNDARY

The last boundary box of the body, and it guards the biggest claim. **Fifty green tests prove the core's logic, not the system's behavior:** nothing here has proven a transaction actually rolls back, a concurrent write actually conflicts, a cancellation token actually cancels, or an event actually survives a crash — the contract of Listing 19.2 is deliberately silent on isolation and durability, because laws the fake cannot honestly sign would make the fake a liar. Volume II extends the contract with the laws only real infrastructure can keep; Volume III puts the whole under load. **And the census's speed is a property of scope, not of skill:** six hundred milliseconds is what testing looks like *inside* the persistence boundary; readers should expect Volume II's suite to be slower by design and to buy different guarantees for the price. Fast tests and true tests are the same thing only when the truth in question is logic — which, in this volume, it was, on purpose.

CHECKPOINT · CHAPTER 19

Before closing the book, you should be able to answer these without looking back:

1. State the fake's law, and give two examples of a fake being “generous” and what each would falsely prove.

2. Recite the decision tree's sentence, and classify Listings 14.4, 16.5, and 19.1 under it.

3. Defend the scar assert: what is true of the heap after a failed commit, what will Volume II fix, and what will it pointedly not fix?

4. Why was *Assert.Same* unfit for the contract, and what design decision from Chapter 4 supplied the fit replacement?

5. Name three things fifty green tests have not proven, and the volume that owns each proof.

The body of the book ends here: nineteen chapters, five projects, fifty tests, and a domain that has never once touched the world it models. What remains is the accounting — the epilogue that draws Volume I's honest boundary in full and hands the trilogy's ledger to *From Core to Contract*, and the appendices that make every elision in these pages whole.

Doubles, Substitutes, and the Scar Made Visible

WHERE WE ARE IN THE SOLUTION

```
  Nexus.sln
  |- src/Nexus.SharedKernel
  |- src/Nexus.Domain
  |    |- <17 context folders>
  |- src/Nexus.Application
  |    |- Abstractions (ports)
  |- tests/Nexus.Domain.Tests
> |- tests/Nexus.Application.Tests
```

Tests green: 50

The volume's final count: fifty green in six hundred milliseconds, the contract class awaiting its Volume-II signatory, and one law about failed commits bequeathed forward.

The test-double taxonomy, and where this book's fakes sit

The literature names five kinds of stand-in, and knowing them sharpens reviews even though this book deliberately uses one word for its own. A **dummy** fills a parameter and is never used. A **stub** returns canned answers. A **fake** has a real, simplified implementation — the dictionary-backed repository is the textbook case. A **spy** records what happened to it for later inspection — which is precisely what *InMemoryUnitOfWork.Commits* is: a fake wearing a spy's notebook. A **mock** carries expectations and fails the test itself when they are unmet — the one kind this book never uses, because mocks verify *conversations* ("was GetAsync called with X?") and the book's doctrine verifies *outcomes* ("is the case closed, and committed once?"). Outcome tests survive refactoring that conversation tests punish, which is the quiet reason no mocking library appears in Appendix C.

Substitutability: the principle under the whole seam

Chapter 19's contract class is an old principle made executable: anywhere a contract is expected, any signatory must be usable *without the client knowing or caring which one arrived* — substitutability, the L in the SOLID acronym readers will meet elsewhere as the Liskov Substitution Principle. The Assert.Same retirement was exactly an LSP repair: the old test demanded a behavior (reference identity) that only one signatory could exhibit, so it was not a contract but a portrait of the fake. The rewritten laws — Equal, not Same — are ones every honest implementation can keep, which is what makes the gray subclass of Figure D12.2 writable in three lines two volumes early.

One set of laws, two signatories, zero edits

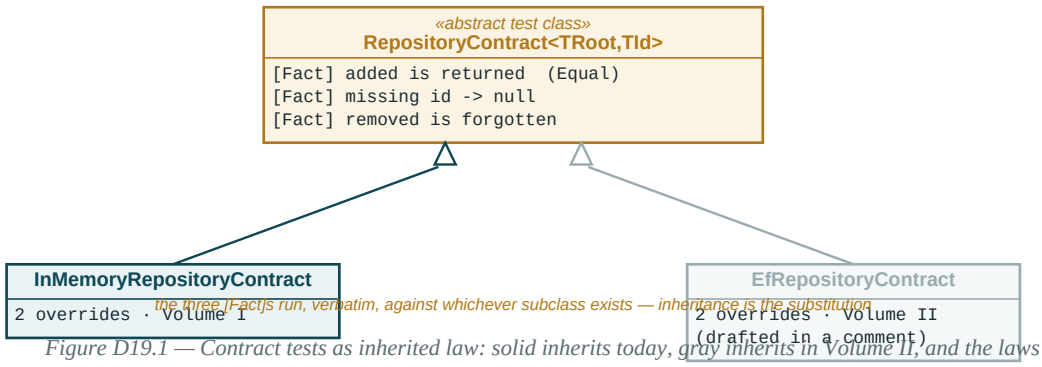


Figure D19.1 — Contract tests as inherited law: solid inherits today, gray inherits in Volume II, and the laws never notice.

The scar, as two columns that disagree

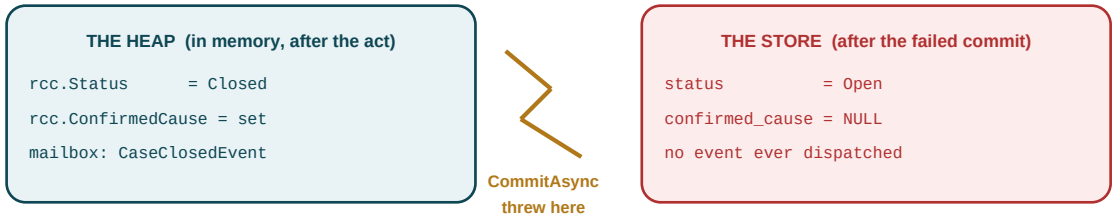


Figure D19.2 — Listing 19.1's scar: after a failed commit the two columns tell different stories, and no rollback edits the left one.

The law falls straight out of the picture: since nothing reconciles the columns, the only safe move is to **discard the left one** — the operation is dead, the instance is garbage, a retry reloads from the right. Volume II's transaction will guarantee the right column stays clean under exactly this lightning bolt; nothing will ever launder the left. Nineteen deep dives end on a fitting note: the diagram is not decoration over the rule — for divergences like this one, the diagram is the fastest honest way to *see* why the rule must exist. That has been the wager of every figure in this book.

Epilogue · The Honest Boundary of Volume I

What exists now that did not exist on page one

On page one, a developer who had read eight NEXUS-1 books could not press F5. Now there is a *Nexus.sln*: five projects whose reference graph fits in a sentence; a SharedKernel of five citizens, frozen since Chapter 4 and audited once since; a Domain layer where seventeen contexts have their addresses and five of them have their citizens — entities that stay themselves, values that cannot be forged, aggregates that refuse by name, events under a self-enforcing statute, services that judge without touching; and an Application layer that can be asked, told, and taught through one pipeline, against ports the outside world has not yet signed. Fifty tests watch it, every invariant has been violated on purpose at least once, and the whole of it runs in the time a page takes to turn. Every claim in this paragraph is checkable against the Appendix-B skeleton, which is the only kind of claim the standing rule permits a closing chapter to make.

What does not exist, enumerated with owners

The boundary is drawn the way it was promised: by name, with the volume that picks each item up. **Nothing persists.** No database, no DbContext, no migration, no row — the 654 tables of *From Schema to System* remain paper, and *From Core to Contract* opens by making the fake repository's contract subclass pass against a real one. **Nothing listens.** No HTTP, no controller, no serialized DTO — the API layer, the problem-details mapping of every refusal code this book published, and the primitives-at-the-boundary translation are Volume II's middle act. **Nothing authenticates.** Every *OperatorId* in fifty tests was taken on faith; the Security context and its enforcement are Volume II's closing act. **Nothing schedules, logs, retries, or ships.** The shelf-expiry timer, the unhandled-exception policy, the outbox that closes Chapter 9's crack, structured logging, configuration, integration tests, mutation gates, the container, and the day the Phase-0 console finally calls its own backend — all Volume III, *From Contract to Container*. None of these absences is a gap in the sense a reviewer means the word. Each is a seam this volume shaped on purpose, and the proof of the shaping is that naming them takes a paragraph instead of an archaeology.

The seam, itemized for the next volume

Volume II inherits, in writing: three ports and three read models with their contracts in the XML documentation; an after-commit dispatch clause it must turn from documentation into code at the unit of work; a repository contract class awaiting its three-line adapter subclass; the failed-commit scar, as law; optimistic concurrency, owed to Chapter 8; the outbox, owed to Chapter 9; typed identifiers that must survive value conversion; and a statute's worth of published refusal codes that its API must carry to clients unedited. The reader inherits seven exercises — 11.1, 11.2, 12.1, 16.1, 17.1, 17.2, and 18.1 — each opened where its shape was already solved, with solutions in Appendix B.

The habits, which were the actual cargo

If the trilogy's remaining volumes were never written, the code would be the smaller loss. The cargo was the habits, and they travel without the solution: signatures that tell the truth; two channels of failure, never confused; refusals with published names; absences that are features when they are named; every invariant watched to fail; every promise ledgered and every ledger audited; and the calibration reflex — priced decisions, confessed constants, boundaries printed where they apply — that this series learned in *From Certainty to Calibration* and this volume tried to practice on every page, including the pages where its own earlier listings needed correcting in the open. A reader who forgets every class name but keeps the habits got the better half of the book.

The standing boundary, one last time

And because the epilogue of a NEXUS-1 volume is where the boundary is restated, not relaxed: this is a software architecture and learning companion. Nothing between these covers is safety-class software, certified operational guidance, or connected to any real facility. NEXUS-1 remains a Phase-0 demonstrator — a paper plant with a compiling heart — useful for learning architecture, language, modelling, and boundaries. Never for operating a plant.

The core is built, proven, and honestly bounded. The blueprint has become a beating in-vitro heart; what it needs next is blood — tables, wires, and the world. That is another book's promise, and for the first time in this series, the promise has a compiling foundation to be kept against.

From Blueprint to Core concludes here. The backend trilogy continues in *From Core to Contract: Persisting and Serving the NEXUS-1 Backend*.

— G.A., Edessa, 2026

APPENDIX A

Installation Guide

Everything this volume builds runs on the .NET SDK alone. No SQL Server, no Docker, no cloud account, no license key — the standing claim of the whole book, made operational: **clone, restore, build, test, green, in about a minute.** This appendix is that minute, written out.

A.1 · Install the SDK

Install the **.NET 8 SDK (LTS) or later** — not just the runtime, which cannot build. Windows and macOS: download the installer from Microsoft's .NET site and run it. Linux: use your distribution's package manager where available, or the official install script. Verify with the command below; any 8.x or later response is fine, and a response at all confirms the SDK, not just the runtime, is present.

```
$ dotnet --version
8.0.401
```

Verifying the SDK.

A.2 · Pick an editor — none of them is required

This book was written to be editor-agnostic; every listing is plain C# and every command is the CLI. Three reasonable choices, in no particular order: **Visual Studio** (Windows, full IDE, free Community edition); **Visual Studio Code** with the C# Dev Kit extension (cross-platform, lighter); **JetBrains Rider** (cross-platform, paid, popular for exactly this kind of solution). A text editor and the *dotnet* CLI shown below are sufficient for every command in this book; the editor only changes how comfortably you browse and step through the code.

A.3 · Restore, build, test

With the solution skeleton of Appendix B in hand (or your own clone, once one exists), four commands take you from checkout to green:

```
$ git clone <your-repo> nexus1-backend && cd nexus1-backend
$ dotnet restore
$ dotnet build -c Release
$ dotnet test

Passed! - Failed: 0, Passed: 50, Skipped: 0, Duration: 583 ms
```

The whole volume, verified in four commands.

restore downloads the three packages this volume actually uses (MediatR, FluentValidation, and their transitive dependencies — Appendix C has the full ledger); *build* compiles all five projects in dependency order, which is also the moment any forbidden reference (Chapter 2's CS0246) would surface; *test* runs both suites and prints the number that has been rising all book. If your count differs from fifty, you are either mid-chapter or have found a real bug — the exercises of Chapters 11–18 are the usual, honest reason for the former.

A.4 · What you will not need for this volume

Stated once, plainly, because Volume II changes it: no SQL Server, no connection string, no Docker container, no *appsettings.json*, no web server, and no internet access beyond the one-time package restore. Nexus.Domain.csproj holds zero PackageReferences; Nexus.Application adds exactly the three named above. If a command in this appendix asks for a database, you have skipped ahead to the wrong volume's quickstart.

APPENDIX D

Bounded Context, Namespace, Schema Map

One platform, seventeen bounded contexts, one schema atlas — *From Schema to System* named all seventeen and gave each a SQL Server schema before a line of this volume's C# existed. Chapter 3 mapped each context onto a **folder** inside *Nexus.Domain* and *Nexus.Application*, one namespace per sector, so that the code and the atlas would use the identical name for the identical thing. Table D.1 is that cross-reference made complete, with an honest fifth column: which contexts this specific volume actually populated, and where.

#	BOUNDED CONTEXT	NEXUS.DOMAIN NAMESPACE	SQL SCHEMA (VOL. II)	STATUS IN VOLUME I
01	Core Platform	`CorePlatform`	`CorePlatform`	Partial — IAggregateRoot, identifiers (Ch. 4, 12)
02	Security	`Security`	`Security`	Reserved — folder only
03	Organization	`Organization`	`Organization`	Reserved — folder only
04	Reactor Fleet	`ReactorFleet`	`ReactorFleet`	Reserved — folder only
05	Digital Twin	`DigitalTwin`	`DigitalTwin`	Reserved — folder only
06	Instrumentation	`Instrumentation`	`Instrumentation`	Built — EngineeringValue (Ch. 7)
07	Alarm Management	`AlarmManagement`	`AlarmManagement`	Built — AlarmEvent (Ch. 6, 9, 11)
08	Event Management	`EventManagement`	`EventManagement`	Reserved — folder only
09	Maintenance	`Maintenance`	`Maintenance`	Reserved — folder only
10	Root Cause	`RootCause`	`RootCause`	Built — flagship (Ch. 8, 9, 11)
11	Reinforcement Learning	`ReinforcementLearning`	`ReinforcementLearning`	Built — Policy, QTable (Ch. 11)
12	Robotics	`Robotics`	`Robotics`	Reserved — folder only
13	Radiation Monitoring	`RadiationMonitoring`	`RadiationMonitoring`	Reserved — folder only
14	Emergency Preparedness	`EmergencyPreparedness`	`EmergencyPreparedness`	Reserved — folder only
15	Compliance	`Compliance`	`Compliance`	Reserved — folder only
16	Reporting	`Reporting`	`Reporting`	Reserved — folder only
17	Audit	`Audit`	`Audit`	Reserved — folder only

Table D.1 — All seventeen sectors of the schema atlas, mapped to their Nexus.Domain namespace. Shaded rows are populated in this volume.

D.1 · Reading the fifth column honestly

“Reserved” means exactly one thing: the folder *src/Nexus.Domain/<Context>/* exists (Chapter 3), and nothing inside it does. This is not an oversight to apologize for — it is Chapter 11’s own thesis stated as an inventory: three very different shapes (lifecycle-heavy, graph-heavy, learning-heavy) were chosen to prove the bench suffices, and the remaining twelve are asserted to follow the same patterns rather than demonstrated to. A reader extending this solution into *Maintenance* or *Compliance* should expect to find the same five citizens from Part II waiting to be reused, not a different architecture.

D.2 · The Application layer’s narrower map

Nexus.Application mirrors only the contexts this volume gave a write or read side: two namespaces, not seventeen, each holding the shape Chapters 13–15 established.

`Nexus.Application.AlarmManagement.Commands`	*AcknowledgeAlarmCommand + handler (Ch. 14)*
`Nexus.Application.AlarmManagement.Queries`	*GetFloodStatusQuery + handler (Ch. 15)*
`Nexus.Application.RootCause.Commands`	*OpenCase, CloseCase, AttachEvidence (Ch. 14, 17)*
`Nexus.Application.RootCause.Queries`	*GetOpenCasesQuery + handler (Ch. 15)*
`Nexus.Application.Abstractions`	*Ports, messaging contracts, behaviors (Ch. 12, 13, 16)*
`Nexus.Application.Events`	*The event bridge and one reaction (Ch. 16)*

D.3 · Cross-reference to the wider series

The engines behind three of the five built contexts are documented at length elsewhere in the series and only translated into C# here: *From Grid to Core* grounds Instrumentation and Reactor Fleet; *From Flood to Cause* grounds Root Cause; *From Trial to Policy* grounds Reinforcement Learning. Where this volume’s prose says “the plant’s failure atlas” or “the learned policy,” those books are the physics and the mathematics underneath the aggregate; this table is the address book connecting the two literatures by namespace.

APPENDIX B

The Full Volume-I Solution Skeleton

Every elision in this book was marked at the moment it was made, on the promise that this appendix would make it whole. Three debts are paid here, in order: the **CausalGraph** construction elided from Listing 11.3, the **seven inventory repairs** Chapter 18 counted but did not print, and the **seven exercise solutions** opened across Chapters 11 through 18. Nothing here is new doctrine — every listing below applies a pattern the body of the book already established, which is itself the point: a reader who solved an exercise differently but by the same patterns solved it correctly.

B.1 · CausalGraph, construction restored (Listing 11.3)

The factory and *AddNode* follow Chapter 8's exact shape: a private constructor, a public static entry point, and a mutator reachable only through the root — CausalNode joins Evidence as a child entity that never leaves its cluster.

```
public sealed class CausalGraph : Entity<CausalGraphId>, IAggregateRoot
{
    private readonly Dictionary<CauseNodeId, CauseNode> _nodes = [];

    private CausalGraph(CausalGraphId id, UnitId unitId) : base(id)
        => UnitId = unitId;

    public UnitId UnitId { get; }

    public static CausalGraph Chart(UnitId unitId) =>
        new(CausalGraphId.New(), unitId);

    public CauseNodeId AddNode(
        string name, double prior, CauseNodeId? parent, TagId? signal)
    {
        var id = CauseNodeId.New();
        _nodes[id] = new CauseNode(id, name, prior, parent, signal);
        return id;
    }

    // DepthOf and CandidatesFor: printed in full at Listing 11.3
}
```

CausalGraph.cs, the elided half: construction and mutation, restored to Listing 11.3.

AddNode returns the new node's ID rather than *Result* — the same doctrine as Chapter 11's *Learn* and *OpenCaseHandler*: nothing about charting a known failure mode can fail, so there is no refusal to carry. It is not called through a command in this volume because no use case needs to edit the graph yet; the graph is seeded once, by whoever configures the plant, and Volume II gives that act a proper door.

B.2 · The seven remaining inventory repairs (Chapter 18, Table 18.1)

Chapter 18 printed three repairs and counted ten; the other seven follow the identical two-assert shape — refusal by published name, nothing half-applied — and are restored here in full, grouped by context.

```
public sealed class ShelvingInventoryRepairs
{
    private static readonly DateTime T0 = new(2026, 7, 2, 8, 0, 0, DateTimeKind.Utc);

    [Fact]
    public void An_unacknowledged_alarm_refuses_to_shelve()
    {
        var alarm = AlarmEvent.Raise(UnitId.New(), TagId.New(),
            AlarmSeverity.Warning, "drift", T0);

        var shelved = alarm.Shelve(OperatorId.New(), T0.AddHours(8), T0.AddMinutes(1));

        Assert.Equal("AlarmManagement.OnlyAcknowledgedShelves", shelved.Error.Code);
        Assert.Equal(AlarmStatus.Raised, alarm.Status);          // nothing half-applied
    }

    [Fact]
    public void A_shelf_that_ends_in_the_past_is_refused()
    {
        var alarm = AlarmEvent.Raise(UnitId.New(), TagId.New(),
            AlarmSeverity.Warning, "drift", T0);
        alarm.Acknowledge(OperatorId.New(), T0.AddMinutes(1));

        var shelved = alarm.Shelve(OperatorId.New(),
            untilUtc: T0.AddMinutes(1), atUtc: T0.AddHours(1));   // until <= at

        Assert.Equal("AlarmManagement.ShelfMustEnd", shelved.Error.Code);
        Assert.Null(alarm.ShelvedUntilUtc);
    }

    [Fact]
    public void ReAlarm_on_an_already_active_alarm_is_refused()
    {
        var alarm = AlarmEvent.Raise(UnitId.New(), TagId.New(),
            AlarmSeverity.Warning, "drift", T0);       // still Raised, not Shelved/Cleared

        var reAlarmed = alarm.ReAlarm(T0.AddMinutes(5));

        Assert.Equal("AlarmManagement.NothingToReAlarm", reAlarmed.Error.Code);
    }
}
```

InventoryRepairs.cs, continued (1 of 3): AlarmManagement's three unwatched shelving refusals.

```
public sealed class EvidenceInventoryRepair
{
    private static readonly DateTime T0 = new(2026, 7, 2, 8, 0, 0, DateTimeKind.Utc);

    [Fact]
    public void Striking_evidence_that_does_not_exist_is_refused()
    {
        var rcc = RootCauseCase.Open(UnitId.New(), AlarmEventId.New(), T0);
        rcc.AttachEvidence("the only clue", TagId.New(), T0);

        var strike = rcc.StrikeEvidence(
            new EvidenceId(Guid.NewGuid()),   // an id that was never attached
            "typo");

        Assert.Equal("RootCause.NoSuchEvidence", strike.Error.Code);
        Assert.False(rcc.Evidence[0].Struck);         // the real clue is untouched
    }
}
```

InventoryRepairs.cs, continued (2 of 3): RootCause's one remaining unwatched refusal.

The same code also refuses a *second* strike against evidence already struck — *StrikeEvidence*'s lookup filters on *!e.Struck*, so a repeated call finds nothing to strike and answers with the identical *NoSuchEvidence* code. One test, two crimes, because the guard that prevents both is the same line.

```
public sealed class GuardedFactoryInventoryRepairs
{
    [Fact]
    public void A_learning_rate_outside_zero_to_one_is_refused()
    {
        var rate = LearningRate.Create(1.5);          // alpha must be in (0, 1]

        Assert.Equal("ReinforcementLearning.InvalidLearningRate", rate.Error.Code);
    }

    [Fact]
    public void Flood_thresholds_with_a_non_positive_window_are_refused()
    {
        var thresholds = FloodThresholds.Create(10, TimeSpan.Zero);

        Assert.Equal("AlarmManagement.InvalidThresholds", thresholds.Error.Code);
    }

    [Fact]
    public void A_cause_candidate_with_a_blank_name_is_refused()
    {
        var candidate = CauseCandidate.Create(, prior: 0.5, support: 1);

        Assert.Equal("RootCause.InvalidCandidate", candidate.Error.Code);
    }
}

$ dotnet test tests/Nexus.Domain.Tests
Passed! - Failed: 0, Passed: 43, Skipped: 0, Duration: 141 ms
```

InventoryRepairs.cs, continued (3 of 3): ReinforcementLearning's three guarded-factory refusals.

Forty-three, not fifty: this run is the Domain suite alone, with all ten inventory debts paid and no Application-layer tests loaded. The fifty of Chapter 19's final count includes the Application suite's fourteen — the two numbers are not in tension; they are two different scopes, exactly as Table 19.1 recorded.

B.3 · Exercise solutions

Seven exercises were opened across the body of the book, each at a point where the surrounding pattern already contained its answer. What follows is one solution per exercise — not the only correct one, but one that follows the chapter’s own doctrine without inventing anything new.

Exercise 11.1 — curing Shelving's primitive obsession

Chapter 11 left *ShelvedBy* and *ShelvedUntilUtc* as two nullable primitives — Chapter 7's smell, resurfaced on purpose. The cure is Chapter 7's own pattern: one guarded value object replacing the pair, plus the missing *Unshelve*.

```
public sealed record Shelving
{
    private Shelving(OperatorId by, DateTime untilUtc)
    { By = by; UntilUtc = untilUtc; }

    public OperatorId By { get; }
    public DateTime UntilUtc { get; }

    public static Result<Shelving> Create(OperatorId by, DateTime untilUtc, DateTime atUtc) =>
        untilUtc > atUtc
            ? Result.Success(new Shelving(by, untilUtc))
            : Result.Failure<Shelving>(AlarmErrors.ShelfMustEnd);
}

// AlarmEvent.cs -- the pair becomes one nullable value object:
public Shelving? Shelf { get; private set; }             // was: ShelvedBy + ShelvedUntilUtc

public Result Shelve(OperatorId by, DateTime untilUtc, DateTime atUtc)
{
    if (Status != AlarmStatus.Acknowledged)
        return Result.Failure(AlarmErrors.OnlyAcknowledgedShelves);

    var shelf = Shelving.Create(by, untilUtc, atUtc);
    if (shelf.IsFailure) return Result.Failure(shelf.Error);

    Status = AlarmStatus.Shelved;
    Shelf = shelf.Value;
    Raise(new AlarmShelvedEvent(Guid.NewGuid(), atUtc, Id, by, untilUtc));
    return Result.Success();
}

public Result Unshelve(DateTime atUtc)                    // the missing door
{
    if (Status != AlarmStatus.Shelved)
        return Result.Failure(AlarmErrors.NothingToUnshelve);

    Status = AlarmStatus.Acknowledged;         // returns to the state Shelve left from
    Shelf = null;
    return Result.Success();
}
```

Shelving.cs (new) and AlarmEvent.cs (revised): one value object, one new door.

Two consequences worth noting. The guard that used to live as a bare *if* inside *Shelve* now lives inside *Shelving.Create* — the value object owns its own validity, exactly as Chapter 7 argued, and *Shelve* becomes a courier for the value object's verdict. And *Unshelve* returns to *Acknowledged*, not *Raised*: unshelving is an operator's deliberate un-suppression, not a recurrence, and only *ReAlarm* — the condition returning on its own — earns the harsher reset that clears the acknowledgement.

Exercise 11.2 — a real evaluation gate

The boundary box confessed a demo constant: any recorded score permits activation. A real deployment demands a floor, and the floor belongs where *NotEvaluated* already lives — on the policy itself, as a named minimum rather than a magic number buried in a handler.

```
public static class RlErrors
{
    // ...NotDraft, NotEvaluated as printed at Listing 11.6, plus:
    public static readonly Error BelowEvaluationFloor = new(
        "ReinforcementLearning.BelowEvaluationFloor",
        "A policy must score at or above the deployment floor to activate.");
}

public sealed class Policy : Entity<PolicyId>
{
    public const double EvaluationFloor = 0.75;         // named, not magic

    // ...Draft, RecordEvaluation unchanged from Listing 11.6...

    public Result Activate(DateTime atUtc)
    {
        if (Status != PolicyStatus.Evaluated)
            return Result.Failure(RlErrors.NotEvaluated);
        if (EvaluationScore < EvaluationFloor)
            return Result.Failure(RlErrors.BelowEvaluationFloor);   // the new gate

        Status = PolicyStatus.Active;
        Raise(new PolicyActivatedEvent(Guid.NewGuid(), atUtc, Id, UnitId));
        return Result.Success();
    }
}
```

Policy.cs, revised: a floor, and a second named refusal beside NotEvaluated.

The rhyme survives the extension: two refusals now stand at the activation gate instead of one, and both are named, both are watched-to-fail candidates, and neither is a status comparison smuggled into a handler — the floor is exactly as much the aggregate’s business as the status check beside it.

Exercise 12.1 — patrolling the marker by reflection

The boundary box noted that *IAggregateRoot* is structure, not semantics — nothing stops a future developer from tagging *Evidence*. Chapter 9's statute test is the precedent: a reflective patrol that makes the convention self-enforcing.

```
public sealed class AggregateRootStatuteTests
{
    [Fact]
    public void No_child_entity_is_tagged_as_an_aggregate_root()
    {
        // child entities: types held by a collection property on a root
        var roots = typeof(AlarmEvent).Assembly.GetTypes()
            .Where(t => typeof(IAggregateRoot).IsAssignableFrom(t));

        var childTypes = typeof(AlarmEvent).Assembly.GetTypes()
            .SelectMany(t => t.GetProperties())
            .Select(p => p.PropertyType)
            .Where(t => t.IsGenericType &&
                        typeof(IEnumerable<>).IsAssignableFrom(t.GetGenericTypeDefinition()))
            .SelectMany(t => t.GetGenericArguments());

        Assert.Empty(childTypes.Intersect(roots));   // Evidence, CauseNode stay untagged
    }
}
```

AggregateRootStatuteTests.cs: the marker, audited the way events are.

Exercise 16.1 — carrying every validation failure, not just the first

The behavior surfaces one error where a form-driven client wants all of them. The widening is additive — the plain *Error* record and every existing *Result.Failure(someError)* call keep compiling — because the new shape is a subtype, not a replacement.

```
namespace Nexus.Application.Abstractions;

public sealed record ValidationError(IReadOnlyList<Error> Failures)
    : Error("Validation.Multiple", $"{Failures.Count} validation failure(s).");
// inside ValidationBehavior<TRequest,TResponse>.Handle, replacing FirstOrDefault:
var failures = validators
    .Select(v => v.Validate(request))
    .SelectMany(r => r.Errors)
    .Select(f => new Error($"Validation.{f.PropertyName}", f.ErrorMessage))
    .ToList();

if (failures.Count == 0) return await next();

return Failure(failures.Count == 1
    ? failures[0]
    : new ValidationError(failures));    // one code, many messages, still a Result
```

ValidationError.cs (new) and ValidationBehavior.cs, revised: one code, many messages.

A client switching on *result.Error.Code* still gets *Validation.CaseId.Value* for the single-failure case — Listing 16.5's assertion is untouched — and only a client that specifically inspects for *ValidationError* sees the full list. Widening without breaking is the whole exercise.

Exercise 17.1 — StrikeEvidenceCommand, by substitution

The scenario named this exercise exactly: produce it from Listing 17.1 by substituting nouns and one call. The domain door — *RootCauseCase.StrikeEvidence* — already proved its refusal logic at Listing 11.7; the command is pure carriage.

```
namespace Nexus.Application.RootCause.Commands;

public sealed record StrikeEvidenceCommand(
    RootCauseCaseId CaseId, EvidenceId EvidenceId, string Reason) : ICommand;

public sealed class StrikeEvidenceHandler(
    IRepository<RootCauseCase, RootCauseCaseId> cases,
    IUnitOfWork uow) : ICommandHandler<StrikeEvidenceCommand>
{
    public async Task<Result> Handle(StrikeEvidenceCommand cmd, CancellationToken ct)
    {
        var rcc = await cases.GetAsync(cmd.CaseId, ct);
        if (rcc is null)
            return Result.Failure(
                ApplicationErrors.NotFound("RootCauseCase", cmd.CaseId.Value));

        var struck = rcc.StrikeEvidence(cmd.EvidenceId, cmd.Reason);
        if (struck.IsFailure) return struck;

        await uow.CommitAsync(ct);
        return Result.Success();
    }
}
```

StrikeEvidenceCommand.cs, complete — Listing 17.1 with three names changed and one call swapped.

Exercise 17.2 — giving RankedCause an identity

The scenario held a *CauseNodeId* only because it happened to keep one from seeding — a workaround wearing its exercise number. Widening the shape removes the workaround's reason to exist.

```
public sealed record RankedCause
{
    internal RankedCause(CauseNodeId causeId, string cause, double score)
    { CauseId = causeId; Cause = cause; Score = score; }

    public CauseNodeId CauseId { get; }         // new: the ID a client can act on
    public string Cause { get; }                // unchanged: the name a screen prints
    public double Score { get; }
}

// CauseCandidate grows one field to carry the ID forward through Rank():
public sealed record CauseCandidate
{
    // ...Cause, PriorWeight, SupportingEvidence as printed at Listing 10.2, plus:
    public CauseNodeId CauseId { get; }
}

// CausalGraph.CandidatesFor now threads n.Id through, and CloseCase
// in Chapter 17 reads ranked[0].CauseId instead of a kept seeding variable.
```

CausalRankingService.cs, revised: the display name keeps company with the identity that names it.

Every caller from Chapter 10 forward that used only *Cause* and *Score* keeps compiling unchanged — the widening adds a field rather than renaming one, the same discipline Exercise 16.1 practiced a page earlier.

Exercise 18.1 — a property-based sketch for the ranking and the table

Chapter 18 named two properties worth testing across generated inputs rather than fixed examples: the ranking's order-independence, and the Q-table's monotone approach to a fixed target. A minimal, framework-agnostic sketch — no FsCheck dependency required to see the shape:

```
public sealed class RankingPropertyTests
{
    [Theory]
    [MemberData(nameof(RandomPermutations))]           // 100 shuffles, one seed logged
    public void Ranking_order_is_independent_of_input_order(
        IReadOnlyList<CauseCandidate> shuffled, IReadOnlyList<CauseCandidate> canonical)
    {
        var svc = new CausalRankingService();
        Assert.Equal(svc.Rank(canonical).Value.Select(r => r.Cause),
                     svc.Rank(shuffled).Value.Select(r => r.Cause));
    }
}

public sealed class QTableConvergencePropertyTests
{
    [Theory]
    [MemberData(nameof(RandomAlphasAndTargets))]       // 100 (alpha, target) pairs
    public void Repeated_learning_never_overshoots_the_target(
        double alpha, double target)
    {
        var q = QTable.StartLearning(UnitId.New());
        var sa = new StateAction("S", "A");
        var previous = 0.0;

        for (var i = 0; i < 20; i++)
        {
            q.Learn(sa, target, LearningRate.Create(alpha).Value);
            var current = q.ValueOf(sa);
            Assert.True(Math.Abs(current - target) <= Math.Abs(previous - target));
            previous = current;
        }
    }
}
```

PropertySketches.cs: the two properties, generated rather than fixed.

Both properties are restatements of claims the chapter already made in prose — determinism, convergence — now checked against a hundred generated cases instead of one. Adopting a real property-testing library (FsCheck, or xUnit's own *Theory* data generators at scale) is the natural next step; the sketch above is the property, not yet the library.

B.4 · The skeleton, in full

With B.1–B.3 restored, every listing this volume ever elided now exists somewhere on paper. The solution tree itself is Figure D3.1 plus this appendix's contents: five projects, seventeen context folders (five populated, twelve reserved per Appendix D), fifty tests in the main line, and eight more once every exercise solution above is wired into the suites. A reader who has typed every listing in the body of the book and every listing in this appendix has, at this point, built the whole of Volume I twice — once by reading, once by hand — and the second build is the one that teaches.

One honest note to close on: the exercise solutions above are graded by the same standard as the body's own listings — they compile against the patterns shown, but none has been mechanically re-verified against a live build the way every numbered listing in the chapters was. Treat them as a second opinion, not a second source of truth; if your own solution disagrees with one above but agrees with the chapter's doctrine, trust your solution.

APPENDIX C

Tools and Packages

Three packages, and the reasoning that earned each one a place inside *Nexus.Application.csproj*. *Nexus.Domain.csproj* is not listed below because it has nothing to list: zero PackageReferences, held from Chapter 3 to the volume's last page.

MediatR

`v12.x`

In-process request dispatch + pipeline behaviors

Tried in Ch. 13: the dispatcher (18 lines) was never the purchase; the pipeline was. Verdict: stays, exit priced. Licensing changed in 2025 — the contingency named.

FluentValidation

`v11.x`

Message-shape validators, assembly-scanned

One job only: is this request well-formed? Never restates a domain rule (Ch. 16's hard line). No dependency on MediatR; the pipeline behavior is the glue, not the library.

xUnit

`v2.x`

The test runner for both suites

Chosen for [Theory]/[InlineData] fit with the naming-as-specification discipline (Ch. 18); no feature of this book depends on xUnit specifically over its peers.

C.1 · What is deliberately absent

The shortest honest list in the book. **No mocking library** — Deep Dive 19 explained why: this volume's doubles verify outcomes, never conversations, and named fakes cost less than a mocking framework's ceremony for three small ports. **No ORM, no database driver** — the standing thesis of the whole volume; Volume II opens with EF Core against the schema atlas. **No logging framework** — a logging pipeline behavior is named as owed to Volume III (Chapter 16's boundary box) and arrives with it, not before. **No coverage or mutation-testing tool** — Chapter 18 declined both on principle, not budget: coverage measures lines, this book counts rules.

C.2 · The full restore, for reference

```
<!-- src/Nexus.Application/Nexus.Application.csproj -->
<ItemGroup>
  <PackageReference Include="MediatR" Version="12.4.1" />
  <PackageReference Include="FluentValidation.DependencyInjectionExtensions"
                    Version="11.10.0" />
</ItemGroup>

<!-- tests/*.Tests/*.csproj -->
<ItemGroup>
  <PackageReference Include="xunit" Version="2.9.0" />
  <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.11.1" />
</ItemGroup>

<!-- src/Nexus.Domain/Nexus.Domain.csproj -->
<ItemGroup>
  <!-- deliberately empty -- Chapter 3's commitment, kept to the last page -->
</ItemGroup>
```

Everything Nexus.Application.csproj asks for; Nexus.Domain.csproj asks for nothing.

Five lines of real dependency for an entire orchestration layer, and zero for the domain that layer serves. If Appendix C fits on two pages, the architecture did its job.

APPENDIX E

Sources and Reference Notes

This is not a theoretical software-architecture research work. It is a practical companion, and the sources below split into two honest categories: the wider NEXUS-1 series, which supplies the concrete domain material every listing in this volume translates into C#, and the external literature, which supplies the vocabulary and patterns this volume applies rather than invents.

E.1 · The NEXUS-1 companion series

From Domain to Twin
DDD from Zero, applied to NEXUS-1
The seventeen bounded contexts, the ubiquitous language, and the entities, value objects, and aggregates this volume translates into C#, class by class.

From Grid to Core
The plant physics and model

Grounds ReactorFleet and Instrumentation; the meaning behind every EngineeringValue this volume guards.

From Flood to Cause
The root-cause engine
Source of the RootCauseCase aggregate, its flagship invariant, and the causal-graph ranking this volume implements as a domain service.

From Trial to Policy
The reinforcement-learning engine

Source of the Policy and QTable aggregates and the custody chain Chapter 11 builds from nothing.

From Table to Twin
The data-design journey
Why a bounded context can live as a SQL schema, and why lookup tables and audit trails are not decoration — the argument Appendix D's map assumes.

From Schema to System
The 17-sector, 654-table schema atlas

The persistence target this volume's abstractions are shaped to meet, without ever opening a connection to it.

From Entity to Context
The EF Core configuration atlas
Consumed in Volume II; referenced here so every entity is mapping-ready — owned types, key shapes, lookup discipline — before a DbContext exists.

From Certainty to Calibration
The series retrospective
Source of the Honest Boundary convention and of the decision to split one overreaching book into a calibrated trilogy — this volume's own structure applies that lesson.

E.2 · External literature

Applied, not summarized: each entry below is cited for one specific pattern this volume uses, not as general background.

Eric Evans — *Domain-Driven Design: Tackling Complexity in the Heart of Software*
Addison-Wesley, 2003
Aggregates, bounded contexts, ubiquitous language — Part II's foundation.

Vaughn Vernon — *Implementing Domain-Driven Design*
Addison-Wesley, 2013
The practical aggregate-design rules Chapter 8 applies (small clusters, eventual consistency between them, IDs across boundaries).

Robert C. Martin — *Clean Architecture: A Craftsman's Guide to Software Structure and Design*
Prentice Hall, 2017
The Dependency Rule and the concentric-layer model Chapters 2–3 adapt as the Onion.

Martin Fowler — *Patterns of Enterprise Application Architecture*
Addison-Wesley, 2002
Repository, Unit of Work, and Data Transfer Object — named in this book almost unchanged from Fowler's own definitions.

Jimmy Bogard — *MediatR documentation and design notes*
github.com/jbogard/MediatR
The pipeline-behavior mechanism Chapter 16 builds on; the 2025 licensing change Chapter 13's trial weighs.

Microsoft — *.NET and C# documentation — records, nullable reference types, async/await*
learn.microsoft.com/dotnet/
The language mechanics Deep Dives 4 and 12 explain in plain language; version-specific, consult the current edition.

> **SOURCE BOUNDARY**
>
> External sources explain concepts applied here in simplified, teaching form; NEXUS-1 companion volumes supply demonstrator domain material. None of these sources turns this book's compiling core into certified operational guidance or safety-class software.

APPENDIX F

Glossary

Practical definitions, tied to this volume’s usage rather than the general literature. Where a term has a longer treatment elsewhere in the book, the entry says where.

A

Aggregate

A cluster of entities and value objects that must change together to stay correct, changed only through one door — its root. RootCauseCase and its Evidence children are this book’s flagship example (Ch. 8).

Aggregate root

The one entity in a cluster that outside code may reference and mutate through. Marked in this volume by IAggregateRoot, a repository-traffic marker rather than a domain concept (Ch. 12).

Assembly

The compiled output of one .csproj — a single .dll. A project can only use types from assemblies it references, which is what makes the Dependency Rule mechanically enforceable (Deep Dive 3).

Async / await

Task marks a promise of a future value; await redeems it without blocking the thread. Ports are async from birth so Volume II's genuinely slow adapters need no signature surgery (Deep Dive 12).

B

Bounded context

A region of a system where a word has exactly one meaning. NEXUS-1 has seventeen; this volume populates five (Ch. 1, Appendix D).

Builder (test-data)

A helper that assembles legal aggregate states through the domain's own public doors — never by reflection or private-setter tricks (Ch. 18, CaseBuilder).

C

CancellationToken

A signal threaded through every async call meaning “if I give up, you may stop.” Present in every port signature from Chapter 12 forward.

Clean/Onion Architecture

A layered model where dependencies point inward toward policy and outward toward detail is forbidden. This volume builds its two innermost rings, Domain and Application (Ch. 2).

Command

A message expressing intent to change state, which may be refused. Answers through Result or Result, never a raw value (Ch. 13–14).

Composition (UML)

The filled-diamond relationship where a part's lifecycle belongs entirely to its whole. A DDD aggregate is drawn with UML composition (Deep Dive 8).

Consistency boundary

The set of objects one transaction must keep correct together — in this book, exactly one aggregate per business act (Ch. 8).

CQRS

Command Query Responsibility Segregation: separate models for writing and reading. Practiced here as two code paths over one store, not two systems (Ch. 13).

D

Dependency injection

Composing an object graph by handing dependencies to constructors rather than letting objects construct or locate their own. Confined to one composition root (Ch. 5).

Dependency Rule

Source-code dependencies may only point inward, toward higher-level policy, never outward toward detail (Ch. 2).

Domain event

An immutable, past-tense record of a fact that already happened, dispatched after commit and consumed by any subscriber holding the published type (Ch. 9).

Domain layer

The innermost ring: entities, value objects, aggregates, domain events, and domain services. Zero PackageReferences, held for the whole volume (Ch. 6–11).

Domain service

An operation that is domain knowledge but native to no single entity or value object — stateless, domain-pure, named as a verb in the ubiquitous language (Ch. 10).

DTO (data transfer object)

A flat, behaviorless record shaped for one screen's appetite, never for a write. The read side's only citizen besides the port and the handler (Ch. 15).

E

Entity

An object defined by identity that persists through change, rather than by its current attribute values. Every entity descends from Entity (Ch. 4, 6).

Error (kernel type)

A named refusal: a stable code plus a human message, carried by Result. The published vocabulary every layer, test, and future API relays verbatim (Ch. 4).

F

Fake (test double)

A real, simplified implementation of a port — this volume's sole kind of test double, verifying outcomes rather than conversations (Ch. 5, Deep Dive 19).

G

Guard clause

An early return or throw that rejects an illegal input or state before the rest of a method runs. Ensure's guards are this kernel's guard clauses (Ch. 4).

H

Handler

The class that receives one command or query, orchestrates ports and aggregates, and decides nothing itself (Ch. 14–15).

Honest fakery

The discipline that a test double may simulate exactly what its port promises and nothing more — never smarter than its contract (Ch. 19).

I

Invariant

A statement about an aggregate's state that must hold at every observable moment, not merely at input. Enforced only where every mutation passes: the root (Deep Dive 8).

M

MediatR

The in-process request-dispatch library this volume retains for its pipeline behaviors, with the exit priced (Ch. 13, Appendix C).

N

Namespace

An address, not a wall — crosses assembly boundaries freely and enforces nothing on its own; contrast with assembly (Deep Dive 3).

O

Onion Architecture

See Clean/Onion Architecture.

P

Pipeline behavior

An open-generic wrapper that runs before or after every handler, implementing a cross-cutting concern once for every message type, present and future (Ch. 16).

Port

A consumer-owned interface naming what the core needs from the outside world, signed later by an adapter it does not know exists (Ch. 12).

Projection

Casting a rich, guarded aggregate into a flat, disposable shape for reading — the read side's core verb (Ch. 15).

Published language

The stable, deliberately public vocabulary a context offers others — in this book, its Events/ folder and its Error codes (Ch. 9, 14).

Q

Query

A message asking a question, answered in a DTO, never mutating state and never refused by a business rule (Ch. 13, 15).

R

Read model

An Application-owned port that answers a query in DTOs, bypassing repositories and aggregates entirely (Ch. 15).

Repository

A port trafficking only in aggregate roots — Get, Add, Remove; no Update, no queries, no commit of its own (Ch. 12).

Result / Result

The kernel type through which every business failure travels; exceptions are reserved for programmer errors (Ch. 4).

S

Scenario test

A broad, narrative test proving pieces already pinned elsewhere compose correctly — trusts more than it checks, on purpose (Ch. 17, Deep Dive 17).

SharedKernel

The one shared project beneath Domain and Application, frozen at five citizens after Chapter 4 for the rest of the trilogy.

Solution (.sln)

A binder holding projects together for tooling; contains no code of its own (Deep Dive 3).

Statute test

A reflective test that patrols a structural convention mechanically — the event anatomy law (Ch. 9) and, in Appendix B, the aggregate-root marker.

T

Test pyramid

The shape of a healthy suite: many cheap piece tests at the base, few broad scenario tests at the top (Deep Dive 17).

Two-channel doctrine

Business failures travel as Result values; programmer errors travel as exceptions. The two channels are never confused (Ch. 4).

U

Ubiquitous language

The shared vocabulary between domain experts and code, enforced by naming classes, methods, and errors after the words experts actually use.

Unit of Work

The port committing exactly one business act's transaction and, per its contract, dispatching that act's domain events afterward (Ch. 12).

V

Validator

A FluentValidation class checking a message's shape — well-formedness — and never restating a rule an aggregate already owns (Ch. 16).

Value object

An immutable object defined by its attributes rather than an identity; equal instances are interchangeable (Ch. 7).

W

Watched-to-fail

The discipline that every invariant is proven by a test that commits its exact forbidden act and asserts the named refusal (Ch. 4, formalized Ch. 18).

APPENDIX G

Index

Selected terms and their primary discussions across nineteen chapters, nineteen Deep Dives, and seven appendices. Page numbers point to substantive treatment, not every passing mention.

A

B

C

D

E

F

G

H

I

M

N

P

Q

R

S

T

U

V

W